REAL-TIME OBJECT OVERLAP DETECTION IN COMPLEX SCENES USING YOLOv8 FOR LOCALIZATION AND CBAM FOR FEATURE ENHANCEMENT

A Thesis Presented to the Faculty, College of Computer Studies
Laguna State Polytechnic University
Los Baños Campus
Los Baños, Laguna

In partial fulfillment of the requirement for the degree
Bachelor of Science in Computer Science
Specialized in *Intelligent Systems*

JOHN MARTIN L. LAGAJINO
JOSHUA R. CASPILLO
ARGIE GARCIA

GENE MARCK B. CATEDRILLA

June 2026

APPROVAL SHEET

The thesis entitled "**REAL-TIME OBJECT OVERLAP DETECTION IN COMPLEX SCENES USING YOLOv8 FOR LOCALIZATION AND CBAM FOR FEATURE ENHANCEMENT**" prepared and submitted by **Joshua R. Caspillo**, **Argie Garcia**, and **John Martin L. Lagajino** in partial fulfillment of the requirements for the degree, Bachelor of Science in Computer Science Specialized in Intelligent System, is hereby recommended for approval and acceptance.

GENE MARCK B. CATEDRILLA
Adviser

Approved and accepted by the Committee on Oral Examination with a grade of _____.

MARIBEL V. CARDONA
Member

CAROLINA R. JOVAL
Member

MYRA V. PAICAGLINO
Member

JEFFERSON L. LERIOS
Member

Accepted as partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science, specialized in Intelligent Systems.

LOYD S. ECHALAR, DIT
Associate Dean, College of Computer Studies

OFELIA B. MANINGAS, Ed. D.
Chairperson, Research and Development Office

Date Signed

RESEARCH CONTRIBUTION NO.	

ACKNOWLEDGEMENT

The successful completion of this Bachelor's Degree in Computer Science, culminating in this thesis, has been significantly supported by numerous individuals whose encouragement, guidance, and unwavering belief to their abilities were indispensable throughout this academic journey.

Their success of this thesis project would not have been possible without the invaluable contributions, insights, and support of the following individuals, to whom they extend their deepest gratitude:

To the esteemed University President, Dr. Mario R. Briones, and the OIC Campus Director, Engr. Manuel Luis R. Alvarez, for their excellent leadership and conducive academic environment they foster at their university.

To their beloved College of Computer Studies Associate Dean, Dr. Loyd S. Echalar, for his warm advice and consistent encouragement.

To their dedicated mentor, Mr. Gene Marck B. Catedrilla, for his invaluable guidance, unwavering support, and insightful contribution throughout their journey with this project.

To their Research Implementing Unit Head, Mr. Jefferson L. Lerios, and the Approval Committee members, Ms. Maribel V. Cardona, Ms. Carolina R. Joval, and Ms. Myra V. Paicaglino, for their invaluable guidance and support in completing this project.

Finally, their heartfelt thanks to Almighty God for his provision of their needs, protection, wisdom, knowledge, and guidance.

THE RESEARCHERS

DEDICATION

The success of this project is respectfully and wholeheartedly dedicated by the researchers to our Almighty God, who gave them knowledge, protection, and guidance. To their families:

Mrs. Belinda R. Caspillo

Mrs. Diana Jane V. Garcia

Mrs. Marilou L. Lagajino

For their boundless love, unwavering support, and immense sacrifices, both financial and moral, were the foundation of their perseverance and dedication to work hard on the project.

To their mentors whose insightful guidance, profound wisdom, and continuous encouragement were invaluable at every stage of this study. Their expertise significantly shaped the development and successful completion of this study.

To their friends, partners, and siblings, who provided encouragement during times of hardship and difficulty, reminding them to persist.

A.G

J.R.C

J.M.L.L

ABSTRACT

This study, titled "*Real-Time Object Overlap Detection in Complex Scenes Using YOLOv8 for Localization and CBAM for Feature Enhancement*," introduces an improved YOLOv8 architecture designed for accurate real-time object detection under overlapping and visually complex conditions. The approach incorporates the Convolutional Block Attention Module (CBAM) into key locations within the YOLOv8 backbone and neck. Multiple placements were evaluated across YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x variants. Models were assessed using precision, recall, F1-score, and mean Average Precision (mAP), while statistical significance was validated through paired-sample t-tests. Based on the experimental results, the YOLOv8+CBAM model consistently outperformed the baseline YOLOv8. Specifically, the integration of CBAM in the neck-before-fusion yielded measurable gains across metrics: an average +0.014 increase in F1-score, +0.0218 improvement in mAP@50 (~2.4% relative increase), and +0.0254 improvement in mAP@50-95 (~4.4%). All differences were statistically significant, with $p < 0.05$ across all evaluated metrics. The experiments were conducted using a dataset of 1,000 annotated poultry images captured under challenging lighting and overlapping scenarios, and results confirm that attention-based enhancement improves YOLOv8's localization accuracy without compromising inference speed. The enhanced architecture remains lightweight and efficient, making it suitable for deployment in low-power agricultural systems and enabling practical real-time monitoring for precision poultry farming applications.

Keywords: *YOLOv8, CBAM, object detection, real-time localization, attention mechanism, deep learning, convolutional neural network (CNN*

TABLE OF CONTENTS

Preliminaries	*Page*
Title Page	i
Approval Sheet	ii
Acknowledgement	iii
Dedication	iv
Abstract	v
Table of Contents	vi
List of Figures	vii
List of Tables	viii
Definition of Terms	ix

CHAPTER

I	**INTRODUCTION**	
	Background of the Study	1
	Objectives	3
	Significance of the Study	4
	Scope and Limitations	8
II	**REVIEW OF RELATED LITERATURE AND STUDIES**	
	Related Literature	9
	Literature Matrix	25
	Synthesis and Relevance of the Study	29
	Conceptual Framework	32
III	**METHODOLOGY**	
	Materials	35
	Software	35
	Hardware	36
	Research Design	37
	Model Development	40
	Data Acquisition	43
	Data Preprocessing	43
	Data Annotation	44
	Model Evaluation	53
	Ethical Consideration	56
IV	**RESULTS AND DISCUSSIONS**	
	Exploratory Data Analysis	57
	Observation and Challenges	59
	Implications for Model Training	60
	CBAM Integration	60

Model Validation 82

V SUMMARY, CONCLUSIONS AND RECOMMENDATIONS

Summary 90
Conclusions 91
Recommendations 95

REFERENCES 98

APPENDICES

Appendix A. Letter of Approval
Appendix B. Certificate of Acceptance
Appendix C. Relevant Source Code
Appendix D. Evaluation Tool
Appendix E. Sample Input / Output
Appendix F. Comment Section
Appendix G. Turnitin and Grammarly Results
Appendix H. User's Guide
Appendix I. Publishable Paper (IEEE format)
Appendix H. Curriculum Vitae

LIST OF FIGURES

No.	Title	Page
1	Taxonomy of YOLO Versions, Enhancements, and Application Domains (2020–2025)	24
2	Conceptual Framework	32
3	YOLOv8+CBAM Model Workflow	39
4	Model Development	40
5	Training Pipeline of YOLOv8+CBAM	41
6	YOLOv8 Architecture	48
7	CBAM Overview	49
8	YOLOv8 w/CBAM Overview	50
9	Visualization of Data Samples	57
10	Roboflow Annotation Results Summary	58
11	YOLOv8+CBAM Architecture	61
12	Debugging YOLOv8+CBAM	62
13	CBAM Integration F1-Score Results and Improvements	64
14	Performance Evaluation of Model Based on mAP50	65
15	Performance comparison of mAP50-95 Results and mAP50 Improvement	67
16	Model Evaluation	68
17	F1-scores Results and Improvements of Various Models scales	69
18	mAP@50 Results and Improvements of Different Model Scales	71

19	mAP@50-95 Results and Improvements of Different Model Scales	72
20	YOLOv8 Performance Model Scales	73
21	Various Folds Performance Results	74
22	YOLOv8+CBAM and YOLOv8 Paired Sample Results	78
23	Model comparison between YOLOv8 and integrated YOLOv8 with CBAM	80
24	Training and Validation Performance Loss	83
25	YOLOv8 Confusion Matrix	85

LIST OF TABLES

No.	Title	Page
1	Summary of YOLO versions and CBAM Enhancements (2020-2025)	21
2	Related Literature Matrix	25
3	Hardware Specifications	37
4	Summary of Hyperparameters Used in YOLOv8 and CBAM–YOLOv8 Training	46
5	Summary of Image Dataset Used for Broiler Detection	59
6	Comparison of YOLOv8 and CBAM+YOLOv8 Performance Metrics Across Folds	75
7	T-test Comparing YOLOv8 and CBAM+YOOv8 Performance Metrics	76
8	Confidence Intervals for Paired Samples T-test results	76
9	Statistical Summary of YOLOv8 and CBAM+YOLOv8	77
10	T-test Comparison of YOLOv8+CBAM and YOLOv8	79
11	Paired Sample Statistics Results	87

DEFINITION OF TERMS

Annotate	The process of labeling or marking objects of interest in images (e.g., drawing bounding boxes and assigning class labels) to create training data for machine learning models.
Bounding Box	A rectangular box used in object detection to indicate the location of an object within an image.
Broiler	A type of chicken bred specifically for meat production. Broilers grow rapidly and are typically used in poultry farming operations.
CBAM	An attention mechanism that enhances the representational power of CNNs by focusing on important features along both the channel and spatial dimensions.
CNN	A deep learning architecture designed to automatically and adaptively learn spatial hierarchies of features from input images, making it effective for image recognition and classification tasks.
Dataset	A structured collection of data, such as labeled images, used to train, validate, and test machine learning models.
Dataset Splitting	The process of dividing a dataset into subsets typically training, validation, and test sets to ensure fair evaluation and avoid overfitting.
Deep Learning	A subset of machine learning that uses neural networks with many layers (deep architectures) to model complex patterns in data.
F1-score	A performance metric that balances Precision and Recall using their harmonic mean, especially useful when dealing with imbalanced datasets.
Google Colab	A free cloud-based platform developed by Google that allows users to write and execute Python code in Jupyter

	notebooks, commonly used for machine learning experiments.
Machine Learning	A field of artificial intelligence that enables systems to learn from data and improve their performance over time without being explicitly programmed.
mAP (mean Average Precision)	A metric that summarizes the performance of an object detection model across all classes and IoU thresholds, calculated as the mean of Average Precision values.
Normalization	A data preprocessing step that scales input data to a specific range (e.g., 0 to 1) or distribution, improving model convergence and performance.
Precision	A metric that measures the proportion of true positive predictions among all positive predictions.
Recall	A metric that measures the ability of a model to correctly identify all relevant instances.
Roboflow	A web-based platform that simplifies image annotation, dataset management, and preprocessing, often used to prepare datasets for training computer vision models.
T-Test	A statistical test used to determine whether there is a significant difference between the means of two groups, often used to validate experimental results.
YOLO	A real-time object detection algorithm that processes an entire image with a single neural network pass, simultaneously predicting bounding boxes and class probabilities.
Vanilla YOLO	Refers to the original or standard version of the *You Only Look Once (YOLO)* object detection model without any architectural modifications or added enhancements. It serves as the baseline for comparison with improved or customized versions.

Cohen's d	A statistical measure used to indicate the effect size or the standardized difference between two means. In research, it quantifies how significant the difference is between experimental results or model performances.
GitHub	A web-based platform for version control and collaborative software development. It allows researchers and developers to store, manage, and share their code repositories, track revisions, and collaborate with others.
Jamovi	An open-source statistical software designed for user-friendly data analysis. It provides a graphical interface for performing statistical tests such as t-tests, ANOVA, regression, and effect size calculations.
Visual Studio Code (VS Code)	A lightweight, open-source code editor developed by Microsoft. It supports various programming languages and tools, making it ideal for developing, debugging, and deploying machine learning models.
T-test	A statistical test used to compare the means of two groups to determine whether there is a significant difference between them. It is often used to evaluate model performance or experimental results.
Concat (Concatenation)	A layer operation in neural networks that combines multiple feature maps or tensors along a specific dimension, enabling the model to merge and learn information from different sources or layers.
Conv (Convolutional Layer)	A core component of Convolutional Neural Networks (CNNs) that applies

	filters to input data to extract spatial features such as edges, textures, and shapes from images.
Upsample	A neural network operation that increases the spatial resolution of feature maps. It is commonly used in decoder stages or detection heads to restore image size after downsampling operations.
C2f (Cross Stage Partial with Two Convolutions and Fusion)	A module introduced in YOLOv8 that enhances feature extraction by combining partial connections and feature fusion, improving model accuracy and computational efficiency.
YAML (Yet Another Markup Language)	A human-readable configuration file format used to define model architectures, training parameters, and dataset paths in machine learning frameworks like YOLO.
SPPF (Spatial Pyramid Pooling – Fast)	A module in YOLO architectures designed to capture multi-scale spatial features by pooling inputs at different scales, improving the model's ability to detect objects of varying sizes efficiently.
IoU (Intersection over Union)	A metric used to measure the overlap between the predicted bounding box and the ground truth bounding box. It indicates how accurately the model localizes objects.

CHAPTER I

INTRODUCTION

BACKGROUND OF THE STUDY

Computer vision has experienced significant growth in recent years, largely driven by advances in deep learning. Object detection, a fundamental aspect of computer vision, involves recognizing and pinpointing objects within images or video. This technology is widely applied across various domains, including healthcare, transportation, security, and agriculture. In agriculture, automated detection systems have improved efficiency and accuracy in monitoring livestock and poultry health, enabling data-driven animal management (Kamilaris & Prenafeta-Boldú., 2018).

YOLO (You Only Look Once) is among the most impactful deep learning object detection algorithms. By combining classification and localization in a single network pass, YOLO is well-suited for real-time applications (Redmon et al., 2016). In contrast to traditional two-stage approaches like Faster R-CNN, it lowers computational demands while preserving high accuracy, enabling efficient object detection in dynamic settings. The algorithm has evolved across multiple versions YOLOv3, YOLOv5, YOLOv8, and the latest YOLOv11 each offering improved architecture and detection performance (Jocher et al., 2023).

In poultry production, YOLO has been widely adopted to detect sick or dead chickens, estimate flock counts, and identify abnormal behaviors (Zhao et al., 2021). These models enhance monitoring in large-scale farms where manual inspection is time-consuming and prone to human error.

Enhancements such as multi-scale training, receptive field optimization, and the integration of attention mechanisms have further strengthened YOLO's adaptability to challenging environments.

Although YOLO offers significant advantages, it faces challenges in complex environments with occlusion and clutter. The Convolutional Block Attention Module (CBAM) enhances feature extraction by applying channel and spatial attention in sequence (Woo et al., 2018). This dual-attention approach allows the model to emphasize important features while reducing background interference, improving detection accuracy in challenging scenes. CBAM has proven effective in precision-demanding domains such as medical imaging, industrial inspection, and agricultural diagnostics (Garshasebi., 2025; Mao et al., 2022).

Integrating CBAM with YOLO enhances the model's ability to detect overlapping chickens and recognize fine-grained visual cues, including postures and feather conditions, even under uneven lighting or dust interference. This synergy combines YOLO's real-time detection capability with CBAM's refined feature attention, reducing false positives in densely populated poultry environments (Lv & Su., 2024; Praveen & Jung., 2025). Lightweight YOLO variants such as YOLOv8n also enable efficient deployment on low-power devices, supporting scalable and cost-effective smart farming applications (Jiang et al., 2022).

This study bridges artificial intelligence and agricultural technology by applying deep learning-based object detection to real-world poultry management. Through the integration of YOLOv8 and CBAM, it demonstrates

how AI can enhance livestock monitoring, reduce human intervention, and promote data-driven precision farming. By combining state-of-the-art AI techniques with practical agricultural needs, the research contributes to the modernization of poultry health monitoring and the advancement of smart farming solutions.

This study also aligns with the United Nations Sustainable Development Goals (SDGs), particularly SDG 2 (Zero Hunger), SDG 9 (Industry, Innovation, and Infrastructure), and SDG 12 (Responsible Consumption and Production). Early detection of malnourishment supports food security by preventing livestock losses, while the integration of deep learning in agriculture promotes sustainable and innovative production practices.

However, existing YOLO-based models face challenges in detecting overlapping broilers due to visual occlusion, clutter, and uneven lighting. This study addresses these challenges by integrating CBAM into YOLOv8 to enhance spatial feature discrimination and improve detection accuracy in complex poultry environments. The attention mechanism enables the model to focus on salient regions while suppressing irrelevant background features, resulting in more robust feature representation.

RESEARCH OBJECTIVES

This research aims to improve object detection performance in crowded and complex scenes by enhancing the YOLOv8 architecture with attention mechanisms. The research focuses on integrating the Convolutional Block Attention Module (CBAM) into YOLOv8 and systematically evaluating its effect across different model configurations. By doing so, the study seeks to

determine not only whether CBAM integration provides measurable improvements but also whether such improvements are statistically significant.

1. To integrate CBAM into the YOLOv8 structure, enabling the model to focus more effectively on discriminative features in crowded scenes.
2. To evaluate different placements of the module within the backbone and neck of YOLOv8 to determine the most effective configuration.
3. To compare different YOLOv8 scales (n, s, m, l, x) with CBAM placements, identifying the optimal configuration in terms of accuracy and efficiency.
4. To determine whether the performance improvements of YOLOv8+CBAM are statistically significant compared to the baseline YOLOv8 using a paired-sample t-test.

SIGNIFICANCE OF THE STUDY

The increasing demand for robust, accurate, and real-time object detection systems in complex environments has propelled continuous innovation in computer vision research. This study contributes to that progress by enhancing the YOLOv8 architecture (Jocher et al., 2023) through the integration of the Convolutional Block Attention Module (CBAM) proposed by (Woo et al., 2018). The integration aims to address YOLOv8's limitations in handling small and overlapping objects (V7 Labs., 2024) by improving feature discrimination and localization performance. Through this enhancement, the study seeks to develop a more adaptable and precise detection framework while maintaining YOLOv8's efficiency in real-time applications. The

significance of this research is described below in relation to its contributions to various stakeholders and the broader field of artificial intelligence.

Academic Contribution

This study contributes to literature by providing empirical evidence on the effect of integrating CBAM into lightweight YOLOv8 variants, demonstrating its statistically significant impact on detection performance.

Deep Learning Engineers and Practitioners

By demonstrating a successful pipeline that combines lightweight detection with high-resolution classification, this research offers a replicable architecture applicable beyond poultry or agricultural use. It holds potential in fields such as industrial quality inspection, medical diagnostics, and autonomous systems, where both speed and precision are critical. For example, the model architecture can be adapted for real-time defect detection in manufacturing or rapid triage in medical imaging. Furthermore, its parameter-efficient structure ensures applicability in resource-limited environments, such as edge devices or embedded systems (Gao et al., 2023).

Computer Vision and Machine Learning Developers

Developers and engineers focused on real-time systems such as surveillance, autonomous navigation, and robotics can benefit from this study's framework for improving object localization in crowded scenes. The enhanced YOLOv8+CBAM model maintains high detection speed while improving accuracy and robustness, especially under occlusion and object overlap. This is consistent with previous findings that attention mechanisms enhance spatial

and channel-level feature extraction (Woo et al., 2018; Lavanya & Pande., 2023), demonstrating how targeted architectural modifications can improve real-world object detection applications.

Industry Practitioners and System Engineers

For practitioners in industries such as security, traffic monitoring, and smart agriculture, this study provides a practical foundation for deploying optimized detection systems capable of performing efficiently in visually complex settings. The integration of CBAM into YOLOv8 presents a model design that balances accuracy and computational cost, addressing challenges identified in prior works that focus on resource-constrained environments (Guo et al., 2024; Jawaharlalnehru et al., 2022). The study's results can serve as a guideline for implementing attention-based detection architectures in industrial and embedded applications.

For Future Researchers

This study encourages future exploration of attention mechanisms within real-time detection models designed for low-power hardware such as NVIDIA Jetson or Raspberry Pi. Continued research on lightweight, attention-based architectures could further optimize accuracy and energy efficiency, advancing the development of sustainable smart farming systems and edge-AI solutions.

Research Gap

While prior studies incorporating the Convolutional Block Attention Module (CBAM) into YOLO architectures have shown improvements in feature extraction and object recognition accuracy, most were conducted in static or

less complex environments. These works primarily focused on general object detection datasets in complex scenes, where object overlap and motion variability were minimal. In contrast, the present study extends the YOLOv8+CBAM integration to a more complex and dynamic agricultural setting involving broiler chickens. Unlike previous implementations, this research specifically optimizes YOLOv8+CBAM for enhanced detection of overlapping objects in crowded poultry scenes. Such conditions present unique challenges in localization, making this approach distinct from conventional YOLO+CBAM studies that have not explicitly addressed real-time occlusion handling in livestock monitoring.

Dataset Quantitative Justification

The dataset for this study consists of 1,000 images of broiler chickens, captured at Perello's Farm in Indang, Cavite. Each image has a resolution of 640 × 640 pixels and was acquired using standard mobile devices under varying environmental conditions, such as different lighting, camera angles, and feeding positions. The dataset was split into 80% for training and 20% for validation, with all images used for model testing and evaluation. By including overlapping chickens and complex farm backgrounds, the dataset presents a diverse and challenging set of scenarios, providing a solid foundation for assessing the model's ability to detect occluded and densely packed objects in real-time poultry settings. Approximately 80% out of the 1000 images contained overlapping instances. This high proportion of occlusion scenarios further stresses the detection model and reflects realistic conditions commonly encountered in commercial poultry farms.

SCOPE AND LIMITATIONS

This study focuses on the development and evaluation of an image-based system for detecting broiler chickens ranging from 3-4 weeks old in controlled farm environments, specifically within a poultry farm in Indang, Cavite. The system integrates YOLO for object detection and CBAM for enhanced object detection accuracy, and classification precision. The dataset used in this study was collected under consistent lighting conditions, with images captured at a camera angle of ≈45° and a distance of ≈150 cm from the chickens. The scope of the study is limited to broiler chickens and primarily addresses controlled farm environments through visual characteristics captured via imaging devices.

The model faces limitations related to computational constraints, which may restrict real-time detection performance. Its accuracy can also be affected by varying lighting conditions, differences in broiler chicken breeds, and the quality of annotated training data. Because the dataset was collected from a single farm, the model may exhibit biases that limit its robustness and generalizability to other environments. Future research may improve performance and adaptability by incorporating multi-modal inputs, such as thermal and RGB data. Additionally, expanding the dataset to include multiple farms and diverse rearing conditions would help reduce dataset bias and enhance generalization across different operational settings. Further optimization through model pruning, quantization, or lightweight architectures could also improve inference speed, making the system more suitable for real-time deployment on edge devices.

CHAPTER II

THEORETICAL FRAMEWORK OF THE STUDY

REVIEW OF RELATED LITERATURE AND STUDIES

Agriculture

Deep Learning and Object Detection in Poultry Monitoring

Recent developments in computer vision have made it possible to automate poultry health monitoring using deep learning–based object detection. The You Only Look Once (YOLO) framework, in particular, has become widely adopted due to its combination of speed and accuracy, making it well-suited for real-time livestock monitoring.

A real-time chicken-health detection model that added a scale-aware receptive-field module to YOLOv5 improved performance in complex farm settings (Zhang et al., 2023). Likewise, a lightweight YOLOv3 variant enabled efficient poultry monitoring on limited on-farm hardware (Shi et al., 2023). Subsequent work using YOLOv5 for identifying sick and dead chickens achieved strong results, with an F1-score of 94%, 100% precision, and 97% mAP@0.5, underscoring YOLO's value for welfare assessment (Syafaah et al., 2024). More recent integration of thermal and visual imaging into YOLOv8 further improved reliability across lighting conditions, reaching a mAP@0.5 of 0.988 and an F1-score of 0.972 (Elmessery et al., 2023).

Overall, YOLO-based systems continue to advance through lightweight designs, enhanced receptive fields, and multimodal fusion—developments that

support the current study's use of CBAM-enhanced YOLOv8 for detecting malnourished chickens.

Edge AI and Real-Time Poultry Monitoring

Deploying deep learning models in real-time farm environments requires hardware-efficient implementations. A health monitoring system utilizing an enhanced FCOS-Lite model paired with MobileNet was developed for Edge AI applications, achieving a mean average precision (mAP) of 95.1% and an F1-score of 94.2%, while maintaining real-time processing speeds above 20 frames per second (Tong et al., 2024).

Beyond accuracy, (Tong et al., 2024) emphasized energy efficiency and bandwidth optimization by employing edge-AI-enabled CMOS sensors and incorporating quantization and knowledge distillation techniques to reduce memory usage without compromising performance. This design approach makes Edge AI particularly relevant for smart farming systems in rural areas, enabling cost-effective and scalable deployment of real-time monitoring solutions.

YOLOv8 Improvements

Traditional and Technological Methods for Object Detection

Object detection has significantly evolved, transitioning from traditional methods based on handcrafted features to contemporary deep learning approaches. Early techniques, such as the Viola–Jones detector (2001) and Histogram of Oriented Gradients (HOG), depended on manually designed features coupled with classical machine-learning classifiers. While effective in

controlled environments, these systems often failed to generalize under complex or real-time conditions due to variations in lighting, occlusion, and object scale (Sapkota et al., 2025).

From 2000 to 2015, traditional object detection typically relied on a three-step process of region proposal, feature extraction, and classification. Sliding-window searches generated candidate regions but were computationally expensive and often redundant (Kaur et al., 2022). Feature extraction used descriptors such as Haar-like features, HOG, and later SIFT and SURF, which improved robustness to scale, rotation, and illumination (Abdalssalam, 2024). Although foundational, these methods lacked the accuracy and speed needed for real-world applications.

The rise of deep learning marked a major shift, allowing systems to learn hierarchical features directly from data. Convolutional Neural Networks (CNNs) replaced handcrafted descriptors with end-to-end feature learning, greatly improving detection accuracy and generalization (Mathur & Gupta, 2024). Frameworks such as YOLO, SSD, and R-CNN further unified detection and classification within architectures capable of real-time performance. Meanwhile, open-source tools like OpenCV and accessible programming frameworks such as Python accelerated experimentation and deployment, broadening the use of deep-learning-based detection (Tuiamat, 2024).

Contemporary object detection research emphasizes efficiency, accuracy, and scalability. One-stage detectors like YOLO and SSD deliver real-time performance, whereas two-stage models such as R-CNN achieve higher accuracy at the cost of slower processing (Lamichhane et al., 2025). These

developments have expanded the application of computer vision across domains including robotics, healthcare, and security. Ongoing work continues to improve performance in challenging conditions, strengthen small-object detection, and extend models to emerging areas like augmented reality and environmental monitoring (Trigka et al., 2025).

In summary, the evolution from traditional, hand-engineered techniques to data-driven deep learning models represents a milestone in computer vision. Early frameworks like Viola–Jones and HOG established the foundation, while modern architectures such as YOLO, SSD, and R-CNN have defined the present era of automated, high-speed, and accurate object detection. This progression directly supports the current study's adoption of YOLOv8 for reliable detection in poultry environments.

Model Enhancements for YOLOv8

YOLOv8 represents one of the most advanced developments in object detection technology, yet it continues to face challenges in identifying small or overlapping objects within cluttered environments. To address these issues, six modified YOLOv8 versions optimized for different object sizes were proposed, demonstrating that targeted architectural adjustments can significantly improve performance while preserving computational efficiency (Rasheed & Zarkoosh., 2025).

Building on this, an improved variant known as YOLO-MIF integrated multi-information fusion (MIF) techniques and outperformed both baseline YOLOv8 and Faster R-CNN in terms of accuracy and processing speed, particularly in grayscale imaging conditions (Wan et al., 2024). This

enhancement highlights YOLO-MIF's suitability for low-light poultry-farming scenarios where traditional RGB imaging may be less reliable.

As these models evolve toward real-world deployment, research increasingly focuses on optimizing YOLOv8 for hardware-limited systems. This trend has led to the development of Edge AI solutions that bring high-performance deep-learning models closer to field applications. Consequently, integrating attention-based and lightweight architectures has become crucial for real-time poultry monitoring systems that must balance detection accuracy, processing speed, and energy efficiency.

Optimizing YOLO Algorithm

Numerous studies have focused on improving the YOLO algorithm to enhance detection accuracy, robustness, and computational efficiency. Architectural modifications such as integrating CSPDarknet and PANet, together with advanced training techniques like mosaic augmentation, multi-scale learning, and anchor box tuning, have been shown to significantly strengthen YOLO's performance across various detection tasks (Singh et al., 2025; Guo et al., 2025).

Further refinements have been tailored to specific real-time applications. For instance, YOLO models employing optimized anchor boxes, non-maximum suppression, and unified detection frameworks demonstrated improved adaptability for video surveillance and autonomous systems (Lavanya et al., 2023). In aerial imaging, enhancements including anchor box clustering, multi-scale training, and refined candidate box screening minimized false positives

and improved precision in UAV-based detection scenarios (Jawaharlalnehru et al., 2022).

The growing demand for efficient and scalable livestock monitoring further underscores the importance of such optimizations. A national chicken inventory of 200.21 million birds as of June 2023 reflected a 2.8% increase from the previous year, highlighting the need for intelligent monitoring systems that ensure animal welfare and operational efficiency (Philippine Statistics Authority [PSA]., 2023).

In summary, continuous optimization of YOLO through architectural redesign and refined training strategies reflects the ongoing effort to balance detection accuracy with real-time performance. These advancements form the technical foundation for the current study's development of an enhanced 17 YOLOv8 model designed for efficient and precise poultry health detection under real-world farm conditions.

Attention Mechanisms

Deep Learning-Based Object Detection

Deep learning has significantly advanced object detection by enabling machines to automatically learn visual patterns with high precision. Convolutional Neural Networks (CNNs) are central to this progress, providing hierarchical features that adapt to different object types and scales (Tang et al., 2025). With pre-trained models and transfer learning frameworks such as YOLO, SSD, and Faster R-CNN, deep learning has also made object detection

more accessible and efficient for both research and real-world use (Brownlee et al., 2021).

Modern detection systems are now widely used in traffic analysis, healthcare, and industrial automation. Combining neural networks with image-processing techniques enables real-time detection, while attention mechanisms and multi-scale feature extraction improve recognition of small or overlapping objects in complex scenes (Zhao et al., 2020; Wang et al., 2021). Ongoing architectural advances also work to balance accuracy and computational efficiency, addressing the trade-off between precision and processing speed (Kumar et al., 2024).

In specialized contexts such as medical imaging, deep-learning-based detectors have demonstrated strong potential for identifying anatomical structures and abnormalities, proving the adaptability of these models across diverse visual tasks (Zhao et al., 2019).

In summary, deep learning has transformed object detection into a data-driven, adaptive process capable of real-time performance across multiple fields. Continuous innovations ranging from CNN-based architectures to attention mechanisms provide the foundation for ongoing improvements such as YOLO optimization, which the following section explores in detail.

Convolutional Block Attention Module

The Convolutional Block Attention Module (CBAM) is a lightweight attention mechanism that improves feature representation in convolutional neural networks by applying channel and spatial attention sequentially.

Introduced in 2018, CBAM generates attention maps along these two dimensions, allowing networks to focus on important features while reducing irrelevant information (Woo et al., 2018). Its design enhances performance in computer vision tasks, including image classification and object detection, while keeping computational costs low, making it a practical addition to modern architectures.

Building on this foundation, subsequent studies have validated and extended CBAM's effectiveness across multiple domains. For instance, CBAM significantly improved accuracy in medical image segmentation, confirming its applicability beyond traditional vision benchmarks (Almarzouqi & Al-Habsi., 2022). More recent research further adapted CBAM to transformer-based and hybrid deep-learning architectures, underscoring its continued relevance and versatility (Zhao et al., 2024).

In summary, CBAM bridges feature extraction and attention refinement through a dual-focus mechanism that strengthens spatial and channel information processing. Its adaptability and lightweight design allow seamless integration into diverse network architectures, supporting ongoing advancements in deep-learning-based detection and recognition.

YOLO (You Only Look Once)

Object detection remains a significant challenge in computer vision, with deep learning driving major improvements in both accuracy and speed. Object detectors are typically categorized into two-stage models, known for higher precision but slower inference, and single-stage models, which prioritize real-time performance. Among the latter, YOLO (You Only Look Once) has emerged

as one of the most widely adopted frameworks, evolving through successive versions that now rival or surpass some two-stage models in both speed and accuracy (Diwan et al., 2022).

Over time, YOLO has become foundational to real-time object detection applications across diverse fields, including robotics, autonomous driving, and video surveillance. Recent analyses trace its evolution from the original YOLO to advanced versions such as YOLOv8, YOLO-NAS, and transformer-based adaptations, noting key improvements in evaluation metrics, post-processing, architecture, and training strategies (Terven et al., 2023).

Overall, YOLO has matured into a robust and efficient framework for real-time object detection. Nevertheless, challenges persist in accurately detecting small, overlapping, or occluded objects—issues that have driven research toward integrating attention mechanisms such as the Convolutional Block Attention Module (CBAM) to enhance feature discrimination and detection precision.

Application of CBAM in YOLO Models

The Convolutional Block Attention Module (CBAM) is widely used in YOLO architectures to enhance feature extraction through spatial and channel attention. In remote sensing, CBAM-based YOLO models improved detection of small and overlapping objects in cluttered aerial images (Garshasebi, 2025; Cheng et al., 2024). In agriculture, CBAM-YOLOv5-C3TR enhanced plant disease detection (Lv & Su, 2024), and the CBAM-STN-TPS-YOLO framework improved recognition of growth structures under occlusion (Praveen & Jung,

2025). Overall, these works show that CBAM helps YOLO emphasize critical features while reducing background noise, boosting accuracy in complex visual environments.

Beyond agricultural applications, CBAM-enhanced YOLO models have also been applied in industrial and medical domains. For instance, they achieved higher detection precision in conveyor-belt monitoring under fog and motion blur (Mao et al., 2022; Liu et al., 2023), improved diagnostic accuracy in bronchoscopy imaging (Yan et al., 2024), and enhanced performance in thermal imaging where visibility is limited (Pham Hong, 2022). Additional results showed that CBAM-YOLOv5s achieved measurable gains in precision and mAP over baseline architectures (Jiang et al., 2022).

Collectively, these findings confirm that CBAM integration consistently strengthens YOLO's robustness across diverse conditions. Building on this foundation, the present study applies a CBAM-enhanced YOLOv8 model for real-world poultry monitoring, specifically addressing challenges of occlusion, crowding, and dust to maintain detection accuracy in farm environments.

Additional RRLs

An optical flow–based camera system was tested for broiler chicken welfare monitoring across 74 flocks, where early movement patterns were significantly correlated with mortality and hockburn, enabling early intervention and showing strong potential for commercial deployment in poultry farms across the UK and Switzerland (Dawkins et al., 2020). The study demonstrates that continuous, non-invasive monitoring of flock activity can serve as a reliable

early-warning tool for welfare issues, supporting data-driven management decisions in large-scale poultry production systems.

Building on this foundation, computer vision has been increasingly integrated into poultry management systems. An enhanced YOLOv8 model incorporating RevCol as the backbone and CoordAttention modules achieved a 90.9% recognition rate, an F1-score of 96.7%, and an AP50:95 of 80.6%, while maintaining real-time performance on Jetson AGX Orin hardware (Wu et al., 2025).

Parallel research in agriculture has demonstrated similar deep-learning strategies. STF-YOLO, a hybrid of Swin Transformer and YOLOv8, effectively detected small tea buds in complex environments, capturing fine-grained features essential for small-object recognition (Shi et al., 2024). Likewise, an enhanced Faster R-CNN model identified feeding behaviors of hens in stacked cages by integrating ResNet101 and PAN, achieving higher precision and recall through IoU loss optimization (Hao et al., 2022).

Further improvements in YOLO-based monitoring have extended detection reliability in challenging farm conditions. An AI-powered YOLOv8 system accurately counted chickens under occlusion, variable lighting, and high density, achieving 93.1% precision and 93.0% recall while outperforming YOLOv5 in real-time flock management (Cruz et al., 2024).

Beyond agriculture, similar advancements have been applied in environmental and industrial settings. A CBAM- and CIoU-enhanced YOLOv6 improved smoke and flame detection for forest fire monitoring, increasing both speed and accuracy (Wang et al., 2023). Likewise, an improved YOLOv4

architecture for pavement damage detection embedded CBAM in the Neck and Head regions, resulting in a nearly 3% rise in mean average precision (Li et al., 2022).

Innovations have also expanded into aerial and energy applications. VisioDECT, a drone detection system integrating CBAM-enhanced YOLO with GELAN-E and EigenCAM visualization, achieved robust detection across varied lighting and weather conditions (Islam et al., 2025). Similarly, a CBAM-BiFPN-YOLOv8 model for wind turbine blade damage detection enhanced attention and feature fusion, yielding consistent gains in detecting small and complex defects (Yu et al., 2024).

Lastly, continued refinements of YOLOv5 architectures highlight the adaptability of attention-based designs. The integration of BiFPN, CBAM, and DIoU-based NMS improved road-object detection under occlusion and scale variation (Chen et al., 2023), while a YOLOv5-CBAM configuration for battery lead-tab defect detection reduced parameters and enhanced precision, offering efficiency and accuracy for manufacturing inspection (Mun et al., 2023).

Consistent improvements in accuracy, precision, and efficiency show the adaptability of YOLO-based models across agricultural, industrial, and environmental applications. Enhancements such as CBAM, BiFPN, and quantization have strengthened small-object detection, reduced occlusion issues, and improved real-time performance. However, most studies focus on specific domains and controlled settings, leaving a gap in large-scale, real-time farm monitoring. To address this, the present study develops an optimized

YOLOv8 model with CBAM integration for practical broiler monitoring in real production environments.

Integration with YOLOv8's Feature Fusion Pipeline

This study builds on YOLOv8's feature-fusion pipeline, which uses Cross Stage Partial (CSP) connections and a Path Aggregation Network (PAN) to effectively combine low- and high-level features. Adding the Convolutional Block Attention Module (CBAM) enhances the model's focus on critical spatial and channel information while suppressing background noise. This integration strengthens YOLOv8's ability to detect small and overlapping objects, improving precision and robustness in complex poultry environments. Overall, the YOLOv8 + CBAM framework boosts feature extraction without compromising real-time performance, supporting accurate and efficient broiler health detection.

Table 1. Summary of YOLO versions and CBAM Enhancements (2020-2025)

YOLO VERSION / MODEL	AUTHORS & YEAR	IMPROVEMENT/ ENHANCEMENT INTRODUCED	OUTCOME/ BENEFIT
YOLO (with Attention and Multi-Scale Feature Extraction)	Zhao et al. (2020)	Integrated attention mechanisms and multi-scale feature extraction	Improved detection of small and overlapping objects in complex scenes
YOLO (UAV-Based Detection Enhancements)	Jawaharlalnehru et al. (2022)	Added multi-scale training and anchor box refinement for UAV detection	Reduced false positives and improved aerial image precision
YOLOv4 + CBAM + K-means++	Li et al. (2022)	Embedded CBAM in Neck and Head regions and applied K-means++ for anchor optimization	Improved mAP by ≈3% in pavement damage detection

CBAM-YOLOv5s	Jiang et al. (2022)	Added CBAM attention to YOLOv5s	Achieved higher precision and mAP than baseline YOLOv5
CBAM-YOLO (Industrial Monitoring)	Mao et al. (2022); Liu et al. (2023)	Applied CBAM for conveyor belt defect detection under fog/motion blur	Increased detection precision under adverse conditions
Enhanced YOLOv5 (BiFPN + CBAM + DIoU-NMS)	Chen et al. (2023)	Combined BiFPN, CBAM, and DIoU-based NMS	Achieved higher precision and mAP in occluded/scale-varied road detection
YOLOv5-CBAM (Defect Detection)	Mun et al. (2023)	Replaced bottleneck blocks with CBAM	Reduced parameters and improved defect segmentation accuracy
YOLOv5 (Scale-Aware Receptive Field Enhancement)	Zhang et al. (2023)	Introduced a scale-aware receptive field module	Improved multi-scale chicken detection accuracy
YOLOv6 + CBAM + CIoU	Wang et al. (2023)	Integrated CBAM and CIoU	Enhanced smoke and flame detection for fire monitoring with better speed and accuracy
YOLOv8 (Thermal–Visual Fusion)	Elmessery et al. (2023)	Combined thermal and visual imaging	Achieved mAP@0.5 = 0.988, F1 = 0.972, effective under low-light conditions
Lightweight YOLOv3	Shi et al. (2023)	Reduced model complexity for real-time use	Enhanced speed–accuracy balance for resource-constrained environments
CBAM-YOLOv5-C3TR	Lv & Su (2024)	Integrated CBAM and Transformer (C3TR)	Improved plant disease detection under occlusion
CBAM-STN-TPS-YOLO	Praveen & Jung (2025)	Added spatial transformer and thin-plate spline (TPS) with CBAM	Enhanced spatial feature focus under complex occlusions
STF-YOLO (Swin Transformer + YOLOv8)	Shi et al. (2024)	Combined Swin Transformer and YOLOv8	Improved fine-grained small object detection
CBAM-BiFPN-YOLOv8	Yu et al. (2024)	Integrated CBAM and BiFPN	Strengthened attention, feature fusion, and

			bounding-box regression
YOLO-MIF (YOLOv8 + Multi-Information Fusion)	Wan et al. (2024)	Incorporated MIF techniques	Outperformed YOLOv8 and Faster R-CNN in grayscale / low-light poultry detection
AI-powered YOLOv8 (Occlusion Handling)	Cruz et al. (2024)	Optimized YOLOv8 for occlusion and lighting variations	Achieved 93.1% precision, 93.0% recall, outperforming YOLOv5
Enhanced YOLOv8 (RevCol + CoordAttention)	Wu et al. (2025)	Used RevCol backbone and CoordAttention	Achieved F1 = 96.7%, AP50:95 = 80.6%, and real-time edge deployment
Modified YOLOv8 (Six Optimized Versions)	Rasheed & Zarkoosh (2025)	Created six variants tuned for different object sizes	Improved detection of small/overlapping objects with efficient computation
CBAM-YOLO (Remote Sensing)	Garshasebi (2025); Cheng et al. (2024)	Applied CBAM to aerial YOLO models	Improved detection of small and overlapping targets in cluttered imagery
CBAM-Enhanced YOLO (Drone Detection)	Islam et al. (2025)	Combined CBAM, GELAN-E, and EigenCAM visualization	Enhanced accuracy and robustness in low-light and weather-varied drone detection
CBAM-YOLOv8 (Present Study)	Current Research (2025)	Integrates CBAM into YOLOv8 for poultry monitoring	Addresses occlusion, dust, and crowding; improves accuracy and robustness for real-world broiler detection

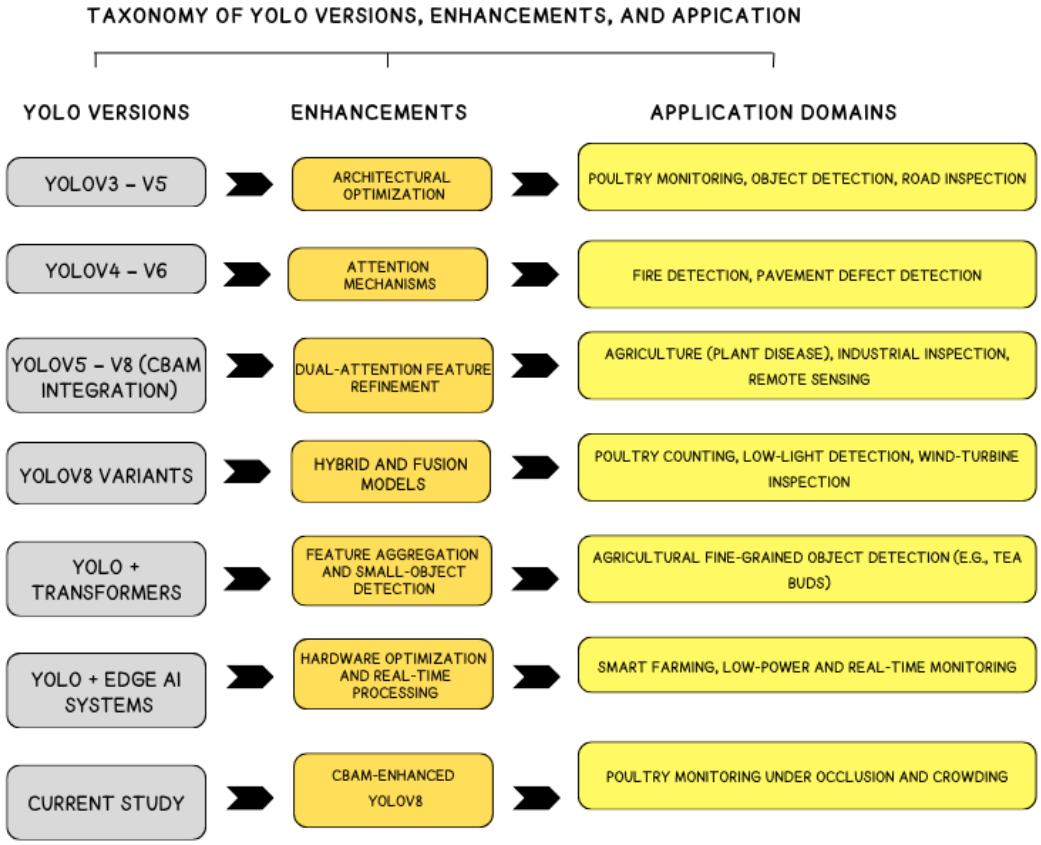


Figure 1. Taxonomy of YOLO Versions, Enhancements, and Application Domains (2020–2025)

The additional studies reinforce that variations of YOLO and attention-enhanced modules like CBAM consistently improve detection accuracy across diverse tasks from poultry counting and behavior monitoring to industrial defect inspection and environmental sensing. Collectively they show both architecture like RevCol, BiFPN, CoordAttention, transformers and deployment such as Jetson, quantization, edge-optimizations strategies can yield practical, real-time systems. However, while these works validate the methods and hardware pathways, relatively few focus specifically on crowded litter-floor broiler detection under real farm conditions, which motivates this study's targeted

evaluation of CBAM placements within YOLOv8 and its assessment across model scales and edge-capable configurations.

Related Literature Matrix

Overlapping object detection can be crucial to certain agriculture and poultry farms. Table 2 summarizes a collection of studies that explore various methodologies, algorithms, and findings, highlighting the use of advanced deep learning approaches.

Table 2. Related Literature Matrix

AUTHOR/S	TITLE	YEAR	AIGORITHM/ TECHNIQUE	FINDINGS
Singh R.K.	How to Improve YOLOv3. Paperspace Blog.	2025	YOLO with CSPDarknet, PANet, anchor tuning	Enhanced detection speed and accuracy for complex scenes.
Rasheed, A. F., & Zarkoosh, M.	Optimized YOLOv8 for multi-scale object detection. Journal of Real-Time Image Processing	2025	Modified YOLOv8	Developed six YOLOv8 variants for detecting small objects. Each version improved detection precision whale keeping computational requirements manageable.
Guo, J., & Liu C.	Optimizing YOLO Algorithm for Efficient Object Detection in Real-Time Systems	2025	YOLOv8 with advanced training methods	YOLOv8 with advanced training methods
Garshasebi, N.	Improving Small Object Detection in Remote Sensing with YOLO11-CBAM and Deep Learning. Earth Science - Journal of China Univ. of Geosciences	2025	YOLOv11, CBAM, extra detection head	Achieved mAP50 of 76.68% on DOTA dataset; robust for small object detection in real-world remote sensing.
Praveen, S., & Jung, Y.	CBAM-STN-TPS-YOLO: Enhancing agricultural object detection through spatially adaptive attention mechanisms. arXiv	2025	YOLO + CBAM + STN +TPS	Outperformed YOLO and STN-YOLO; reduced false positives by 12%; deployable on edge devices.

Islam, M. S. B., Chowdhury, M. E. H., Hasan-Zia, M., Kashem, S. B. A., Majid, M. E., Kunju, A. K. A., Khandakar, A., Ashraf, A., & Nashbat, M.	VisioDECT: A novel approach to drone detection using CBAM-integrated YOLO and GELAN-E models	2025	YOLOv5/YOLOv8 + CBAM + GELAN-E + EigenCAM	High accuracy and recall across conditions; CBAM improved small/occluded drone detection; GELAN-E outperformed YOLOv8 in low-light scenarios.
Wu, Z., Yang, J., Zhang, H., & Fang, C	Enhanced Methodology and Experimental Research for Caged Chicken Counting Based on YOLOv8	2025	YOLOv8-s enhanced + RevCol backbone + CoordAttention modules (YOLO-CCA)	Achieved 90.9% recognition rate with F1 score of 96.7% and AP50:95 of 80.6%; real-time detection at ~90.9 FPS on Jetson AGX Orin, enabling efficient and low-cost poultry monitoring.
Lv, M., & Su, W.H.	YOLOV5-CBAM-C3TR: an optimized model based on a transformer module and attention mechanism for apple leaf disease detection. *Frontiers in Plant Science*	2024	YOLOv5 + CBAM + Transformer (C3TR)	Improved mAP@0.5 by 8.25% over baseline YOLOv5; achieved 92.4% accuracy.
Cheng, A., Xiao, J., Li, Y., Sun, Y., Ren, Y., & Liu, J.	Enhancing Remote Sensing Object Detection with K-CBST YOLO: Integrating CBAM and Swin-Transformer. *Remote Sensing*	2024	YOLOv5 + Swin-Transformer + CBAM + smooth NMS	Achieved 68.3% mAP (DIOR) and 78.4% mAP (DOTA); better at cluttered and dense object detection.
Sapkota R. & Karkee M.	Optimizing YOLO Algorithm for Efficient Object Detection in Real-Time SystemsComparing YOLOv11 and YOLOv8 for instance segmentation of occluded and non-occluded immature green fruits in complex orchard environments	2024	YOLOv11, YOLOv8	YOLOv11 was more accurate, YOLOv8 was faster in orchard tests.
Mehdizadeh, S. A., Siriani, A. L. R., & Pereira, D. F.	Optimizing deep learning algorithms for effective chicken tracking through image processing. AgriEngineering	2024	YOLOv8n	Achieved >98% tracking accuracy; enabled behavioral and welfare monitoring.
Wan, D., Lu, R., Hu, B., Yin, J., Shen, S., Xu, T., & Lang, X.	YOLO-MIF: Improved YOLOv8 with multi-information fusion for object detection in gray-scale images. Advanced Engineering Informatics	2024	YOLO-MIF	Outperformed YOLOv8 and Faster R-CNN in grayscale image detection. Fusion of multiple image cues enhanced accuracy and speed.
Tong, Q., Wang, J., Yang, W., Wu, S., Zhang, W.,	Edge AI-enabled chicken health detection based on enhanced FCOS-Lite and knowledge	2024	FCOS-Lite + MobileNet	Achieved 95.1% mAP and 94.2% F1-score in real-time with over 20 FPS. The system was

Sun, C., & Xu, K.	distillation. Computers and Electronics in Agriculture			optimized for energy efficiency and bandwidth-limited environments.
Syafaah, L., Faruq, A., & Setyawan, N	Sick and dead chicken detection system based on YOLO algorithm. Ingénierie des Systèmes d'Information	2024	YOLOv5	Achieved F1-score of 94%, 100% precision, and 97% mAP@0.5 in detecting unhealthy chickens. Demonstrates superiority over manual inspection methods.
Yan, J., Zeng, Y., Lin, J., Pei, Z., Fan, J., Fang, C., & Cai, Y.	Enhanced object detection in pediatric bronchoscopy images using YOLO-based algorithms with CBAM attention mechanism. *Heliyon*	2024	YOLO + CBAM	Improved detection of bronchial structures despite occlusions, mucus, or blood; assisted diagnosis.
Yu, H., Wang, J., Han, Y., Fan, B., & Zhang, C.	Research on an Intelligent Identification Method for Wind Turbine Blade Damage Based on CBAM-BiFPN-YOLOv8	2024	YOLOv8 + CBAM + BiFPN + EIoU	Gains of +1.6% precision, +1.0% recall, +1.4% mAP@0.5; effective for small/complex defect detection with minimal computational cost.
Guo, Y., Aggrey, S. E., Yang, X., Oladeinde, A., Qiao, Y., & Chai, L.	Detecting broiler chickens on litter floor with the YOLOv5-CBAM deep learning model. Artificial Intelligence in Agriculture	2023	YOLOv5 + CBAM	Achieved 97.3% precision and 96.5% mAP@0.5 by improving occlusion handling. Outperformed traditional object detectors in dense poultry environments.
Tong, Q., Zhang,E., Wu, S., Xu, K., & Sun, C	A real-time detector of chicken healthy status based on modified YOLO.	2023	YOLOv5 with receptive field enhancement	Achieved improved detection accuracy in complex poultry farm environments.
Liu, J., Qiao, H., Yang, L., & Guo, J.	Improved Lightweight YOLOv4 Foreign Object Detection Method for Conveyor Belts Combined with CBAM. *Applied Sciences*	2023	YOLOv4 + GhostNet + CBAM + K-means++ + depth-separable convolutions	Achieved 99.32% accuracy and 54.7 FPS; significantly outperformed baseline YOLOv4.
Wang, A., Liang, G., Wang, X., & Song, Y.	Application of the YOLOv6 Combining CBAM and CIoU in Forest Fire and Smoke Detection	2023	YOLOv6 + CBAM + CIoU + AMP	Improved detection speed and accuracy for smoke and fire; better bounding box regression; effective for early fire warning systems.
Chen, J., Zhuang, Y., Li, Y., & Yu, H.	Enhanced YOLOv5: An Efficient Road Object Detection Method	2023	YOLOv5 + BiFPN + CBAM + DIoU-NMS	+1.6% mAP and +5.3% precision; improved detection of occluded/overlapping road objects.

Mun, H., Kim, T., & Lee, S.	Design and Implementation of Defect Detection System Based on YOLOv5-CBAM for Lead Tabs in Secondary Battery Manufacturing	2023	YOLOv5-CBAM (CBAM in C3 layer) + defect segmentation	Reduced parameters by >50% while boosting detection by ~2%; effective for defect measurement and classification in manufacturing
Elmessery, W. M., Gutiérrez, J., Abd El-Wahhab, G. G., Elkhaiat, I. A., El-Soaly, I. S., Alhag, S. K., Al-Shuraym, L. A., Akela, M. A., Moghanm, F. S., & Abdelshafie, M. F.	YOLO-Based Model for Automatic Detection of Broiler Pathological Phenomena through Visual and Thermal Images in Intensive Poultry Houses	2023	YOLOv5, YOLOv7, YOLOv8	YOLOv8 with thermal imaging achieved the highest accuracy (mAP50 = 0.988; F1 = 0.972). Thermal images outperformed visual images under poor lighting. Mosaic data augmentation improved detection and generalization. Effective in identifying multiple broiler pathologies.
Zhu, X., Wu, C., Yang, Y., Yao, Y., & Wu, Y.	Automated chicken counting using YOLO-v5x algorithm. In Proceedings of the 8th International Conference on Systems and Informatics (ICSAI) (pp. 1-6).	2022	YOLOv5x with Mosaic & TTA	Reached 95.87% detection accuracy at 23 ms per frame. Enhanced speed and robustness make it viable for high-density monitoring.
Diwan T, Anirudh G, Tembhurne JV	Object detection using YOLO: challenges, architectural successors, datasets and applications. Multimed Tools Appl.	2022	YOLOv1 - YOLOv5	Highlighted YOLO's increasing speed and accuracy through versions.
Xu, M., Hao S., Xu Z., Xu, M., Siya S., Hu, W., & Wang, J.	Foreign object detection in coal mine conveyor belt based on CBAM-YOLOv5. *Journal of China Coal Society*	2022	YOLOv5 + CBAM + defogging + deblurring + ASFF	Achieved 96.6% precision (+4.8% vs baseline) at 56.5 FPS on 1080p video.
Pham, H. V., Ninh, H., & Tran, T. H.	CBAM-YOLOv5 for infrared image object detection. SPIE Proceedings	2022	YOLOv5 + CBAM	Achieved 90.2% mAP across 8 thermal image categories; exceeded baseline YOLOv5.
Jiang, T., Li, C., Yang, M., & Wang, Z.	An Improved YOLOv5s Algorithm for Object Detection with an Attention Mechanism. Electronics	2022	YOLOv5s + CBAM + CIoU loss	On VOC2007, improved precision (+4.52%), recall (+1.18%), and mAP (+3.09%) vs baseline YOLOv5s.
Li, L., Fang, B., & Zhu, J.	Performance Analysis of the YOLOv4 Algorithm for Pavement Damage Image Detection with Different Embedding Positions of CBAM Modules	2022	YOLOv4 + CBAM + K-means++	Embedding CBAM in Neck and Head improved mAP by ~2.96%; sharper crack features; reduced background noise.

SYNTHESIS OF THE STUDY

Recent studies highlight the strength of YOLO algorithms in real-time object detection because they balance accuracy with processing speed. Many researchers have worked on improving YOLO's basic models to handle issues such as detecting small or overlapping objects, working in cluttered environments, and running efficiently on lightweight devices. For example, (Rasheed & Zarkoosh., 2025) improved YOLOv8 for multi-scale object detection, while (Wan et al., 2024) developed YOLO-MIF to achieve better accuracy in grayscale and low-light conditions. Similarly, (Li et al., 2022) showed that adding CBAM to YOLOv4 improved pavement crack detection, demonstrating how attention mechanisms can make feature extraction more effective.

These studies show that improving YOLO through feature fusion, attention modules, or anchor optimization can lead to much better detection performance. A common trend is combining YOLO with Convolutional Block Attention Modules (CBAM). (Wang et al., 2023) applied CBAM with CIoU in YOLOv6 for forest fire detection, which made smoke and flame recognition faster and more accurate. (Mun et al., 2023) replaced the bottleneck block in YOLOv5 with CBAM for detecting defects in battery production, which reduced the model's size while improving accuracy. (Chen et al., 2023) added CBAM and BiFPN to YOLOv5 for road object detection, achieving higher precision in busy traffic scenes. (Yu et al., 2024) improved YOLOv8 by combining CBAM with BiFPN to detect wind turbine blade damage, helping the model identify small and subtle defects more reliably.

Despite numerous advancements in object detection through YOLO architectures and attention mechanisms such as CBAM, most studies have focused on general object detection datasets or simpler environments. Limited attention has been given to complex agricultural settings, particularly in poultry farms where occlusion, overlapping objects, and dust interference often degrade detection accuracy. Moreover, while YOLOv8 offers improved speed and localization capabilities, its integration with CBAM has not been extensively studied for dense broiler environments. This research aims to address this gap by enhancing YOLOv8 with CBAM to achieve more robust real-time detection performance under challenging conditions.

Although recent studies have demonstrated the effectiveness of integrating CBAM with various YOLO architectures in a wide range of computer vision tasks, the specific combination of YOLOv8 and CBAM remains underexplored, particularly in highly dense and visually complex poultry environments. Existing implementations generally concentrate on general-purpose object detection or rely on controlled laboratory conditions where targets are clearly visible and environmental factors are minimized. As a result, these approaches do not fully account for the unique challenges encountered in actual broiler farm settings such as severe occlusion caused by overlapping birds, inconsistent or low lighting, the presence of dust and feather particles, and frequent movement within confined spaces.

These challenges notably diminish detection accuracy and model reliability, highlighting a research gap in applying advanced attention mechanisms to improve performance under real-world farm conditions. This

study seeks to address this gap by developing and evaluating a CBAM-enhanced YOLOv8 model tailored for broiler environments. By integrating CBAM, the model is expected to prioritize the most informative spatial and channel-wise features, enhancing detection accuracy, robustness to environmental noise, and real-time performance. By tailoring the method to the complexities of poultry farming conditions, this study seeks to contribute a more resilient and practical detection approach for precision livestock monitoring.

Additionally, the proposed approach emphasizes practical deployability by considering computational efficiency and real-time constraints common in commercial broiler farms. Beyond accuracy improvements, the study evaluates the trade-offs between performance gains and inference speed to ensure suitability for edge and low-power devices frequently used in precision livestock systems.

Through systematic experimentation and comparative analysis with baseline YOLOv8 configurations, this research aims to provide empirical evidence on the effectiveness of attention-driven feature refinement in challenging agricultural environments, thereby supporting scalable and cost-effective adoption of intelligent poultry monitoring solutions.

Furthermore, the findings of this study are expected to offer design insights for integrating attention mechanisms into object detection frameworks applied to other livestock and agricultural monitoring tasks. This can help inform future research on developing adaptive, environment-aware deep learning models that maintain reliable performance under varying real-world operational conditions.

Conceptual Framework

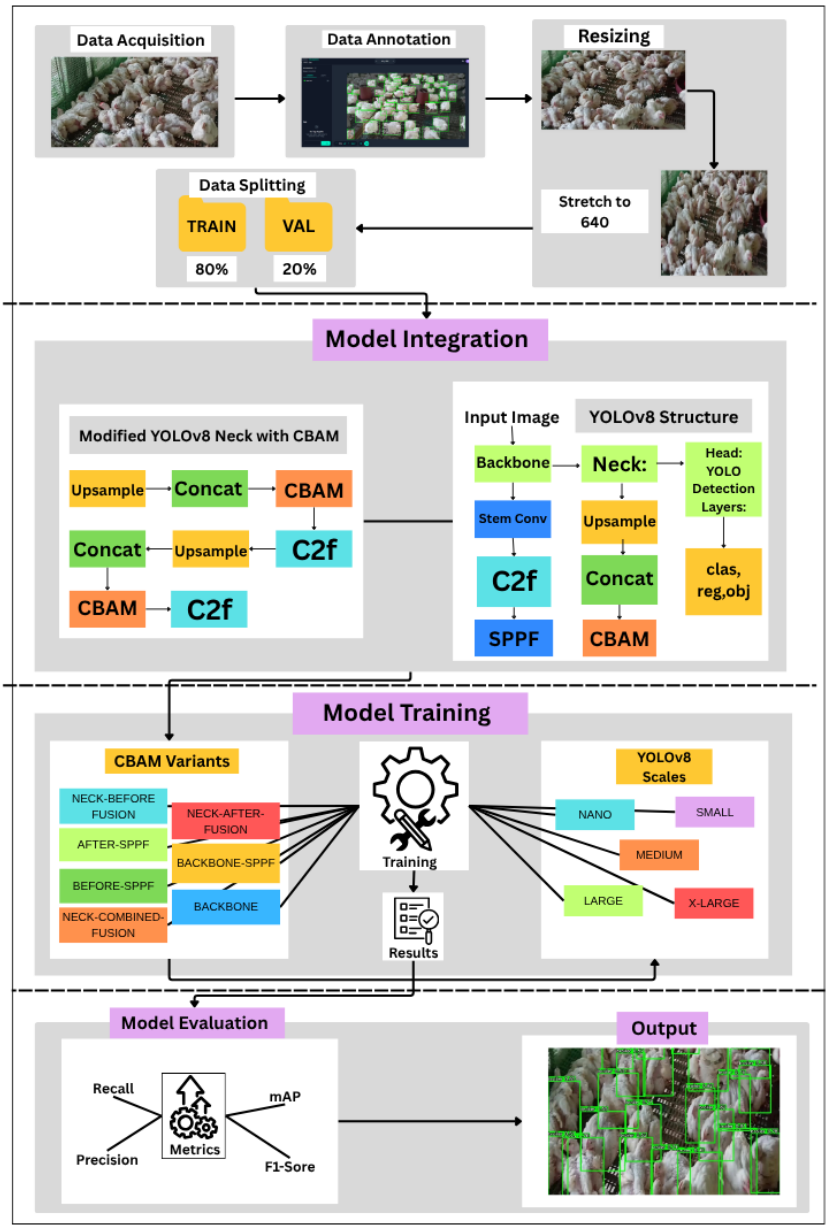


Figure 2. Conceptual Framework

Figure 2 depicts the conceptual framework of the modified YOLOv8 model designed to improve chicken detection in crowded and overlapping farm settings. The workflow begins with data acquisition, collecting chicken images from a local poultry farm. These images were then cleaned, annotated with bounding boxes, and resized for consistency. Finally, the dataset was split into

training (80%) and validation (20%) sets to support balanced model training and evaluation.

For model integration, the standard YOLOv8 architecture was augmented by embedding the Convolutional Block Attention Module (CBAM) within the network. The YOLOv8 backbone extracts critical visual features, while its neck fuses multi-scale information to detect objects of various sizes. CBAM was strategically placed within the neck between concatenation and C2f layers to highlight essential spatial and channel features while reducing background noise. This modification improves feature representation and detection precision, particularly in complex scenes where chickens overlap or cluster closely.

During model training, multiple CBAM placement variants were tested (e.g., before SPPF, after SPPF, backbone-SPPF, etc.) to identify the most effective configuration. Once the optimal placement was determined, the model was further trained across different YOLOv8 scales (Nano, Small, Medium, Large, and X-Large) to find the most suitable balance between accuracy and computational efficiency. The best performing model was selected based on its overall detection results.

In the evaluation phase, the model's performance was measured using metrics including precision, recall, F1-score, and mean Average Precision (mAP). Results indicate that the model reliably and accurately detects chickens, even in real-world farm conditions characterized by high crowding and occlusion. Overall, this framework establishes a comprehensive and systematic approach to improving YOLOv8's detection capability through CBAM

integration and multi-scale optimization. The enhanced model shows strong robustness and generalization potential, making it suitable for practical deployment in precision livestock monitoring systems.

Additionally, both statistical and qualitative analyses were performed to further validate the robustness of the proposed framework. Quantitative assessments, such as fold-level consistency and paired statistical tests, confirmed that the observed improvements were significant and not due to chance, while qualitative examination of detection results demonstrated enhanced localization and fewer missed detections. Together, these analyses strengthen the credibility of the CBAM-enhanced YOLOv8 framework as a practical and effective solution for automated chicken detection in complex farming environments. Furthermore, the complementary use of quantitative and qualitative validation provides a more holistic understanding of model behavior beyond metric-based evaluation alone.

The systematic evaluation across multiple model scales demonstrates the flexibility of the proposed approach in addressing different deployment constraints, from resource-limited edge devices to high-performance servers. This adaptability ensures that the CBAM-enhanced YOLOv8 framework can be tailored to diverse operational settings without compromising detection reliability. Moreover, the findings provide a strong foundation for future extensions toward real-time monitoring and decision-support systems in intelligent poultry farming. Such extensions could support early anomaly detection, welfare assessment, and data-driven management strategies to further enhance productivity and animal well-being

CHAPTER III

METHODOLOGY

This chapter contains different application requirements, processes, methodologies, and techniques used by the researcher in the development of the model.

Materials

The researchers used the following materials and resources to conduct the experiment:

Software

The model in this study was developed, tested, and implemented on various carefully selected softwares. The list of specific software used is provided below.

Paperspace

Paperspace was used as the primary cloud-based computing platform to train and test deep learning models. It provided access to GPU-accelerated virtual machines, enabling efficient processing of large datasets and reducing overall training time.

GitHub

GitHub served as the version control and project management platform. It was utilized to maintain the project repository, manage code revisions, and integrate custom modules into the base model. This ensured reproducibility of experiments and streamlined collaboration during development. Issue tracking and documentation features further supported organized development and clear tracking of implementation decisions.

Visual Studio Code (VS Code)

Visual Studio Code was employed as the integrated development environment (IDE) for writing, debugging, and managing code. Its extensions and support for Python facilitated seamless integration with GitHub and cloud platforms, making it the main tool for development and experimentation.

Google Colab

Google Colab was used as an alternative cloud-based environment for running Python notebooks. It provided free GPU and TPU resources that allowed rapid prototyping, testing of code, and validation of model performance before deployment on Paperspace.

Roboflow

Roboflow was used for dataset preparation and management. It facilitated image annotation, dataset augmentation, and preprocessing, ensuring that the training data was properly formatted and optimized for use with object detection models.

Jamovi

Jamovi was used for the statistical analysis of experimental results, particularly the paired sample t-tests comparing CBAM-modified YOLOv8 models with the vanilla YOLOv8 baseline.

Hardware

The model in this study was developed, tested, and implemented on a local machine with the following specifications.

Laptop

The laptop was used as a personal workstation to build and test the model. This local machine provided the power and resources required for model development.

Table 3. Hardware Specifications

Hardware Minimum Requirements	
CPU	AMD Ryzen 7 5825U with Radeon Graphics 2.00 GHz
MEMORY	16GB
VRAM	N/A
STORAGE	512GB SSD
SYSTEM TYPE	64-bit operating system, x64-based processor
GPU	Integrated AMD Radeon (TM) Graphics
RUNTIME	6hrs 30mins

RESEARCH DESIGN

This study used an experimental research design to examine the impact of integrating the Convolutional Block Attention Module (CBAM) into the YOLOv8 architecture for object detection. While the dataset consisted of chicken images, the main emphasis was on evaluating the performance of the algorithms rather than the specific detection subject. The subjects of this study were the YOLOv8 model and its modified version incorporating the CBAM module. The experiment involved systematically embedding CBAM into different locations within the YOLOv8 structure to determine the optimal placement in terms of detection performance. Performance was assessed

using standard object detection metrics to enable an objective comparison between the baseline and CBAM-enhanced models.

Dataset Setup

The dataset for this study comprises 1,000 images of broiler chickens captured at Perello's Farm in Indang, Cavite, with a resolution of 640 × 640 pixels. It was split into 80% for training and 20% for validation, without employing k-fold cross-validation. To improve model generalization, data augmentation techniques including random flipping, scaling, rotation, and brightness adjustment were applied.

Model training and evaluation were performed in a controlled experimental setup to ensure a fair and consistent comparison between the baseline YOLOv8 and the CBAM-enhanced variants. All models were trained using identical hyperparameters, training epochs, and hardware to minimize confounding factors that might affect performance. Detection performance was quantitatively evaluated using standard metrics precision, recall, F1-score, and mean Average Precision (mAP) providing an objective assessment of the impact of CBAM integration on detection accuracy and robustness.

In addition, inference speed and computational overhead were measured to evaluate the practicality of deploying the proposed model in real-time poultry farm environments. This ensured that performance improvements achieved through CBAM integration did not come at the cost of excessive latency or resource consumption. This confirms suitability for real-time deployment in resource-constrained farm settings.

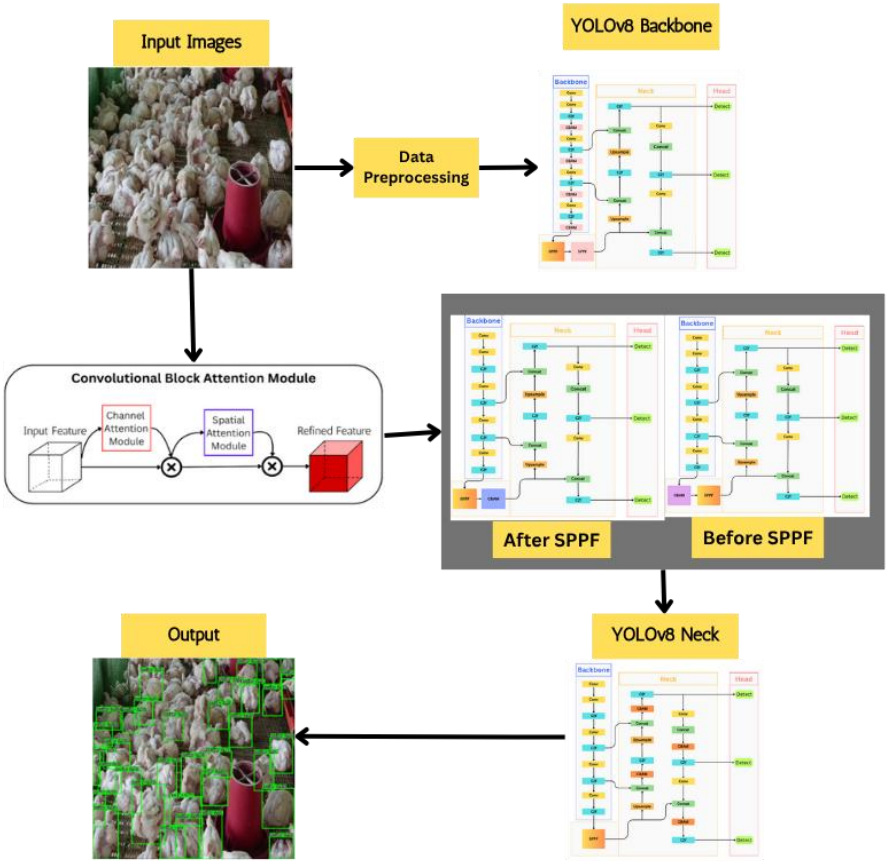


Figure 3. YOLOv8+CBAM Model Workflow

Figure 3 provides the overall workflow of the proposed YOLOv8+CBAM model. Figure 3 shows the key stages of the detection pipeline, including data preprocessing, feature extraction, attention enhancement using the Convolutional Block Attention Module (CBAM), and object localization. The CBAM module was inserted into the YOLOv8 backbone to refine feature representation and improve the model's ability to detect overlapping objects. This workflow illustrates how attention-enhanced features are propagated through the network to support more accurate and stable detection results. Overall, the diagram clarifies the integration of CBAM within YOLOv8 and its role in enhancing feature discrimination throughout the detection process. This structured pipeline support improved robustness under complex farm condition

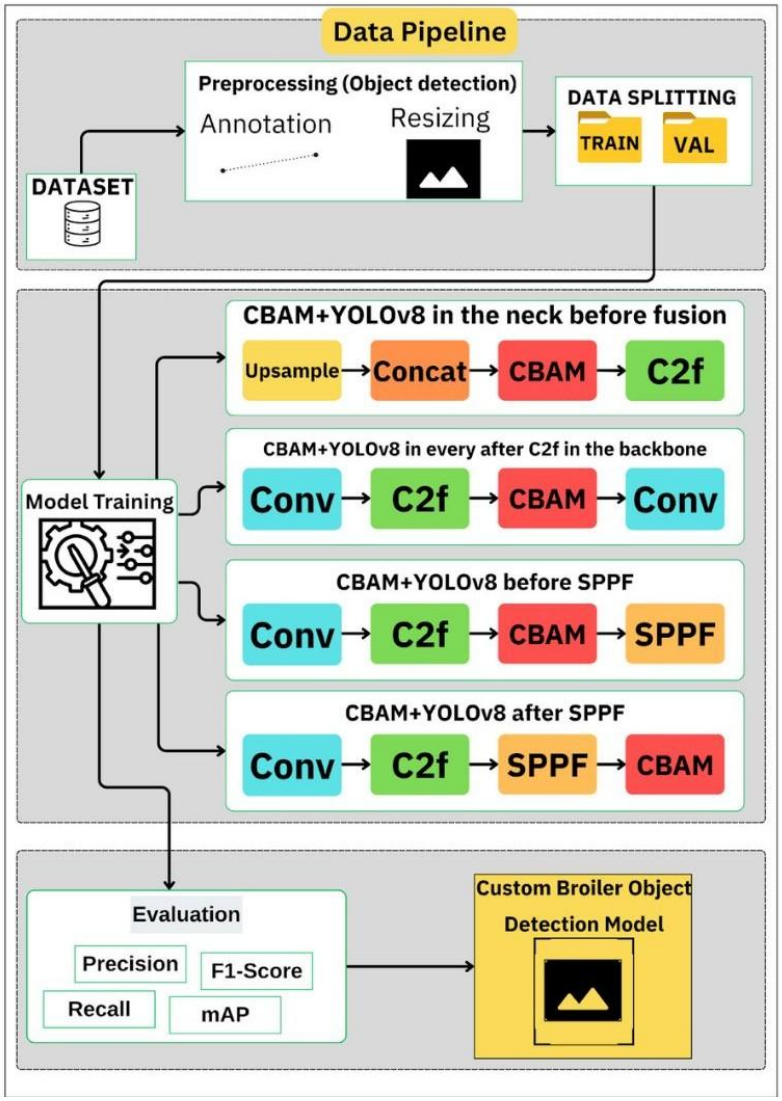


Figure 4. Model Development

Figure 4 depicts the workflow for developing a custom broiler object detection model using a CBAM-enhanced YOLOv8. The process starts with the data pipeline, where the dataset is preprocessed through annotation and resizing, then split into training and validation sets. During training, the Convolutional Block Attention Module (CBAM) is integrated at various points within YOLOv8: in the neck before feature fusion, after upsampling and concatenation, after each C2f block in the backbone, before the Spatial Pyramid Pooling-Fast (SPPF) module, and after the SPPF module. These configurations

are trained to optimize feature extraction and detection performance. Finally, models are evaluated using metrics such as precision, recall, F1-score, and mean Average Precision (mAP), resulting in a robust custom broiler object detection model tailored for the task.

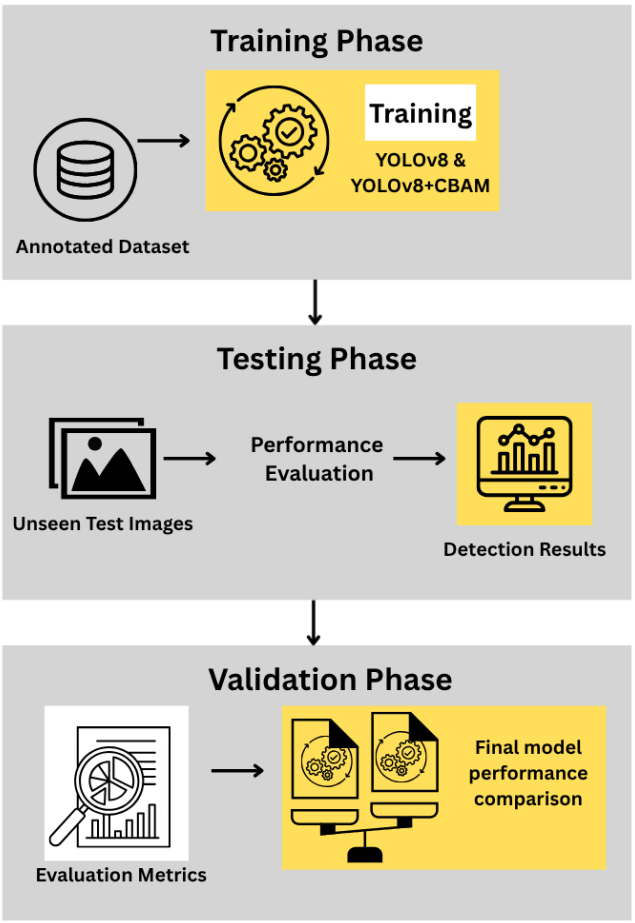


Figure 5. Training Pipeline of YOLOv8+CBAM

Figure 5 presents the flow of the experimental phase of this study, which consists of three main stages: training, testing, and validation. In the training phase, the YOLOv8 and CBAM-enhanced YOLOv8 models were trained using the prepared dataset and defined parameters. The testing phase evaluated the trained models on unseen data to assess their detection performance and robustness. Finally, the validation phase analyzed the models' outputs using

metrics such as precision, recall, F1-score, and mean Average Precision (mAP) to determine their overall accuracy and effectiveness.

To clearly illustrate the algorithmic process used in developing and evaluating the model, the following pseudo-code outlines the workflow for the CBAM–YOLOv8 model training and testing procedure.

Algorithm 1. Training and Evaluation Workflow for CBAM–YOLOv8 Model

1. Input: Annotated poultry image dataset D

2. Split D into training (80%) and validation (20%) sets

3. Preprocess dataset:

    a. Resize images to 640×640 pixels

    b. Apply augmentations (rotation, flipping, brightness, mosaic)

4. Initialize baseline YOLOv8m model

5. Integrate CBAM into YOLOv8 architecture:

    a. Option 1: After each C2f block in Backbone

    b. Option 2: Before or after the SPPF module

6. Set training parameters:

    a. Epochs = 100, Batch size = 16, LR = 0.001, Optimizer = Adam

7. Train YOLOv8 and CBAM–YOLOv8 models on training data

8. Validate both models using validation data

9. Evaluate models using metrics:

    a. Precision, Recall, F1-score, mAP@0.5

10. Compare YOLOv8 vs. CBAM–YOLOv8 performance

Output: Optimized CBAM–YOLOv8 model for broiler detection

Data Acquisition

The dataset used in this study was collected on-site at a poultry farm using a mobile phone camera. The dataset consisted of 1,000 annotated images containing a total of 27,561 labeled broiler chickens, captured under varying lighting conditions, chicken orientations, and environmental settings. Images were taken under natural farm conditions to reflect realistic scenarios, including variations in brightness, shadows, background clutter, and partial occlusions. The dataset was divided into 800 images for training and 200 images for validation, following an 80:20 ratio to ensure balanced model evaluation.

The use of a mobile phone camera ensured accessibility and practicality in data collection while maintaining sufficient image resolution for object detection tasks. The collected images were later organized and annotated to serve as the foundation for model training and evaluation.

Data Preprocessing

The acquired dataset underwent a series of preprocessing steps to make it suitable for training in the YOLOv8 framework. All preprocessing procedures

were conducted using Roboflow, a platform designed for dataset management and augmentation.

Data Annotation

Each image was annotated to identify bounding boxes around chickens, which served as the ground truth labels for training and validation. Roboflow's annotation tools were employed to generate labels in YOLO format, ensuring compatibility with the model's input requirements.

Data Resizing

To comply with YOLOv8's input specifications, all images were resized to a resolution of 640 × 640 pixels. This uniform resolution allowed the model to process images efficiently while maintaining consistent feature extraction across the dataset.

Data Augmentation

To improve model generalization and robustness, data augmentation techniques were applied using Roboflow's built-in augmentation pipeline. These included random rotation, flipping, brightness and contrast adjustment, which simulated diverse visual conditions such as varying illumination and viewing angles. This process expanded the effective size and diversity of the training dataset, helping the model perform better in real-world farm environments. Additionally, augmentation helped reduce overfitting by encouraging the model to learn more invariant and discriminative features rather than memorizing specific patterns from the original dataset.

Data Splitting

The dataset was divided into training and validation sets to support model development and performance comparison. A total of 1,000 images were used in the study, with 800 allocated for training and 200 reserved for validation. A separate test set was not used because the primary goal of the study was to compare our model to YOLOv8 standalone rather than report final deployment performance, and splitting the dataset further would have reduced the number of training samples too much for reliable learning. The chosen division provided enough data for effective training while maintaining a dedicated subset for unbiased evaluation.

Training Parameters

The experiment utilized the YOLOv8m (medium) variant, identified as the most optimal configuration through preliminary tests comparing YOLOv8 scales (n, s, m, l, and x). This variant was selected for its balanced trade-off between detection accuracy and computational efficiency.

Both the YOLOv8 and CBAM-enhanced YOLOv8 models were trained under identical configurations to ensure a fair comparison. The training was conducted using the Ultralytics YOLOv8 framework on the Paperspace cloud platform with GPU acceleration.

Key parameters were defined in the YAML configuration file and adjusted to balance accuracy and training time. The models were trained for 100 epochs with a batch size of 16 and an initial learning rate of 0.001, using the Adam optimizer for gradient updates. The input image size was 640×640

pixels, and data augmentation (rotation, flipping, brightness adjustment, and mosaic) was applied during training to enhance robustness.

The confidence threshold was set to 0.25 and the IoU threshold to 0.45 for bounding box predictions. Early stopping and model checkpointing were also implemented to prevent overfitting and retain the best-performing weights based on validation performance metrics.

Table 4. Summary of Hyperparameters Used in YOLOv8 and CBAM–YOLOv8 Training

Hyperparameter	Value	Description
Model Variant	YOLOv8m	Medium-scale configuration chosen for balance between speed and accuracy
Epochs	100	Total number of complete training iterations
Batch Size	16	Number of images processed per training step
Learning Rate (LR)	0.001	Initial step size for optimizer updates
Optimizer	Adam	Used for adaptive gradient-based optimization
Image Size	640 × 640 px	Input image resolution standardized for YOLOv8
Confidence Threshold	0.25	Minimum confidence for valid detections
IoU Threshold	0.45	Overlap threshold for non-maximum suppression
Data Augmentation	Rotation, flipping, brightness adjustment, mosaic	Enhances model robustness and generalization
Early Stopping	Enabled	Prevents overfitting and retains best weights
Training Platform	Paperspace (GPU)	Computational environment used for model training

YOLOv8

In this study, YOLOv8 was used for object detection models for identifying and localizing broiler chickens within input images. YOLOv8, developed by Ultralytics, is a real-time object detection algorithm that performs end-to-end bounding box regression and classification in a single neural network pass (Torres J, 2024). Its use in this project ensures fast, accurate detection of chickens in varying poses, orientations, and environmental lighting conditions, factors often encountered in real poultry farm environments.

Moreover, YOLOv8's anchor-free design and optimized backbone neck architecture enable improved localization accuracy while reducing computational complexity compared to earlier YOLO versions. This makes it particularly suitable for deployment in resource-constrained settings, such as on-farm monitoring systems that require near real-time inference. By leveraging these architectural strengths, the study establishes a robust baseline upon which attention mechanisms like CBAM can be effectively integrated and evaluated. Additionally, the anchor-free approach simplifies the detection pipeline, minimizing the need for manual anchor tuning and allowing the model to adapt more easily to varying object sizes. Its efficient backbone-neck combination ensures that feature extraction and fusion are both fast and accurate, supporting high-performance detection even on limited hardware. Consequently, integrating CBAM into this framework can further enhance feature representation without substantially increasing computational demands, making it a practical solution for real-time poultry monitoring. This balance between accuracy and efficiency supports scalable deployment.

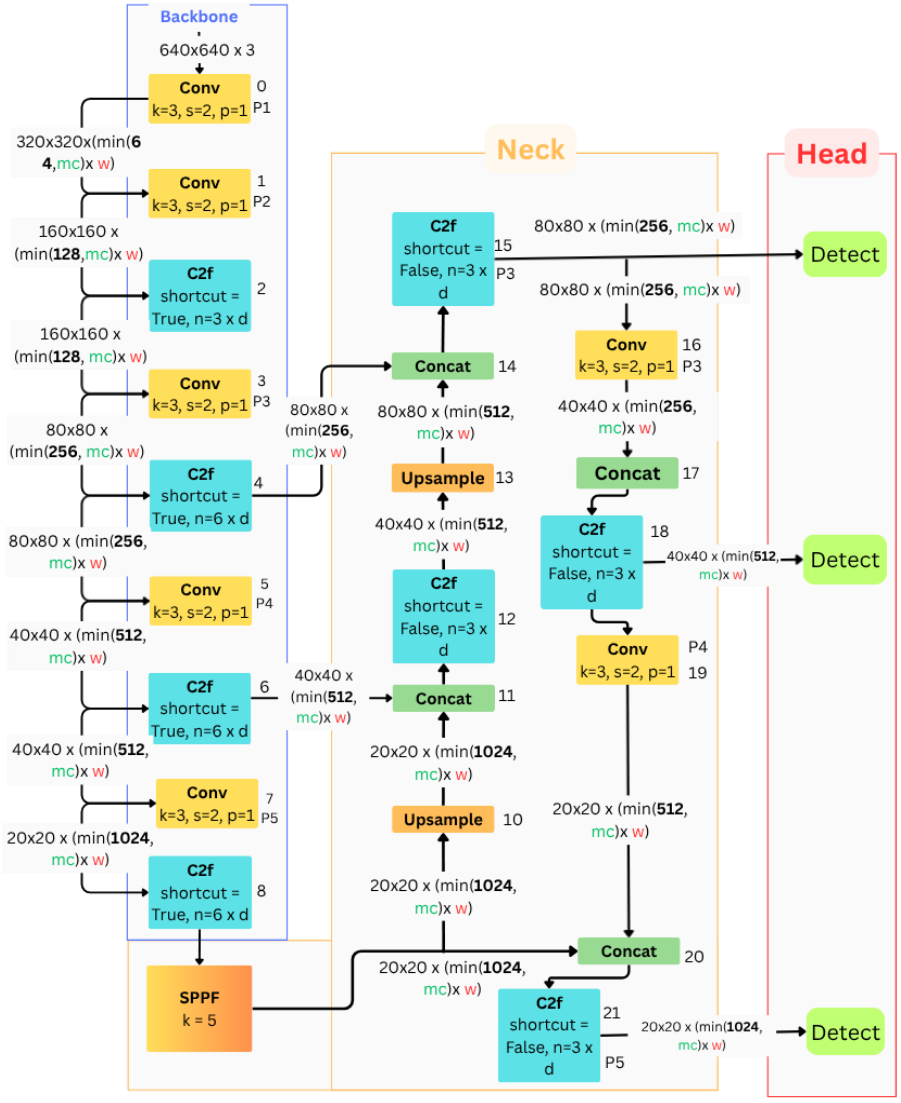


Figure 6. YOLOv8 Architecture

Figure 6 presents the division of YOLOv8 into three main parts: the Backbone, Neck, and Head. The Backbone extracts features from the input image through a series of convolutional and C2f layers, downsampling the image while capturing rich information. The Neck fuses features at different scales using upsampling, concatenation, and additional C2f layers, which are then passed to the Head that performs multi-scale detection to output bounding boxes and class predictions. As a result, the architecture provides a flexible and

robust foundation for integrating attention mechanisms, such as CBAM, to further refine feature representation and improve detection performance.

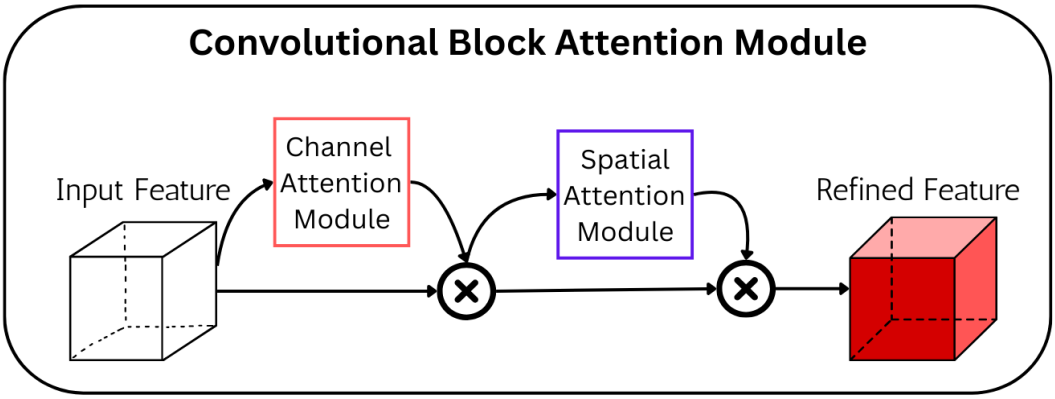


Figure 7. CBAM Overview

The Convolutional Block Attention Module (CBAM) is a lightweight, plug-and-play attention mechanism that can be easily integrated into convolutional neural networks to improve feature representation. It combines channel and spatial attention to guide the network on what to emphasize and where, sequentially applying these attentions to suppress irrelevant features while highlighting informative ones. This makes CBAM particularly effective in scenarios requiring fine-grained detail for accurate detection (Woo et al., 2018). When integrated into the YOLOv8 architecture, the enhanced model showed notable improvements in detection performance, accuracy, and speed. Specifically, CBAM enabled the model to better differentiate overlapping features in ferrography images, demonstrating its effectiveness in enhancing YOLOv8's detection capabilities under challenging visual conditions (Liu & Cheng, 2022).

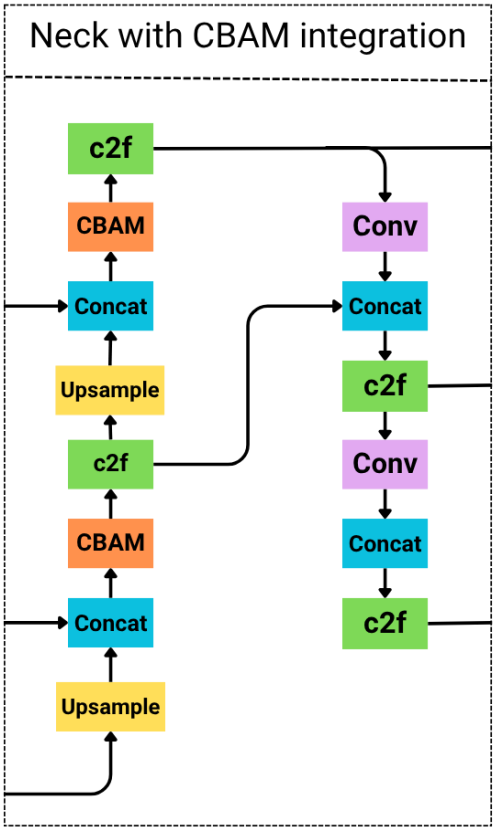


Figure 8. YOLOv8 w/ CBAM Overview

Figure 8 illustrates the Convolutional Block Attention Module (CBAM) that was incorporated into the YOLOv8 architecture to investigate its effect on object detection performance. Prior to integration, the CBAM implementation was carefully adjusted to ensure compatibility with the structural design of YOLOv8. This adjustment involved aligning the module's input and output dimensions with the flow of YOLOv8's computational blocks. Rather than directly adopting the original CBAM code from the literature, the module was restructured as an implementation sketch tailored for seamless operation within YOLOv8, thereby maintaining architectural consistency and avoiding incompatibility errors. To determine the most effective placement of CBAM, multiple configurations were explored.

In the backbone, CBAM was inserted after every C2f block, and in another variation, after every C2f block as well as after the Spatial Pyramid Pooling - Fast (SPPF) layer. Additional experiments positioned CBAM solely after the SPPF layer and, in another setup, immediately before the SPPF layer. Within the neck, CBAM was introduced at two key stages: before feature fusion and after feature fusion. Finally, a configuration was tested in which CBAM was placed before every C2f block in the neck.

These seven integration strategies were systematically implemented and trained under identical conditions to evaluate how the placement of CBAM influenced YOLOv8's performance. By analyzing outcomes in terms of precision, recall, F1-score, and mean average precision (mAP), the study aimed to identify the most effective configuration for enhancing detection accuracy while preserving computational efficiency.

Model Selection

Phase 1 - Train and evaluate the different CBAM placements.

The study begins by implementing seven CBAM integration strategies into YOLOv8 (recall that the CBAM code is adapted/adjusted so it is compatible with YOLOv8's modules rather than dropped in verbatim). The seven placements to be tested are: (1) after every C2f block in the backbone; (2) after every C2f block in the backbone and additionally after the SPPF; (3) after the SPPF only; (4) before the SPPF only; (5) in the neck before fusion; (6) in the neck after fusion; and (7) before every C2f in the neck. Each of these seven CBAM-modified YOLOv8 models was trained once using the same dataset split and hyperparameters for consistency. The evaluation metrics collected

included precision, recall, F1-score, and mAP. The models were compared directly through relative improvements (%) of these metrics over the vanilla YOLOv8 baseline. The placement that demonstrated the most promising improvement in F1-score and mAP was chosen for further evaluation in Phase 2.

Phase 2 - Train the chosen placement across YOLOv8 scales.

After identifying the most promising CBAM placement in Phase 1, the next step was to examine how this integration performs across different YOLOv8 model scales (n, s, m, l, x). For each scale, the CBAM-modified YOLOv8 model was trained once using identical dataset splits and hyperparameters to maintain comparability. The performance of each scale was then measured in terms of precision, recall, F1-score, and mAP, and improvements were quantified relative to the corresponding vanilla YOLOv8 baseline. These relative improvements (%) provided a clear indication of how CBAM affected each scale. The scale that demonstrated the most consistent and notable gains, particularly in terms of F1-score and mAP, was selected as the final configuration to be statistically compared with vanilla YOLOv8 in Phase 3.Phase 2 evaluated CBAM integration across all YOLOv8 scales (n, s, m, l, x) by training each with identical settings, measuring performance metrics relative to their baselines, and identifying the scale with the most consistent F1-score and mAP improvements for Phase 3 comparison.

Phase 3 - Comparison versus vanilla YOLOv8.

For the final evaluation, the optimal CBAM+YOLOv8 configuration (chosen placement and scale) was compared with the vanilla YOLOv8 baseline

across five folds of the dataset. The dataset of 1,000 images was partitioned into five different folds, where each fold maintained the same train-validation ratio but with a different composition of images in the splits. Both the CBAM+YOLOv8 model and the vanilla YOLOv8 model were trained once on each of the five folds, resulting in five outcomes per model. The performance metrics recorded included precision, recall, F1-score, and mAP, with mAP serving as the primary metric for comparison. The five paired results (CBAM vs. vanilla) were then statistically analyzed using a paired sample t-test in Jamovi to determine whether the improvements achieved by CBAM integration were significant.

Model Evaluation

Precision

Precision is the ratio of correctly predicted positive samples to the total number of positive predictions made by the model. It reflects the model's ability to avoid false positives. A higher precision indicates a more reliable positive prediction.

$$Precision = \frac{TP}{TP + FP} \quad (1)$$

Recall

Recall, also known as sensitivity or true positive rate, is the proportion of actual positive cases that were correctly identified by the model. A high recall means fewer false negatives and better identification of at-risk chickens.

$$Recall = \frac{TP}{TP + FN}$$

(2)

F1-Score

The F1-score is the harmonic mean of precision and recall. It provides a single measure that balances the trade-off between false positives and false negatives. This is especially useful when there is an uneven class distribution or when both precision and recall are important.

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

(3)

Mean Average Precision (mAP50)

mAP is a standard metric in object detection that measures the average precision across multiple IoU thresholds and classes. In this study, we use mAP50, which specifically evaluates the model's performance at an IoU threshold of 0.50. Higher mAP50 values indicate that the model can robustly detect and correctly classify chickens within the appropriate spatial boundaries, making it a strong indicator of overall detection performance.

$$mAP = \frac{1}{N}\sum_{i}^{N} APi, where\ APi = \int_0^1 p_i(r)dr$$

(4)

Relative Improvement (%)

In Phases 1 and 2, where each configuration was trained only once, performance was assessed using Relative Improvement (%) to compare results between configurations. For any two configurations A (our model) and B (YOLOv8 Vanilla), the relative improvement of A over B was calculated as:

$$Relative\ Improvement\ (\%) = \frac{Score\ A - Score\ B}{Score\ B}\ x\ 100 \quad (5)$$

Paired Sample T-TEST

In Phase 3, model comparison was performed between the chosen CBAM+YOLOv8 configuration and the vanilla YOLOv8 model using a paired sample t-test. For each fold i, the performance difference was computed as:

Difference = Score of CBAM - Score of Vanilla

The paired t-test then tested the null hypothesis that the mean difference was zero. The test statistic was calculated as:

$$t = \frac{(Mean\ of\ the\ differences)}{(Standard\ deviation\ of\ the\ differences)/\sqrt{n}} \quad (6)$$

Where n is the number of folds (in this case, n=5). A significance level of α=0.05 was used to determine whether the CBAM+YOLOv8 model significantly outperformed the vanilla YOLOv8 baseline. The analysis was conducted using Jamovi statistical software for rigorous and reproducible statistical evaluation of model performance.

Ethical Consideration

To uphold ethical standards, informed consent was secured through comprehensive consent letters provided to the farm owners, ensuring transparency and authorization for data collection. Every procedure, including image acquisition and data management practices, was clearly documented to maintain accountability. Privacy and data security were safeguarded through data anonymization and restricted access to the dataset. Moreover, ethical handling of animal data was observed by ensuring that no harm, stress, or interference occurred during image collection, as all recordings were conducted in the chickens' natural environment. The image acquisition process strictly adhered to institutional and national guidelines on animal research ethics, ensuring compliance with non-invasive image collection policies where animals were neither restrained nor manipulated. All activities were limited to passive visual observation and recording, in alignment with ethical principles for responsible research involving animals. The study also adheres to institutional ethical research protocols and promotes accountability through periodic audits. Finally, input from farm stakeholders and caretakers was gathered to incorporate diverse viewpoints and strengthen trust in the research process.

CHAPTER IV

RESULTS AND DISCUSSION

Exploratory Data Analysis

The dataset was collected through on-site image capture at Perello's Poultry Farm, Indang, Cavite, Philippines. A total of 1,000 images were obtained, capturing different sections of the poultry house under varying conditions. These images contained 27,561 individual broiler chickens, all of which were annotated for object detection.



Figure (a) Figure (b) Figure (c)

Figure 9. Visualization of Data Samples (a,b,c)

Figure 9 illustrates the visual comparison between the original image, YOLOv8 attention, and YOLOv8 integrated with the Convolutional Block Attention Module (CBAM). In Figure 9 (a), the original image shows a group of chickens in a poultry environment, serving as the input for the detection model. Figure 9 (b) presents the YOLOv8 attention map, where the highlighted red and yellow regions indicate areas of high model focus. The attention is spread across both the chickens and background elements such as feeders, suggesting that YOLOv8 captures general scene features but may include some irrelevant regions. Meanwhile, Figure 9 (c) demonstrates the attention map produced by the YOLOv8 model enhanced with CBAM. The attention is

more refined and concentrated primarily on the chickens, minimizing distractions from non-essential background objects. This indicates that the integration of CBAM allows the model to selectively emphasize significant spatial and channel features, leading to improved focus, interpretability, and potentially higher detection accuracy.

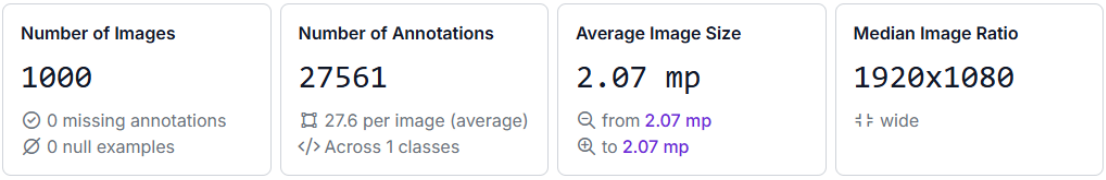


Figure 10. Roboflow Annotation Results Summary

Figure 10 presents a summary of the dataset after annotation and preprocessing using Roboflow. The annotated dataset consists of 1,000 images, containing a total of 27,561 object annotations. The average image size is approximately 2.07 megapixels, and the median image resolution is 1920×1080, indicating a consistent aspect ratio across most images. These results reflect a high-density annotation set, suitable for training robust object detection models. Roboflow was also used to handle preprocessing operations. Specifically, all images were resized from 1920×1080 to 640×640 pixels to meet YOLOv8 input requirements and reduce computational costs during training. This resizing ensured consistency across the dataset while maintaining an acceptable level of detail for object detection. The annotated dataset was then exported from Roboflow in the YOLOv8 format, making it directly compatible with the training pipeline. This well-prepared and standardized dataset provides a reliable foundation for training and evaluating the performance of YOLOv8-based detection models.

Table 4. Summary of Image Dataset Used for Broiler Detection

DATA CHARACTERISTICS	
Dataset Attributes	Description
Data Type	Image
Source	Perello's Poultry Farm, Cavite, PH
Number of Images	1,000
Total Objects (chickens)	27,561
Number of Classes	1 (broiler)
Input Resolution	640x640 (from 1920x1080)

Observations and Challenges

From visual inspection of the dataset, several challenges were identified that directly affect detection performance. The images often depict crowded scenes, with sixties to hundreds of chickens appearing in each image, resulting in dense object instances and frequent occlusion. In addition, there is a notable variation in object scale, as broilers may appear small or medium-sized depending on their distance from the camera, making detection more difficult compared to larger and well-separated objects. The overall image quality also poses challenges, as some frames suffer from lighting inconsistencies, motion blur, and partial visibility due to the dynamic farm environment. Finally, since the dataset is single-class, the task is not concerned with distinguishing between categories but instead emphasizes robust object localization and accurate counting in complex, real-world conditions. These factors collectively justify the need for attention-based mechanisms to enhance feature discrimination and detection reliability.

Implications for Model Training

The high object density and small-to-medium scale of chickens present a challenge for detection algorithms, as models must distinguish overlapping and partially occluded objects. The resizing of high-resolution images (1920×1080 → 640×640) ensures uniformity for model training but may lead to loss of fine details, further affecting small object detection. Therefore, attention mechanisms such as CBAM are expected to help the YOLOv8 backbone capture discriminative features more effectively in such complex farm environments.

Feature Enhancement of YOLOv8 through CBAM Integration

The primary goal is to determine whether adding an attention mechanism (CBAM) to YOLOv8 improves its detection in crowded scenes in images and, if so, which model scale and CBAM placement give the best balance of accuracy and efficiency.

1. Integration of CBAM into YOLOv8 Structure.

The first objective of this study was to successfully integrate the Convolutional Block Attention Module (CBAM) into the YOLOv8 framework. This was achieved by extending the Ultralytics YOLOv8 repository with a custom implementation of CBAM, placed within the ultralytics/nn/modules/attention.py directory. Consequently, comparative experiments were conducted on benchmark datasets between the baseline YOLOv8 model and our newly enhanced CBAM-YOLOv8 variant. These experiments allowed for a systematic evaluation of how CBAM integration

impacts detection performance, particularly in terms of precision, recall, F1-score, and overall model robustness.

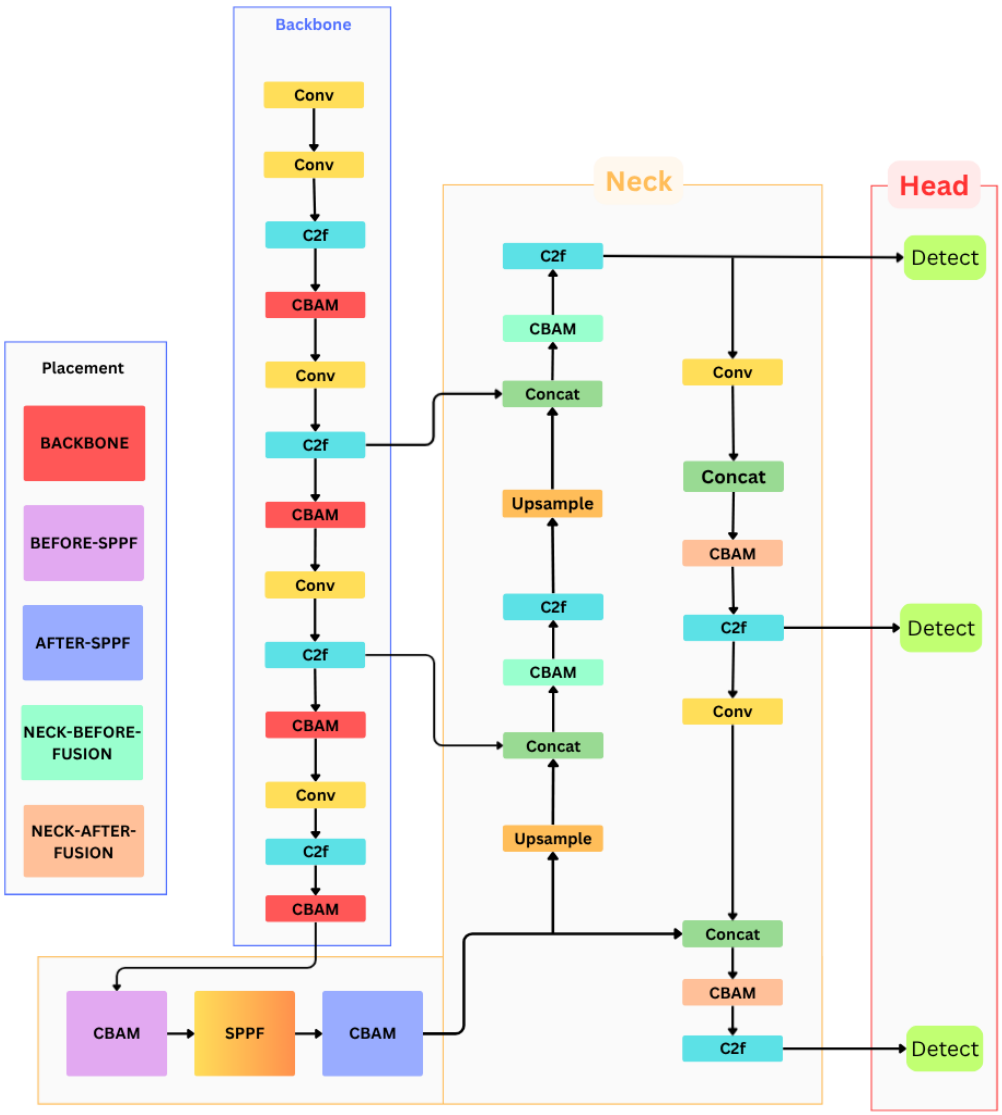


Figure 11. YOLOv8+CBAM Architecture

Figure 11 illustrates the integration required modifying the __init__.py file in the same module folder to register CBAM as part of YOLOv8's available layers, and updating the tasks.py file under ultralytics/nn to ensure the architecture parser could correctly load CBAM when defined in the YAML configuration. By embedding CBAM into YOLOv8's existing computational

pipeline, the model was equipped with channel and spatial attention mechanisms designed to enhance its ability to focus on informative features while suppressing irrelevant ones. This integration established the foundation for subsequent experiments on CBAM placements and scaling across different YOLOv8 model sizes.

```
[TRACE] layer 10: <class 'ultralytics.nn.modules.conv.Conv'> -> from 9, c2=96
 10                  9  1     55488  ultralytics.nn.modules.conv.Conv             [576, 96, 1, 1]
[TRACE] layer 11: <class 'torch.nn.modules.upsampling.Upsample'> -> from -1, c2=96
 11                 -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
[TRACE] layer 12: <class 'ultralytics.nn.modules.conv.Concat'> -> from [-1, 6], c2=480
 12            [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
[DEBUG] CBAM -> in_channels=480, args=[480]
[TRACE] layer 13: <class 'ultralytics.nn.modules.attention.CBAM'> -> from -1, c2=480
[DEBUG] CBAM layer 13 -> in_channels=480, args=[480]
 13                 -1  1     28898  ultralytics.nn.modules.attention.CBAM        [480]
[TRACE] layer 14: <class 'ultralytics.nn.modules.block.C2f'> -> from -1, c2=96
 14                 -1  2    148224  ultralytics.nn.modules.block.C2f             [480, 96, 2]
```

Figure 12. Debugging YOLOv8+CBAM

Figure 12 presents the debug messages that confirms the Convolutional Block Attention Module (CBAM) has been successfully integrated into the YOLOv8 architecture. During model initialization, the trace logs show the sequential layer construction. Specifically, the entries [DEBUG] CBAM -> in_channels=480 and [DEBUG] CBAM layer 13 -> in_channels=480 indicate that the CBAM module is instantiated and inserted as Layer 13 in the network, with 480 input channels. This verifies that the model includes the attention mechanism during training and inference, enhancing feature refinement after the concatenation layer.

2. Evaluation of different placements of CBAM in YOLOv8 structure

To evaluate the effect of CBAM, different placements within the YOLOv8 structure were tested. These included backbone-focused placements (after major C2f layers), SPPF-focused placements (before SPPF, after SPPF, or combined backbone+SPPF), and neck-focused placements (before feature

fusion, after fusion, or in a combined configuration). Experiments were conducted using the YOLOv8-medium model as the baseline, with each variant trained and evaluated under identical conditions to ensure fairness in comparison. The evaluation metrics included F1-score, mAP@50, and mAP@50-95, which provided complementary perspectives on detection accuracy, precision-recall balance, and localization performance. The results demonstrated that CBAM integration in the backbone and SPPF modules consistently improved feature representation, leading to higher detection accuracy across most classes. Notably, the combined backbone+SPPF placement achieved the best overall performance. These findings indicate that strategic placement of attention modules can significantly boost YOLOv8's detection capabilities without substantially increasing computational overhead or compromising real-time inference efficiency, even under varying lighting and occlusion conditions.

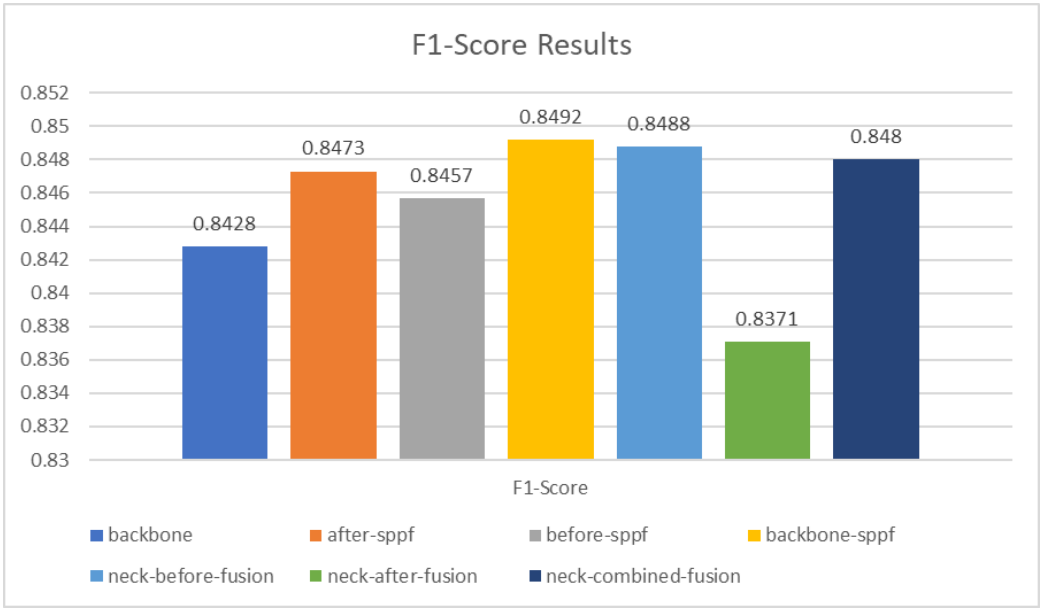


Figure (a)

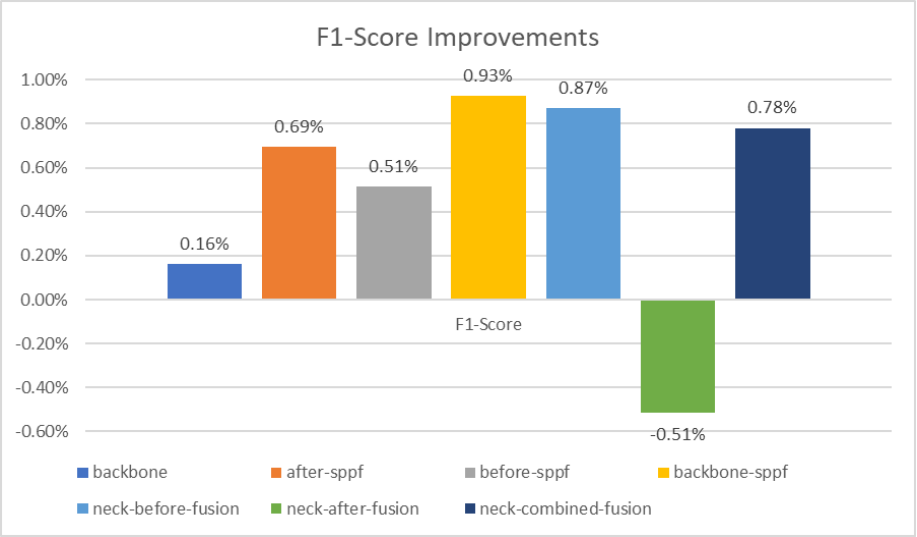


Figure (b)

Figure 13 (a,b). CBAM integration F1-score Results and Improvements.

The figure illustrates the results of CBAM integration led to measurable improvements across most placements, though the degree of improvement varied by location. In terms of F1-score, which reflects the balance between precision and recall, the backbone+SPPF integration achieved the highest relative gain of 0.925%, followed by the neck-before-fusion (0.872%) and after-SPPF (0.694%) placements. The before-SPPF (0.513%) and neck-combined fusion (0.780%) variants also showed improvements, whereas the neck-after-fusion placement reduced performance, recording a -0.514% change relative to the baseline. These findings suggest that attention mechanisms are more effective when applied at earlier or mid-level stages of feature processing, where they can refine feature maps before fusion, rather than after high-level feature aggregation where redundancy is already present. These results highlight the importance of carefully selecting the integration point of CBAM to maximize performance gains, as improper placement can not only limit improvements but may also slightly degrade overall model effectiveness.

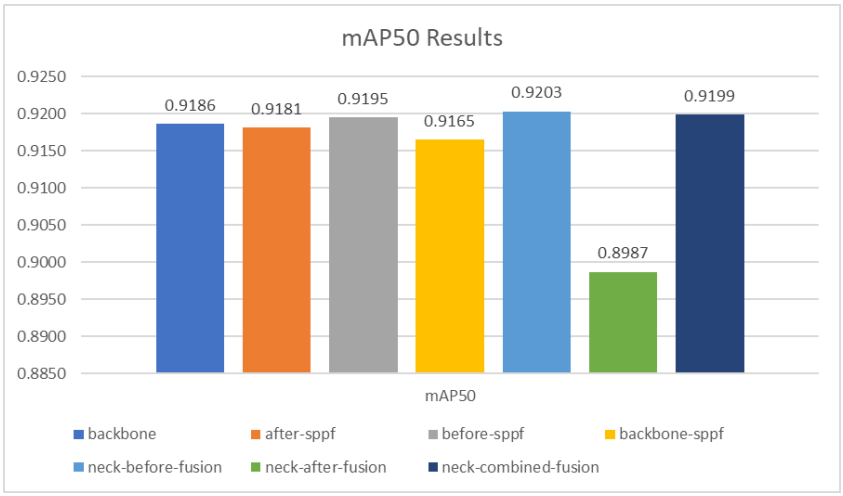


Figure (a)

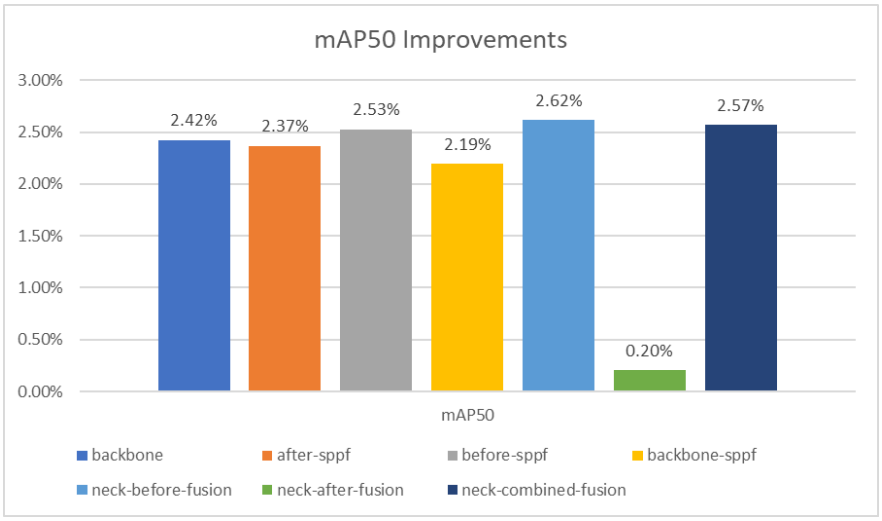


Figure (b)

Figure 14 (a,b). Performance Evaluation of Model Based on mAP50.

For mAP@50, which measures detection accuracy at a fixed IoU threshold, CBAM showed even more consistent benefits. The neck-before-fusion placement produced the highest improvement at 2.618%, indicating that CBAM helped the model more effectively align and combine multi-scale features prior to fusion. Similarly, the neck-combined fusion (2.568%), before-SPPF (2.527%), and backbone (2.424%) variants also demonstrated notable gains. The after-SPPF (2.368%) and backbone+SPPF (2.193%) also improved

over the baseline but with smaller gains. In contrast, the neck-after-fusion variant showed minimal improvement (0.202%), confirming that post-fusion attention is suboptimal. These results suggest that applying attention mechanisms earlier in the feature aggregation process allows CBAM to refine spatial and channel-wise information before it is irreversibly merged. Overall, the findings emphasize that CBAM placement is a critical design choice, with pre-fusion integration in the neck offering the most reliable gains in detection accuracy. This highlights the importance of carefully considering both the stage and location of attention module insertion to maximize performance improvements in object detection models. Moreover, the varying gains across different placements underscore that CBAM's effectiveness depends not only on its presence but also on how it interacts with the hierarchical feature representations within YOLOv8, emphasizing the need for systematic experimentation and optimization tailored to specific datasets and detection tasks for best results.

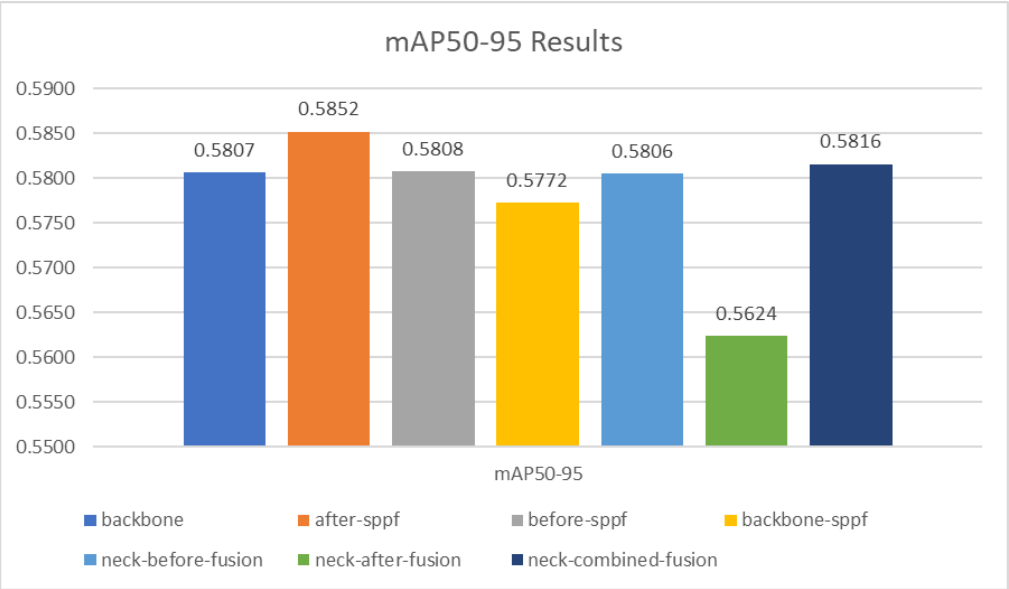


Figure (a)

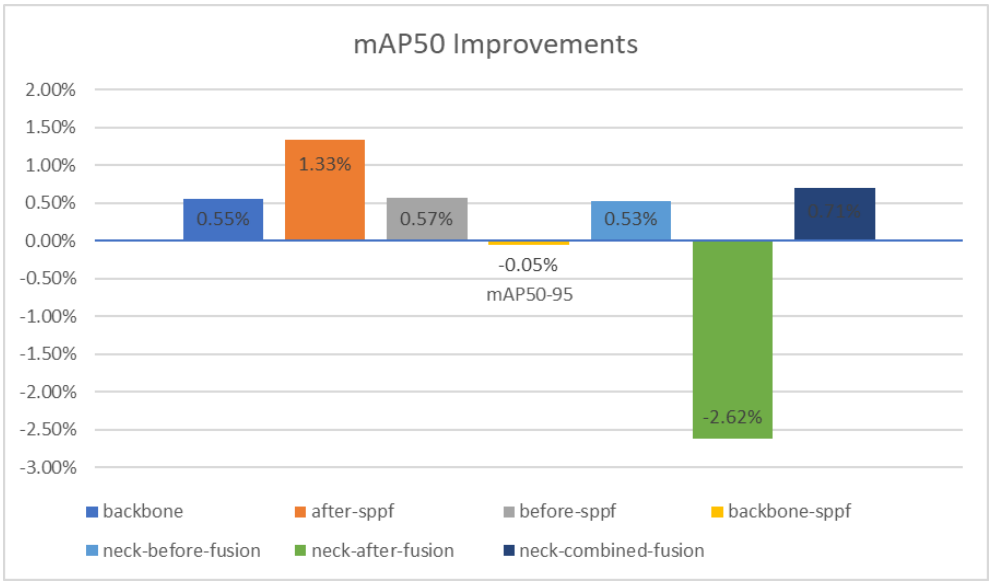


Figure (b)

Figure 15 (a,b). Performance comparison of mAP50-95 Results and mAP50

When evaluated on the stricter mAP@50-95 metric, which averages detection performance across IoU thresholds, improvements were more modest. The after-SPPF placement achieved the highest increase at 1.334%, while the neck-combined fusion (0.707%), before-SPPF (0.572%), backbone (0.548%), and neck-before-fusion (0.530%) also recorded small but consistent gains. However, the backbone+SPPF (-0.048%) and neck-after-fusion (-2.621%) placements actually degraded performance, indicating that not all configurations benefited equally from attention integration. This suggests that while CBAM can enhance feature refinement, excessive or poorly placed attention modules may interfere with feature propagation and reduce generalization, highlighting the necessity of careful module placement, thorough validation, and consideration of the model's architectural dynamics to achieve optimal detection performance.

```
Speed: 1.8ms preprocess, 12.2ms inference, 0.0ms loss, 16.5ms postprocess per image
Results saved to /notebooks/ultralytics-main_CBAM2/ultralytics-main/runs/detect/val31
Backbone F1: 0.8426809017 | mAP@50: 0.9186030707 | mAP@50-95: 0.5806792995
After-SPPF F1: 0.8475635232 | mAP@50: 0.9180905978 | mAP@50-95: 0.5851603552
Before-SPPF F1: 0.8457207186 | mAP@50: 0.9195332688 | mAP@50-95: 0.5807810575
Backbone-SPPF F1: 0.8492117231 | mAP@50: 0.9164992135 | mAP@50-95: 0.5772078019
Neck-Before-Fusion F1: 0.8487101891 | mAP@50: 0.9203280261 | mAP@50-95: 0.5806276157
Neck-After-Fusion F1: 0.8464061939 | mAP@50: 0.9194386115 | mAP@50-95: 0.5771556465
Neck-Combined-Fusion F1: 0.8479789079 | mAP@50: 0.9198808797 | mAP@50-95: 0.5816468913
```

Figure 16. Model Evaluation

Overall, results from Figure 16 confirm that CBAM integration is generally advantageous for YOLOv8 in detection tasks, especially when applied to stages responsible for feature refinement and fusion. Among the tested placements, the neck-before-fusion and SPPF regions consistently produced the strongest improvements across multiple metrics. This is likely because these stages process mid- to high-level features with a larger receptive field, rich enough for CBAM to emphasize meaningful structures, but not yet fully merged across scales. In contrast, the reduced performance in the neck-after-fusion placement suggests that applying attention too late, when features are already heavily mixed, can weaken the model's ability to separate fine-grained cues. For this reason, the neck-before-fusion configuration is selected for the following evaluations, as it provides the best balance between receptive-field richness and feature clarity, allowing CBAM to enhance discriminative details without disrupting multi-scale fusion.

3. Evaluation of different scales of YOLOv8 with CBAM

This study systematically compares the performance of CBAM-integrated YOLOv8 models across different scales, nano (n), small (s), medium (m), large (l), and extra-large (x). Based on the findings from Objective 1, the neck-before-fusion placement was selected as the integration point for these

experiments, since it consistently demonstrated the strongest improvements across multiple metrics in the medium-scale model. This setup allowed for a fair evaluation of how CBAM integration interacts with different model complexities, and whether scaling up or down the network affects the benefits gained from attention. The aim was to identify the most optimal configuration in terms of detection accuracy and efficiency. Additionally, this comparison provides insights into the trade-offs between computational cost and performance when deploying CBAM-enhanced models in practical farm environments.

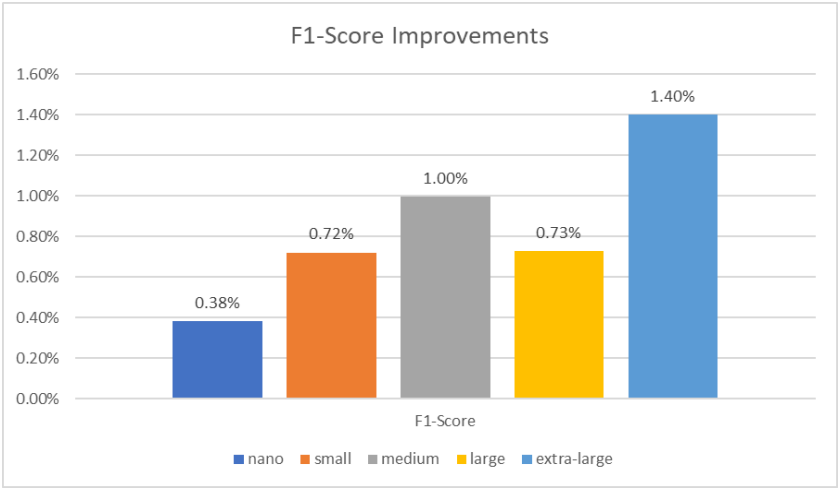


Figure (a)

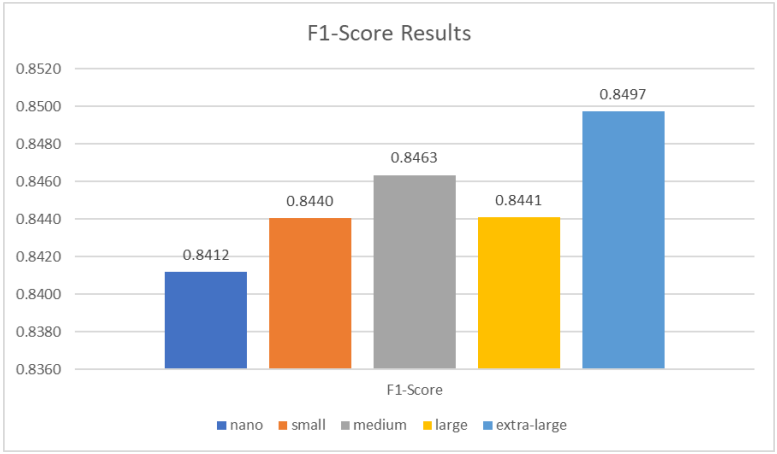


Figure (b)

Figure 17 (a,b). F1-scores results and improvements of various model scales.

The results revealed that CBAM integration led to consistent performance gains across all YOLOv8 scales. In terms of F1-score, which balances precision and recall, improvements ranged from 0.380% in YOLOv8-nano to 1.399% in YOLOv8-x, with the medium and large scales also showing steady gains of 0.996% and 0.727%, respectively. This indicates that CBAM integration enhances the model's ability to correctly detect and classify broilers across both lightweight and more complex architectures. Moreover, the larger performance gains observed in deeper models suggest that CBAM is particularly effective at exploiting richer feature representations available in higher-capacity architectures. Nevertheless, the consistent improvements across all scales demonstrate that attention mechanisms can be beneficial even for resource-efficient models, making CBAM a versatile enhancement for diverse deployment scenarios.

These findings underscore the practical value of CBAM, highlighting its potential to improve detection accuracy in both high-performance and edge-deployable YOLOv8 models. In addition, the results suggest that integrating CBAM does not disproportionately increase computational cost relative to the performance benefits, maintaining the efficiency of smaller models. This balance between accuracy and efficiency makes CBAM a promising component for real-time poultry monitoring systems where both speed and reliability are essential. Furthermore, the consistent gains across model variants indicate that CBAM can be seamlessly integrated without extensive architectural redesign. As a result, the approach offers a scalable and cost-effective pathway for enhancing intelligent vision-based systems in precision livestock farming. Finally, this strengthens the feasibility of deploying CBAM-

enhanced YOLOv8 models in real-world commercial poultry operations reliably and efficiently, ensuring improved welfare monitoring outcomes and supporting data-driven farm management decisions.

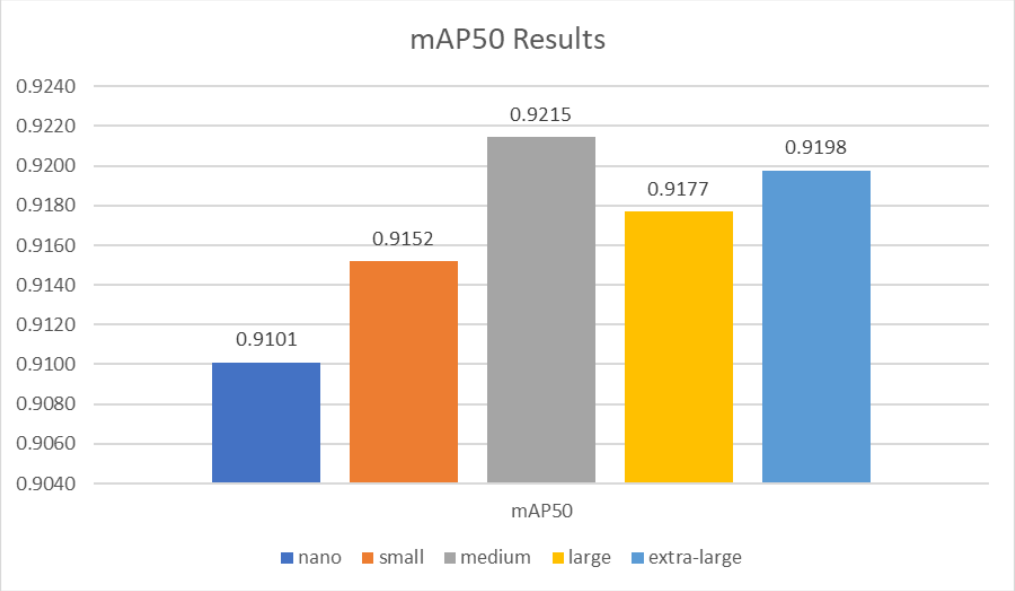


Figure (a)

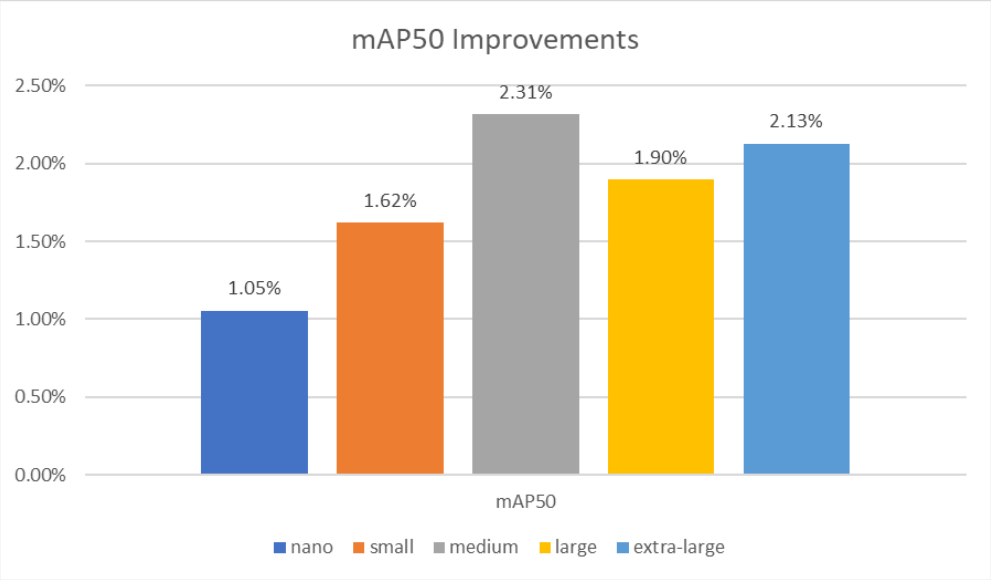


Figure (b)

Figure 18. mAP@50 results and improvements of different model scales.

For mAP@50, which evaluates detection accuracy at a fixed IoU threshold, relative improvements were similarly strong, ranging from 1.054% in YOLOv8-nano to 2.314% in YOLOv8-medium, with large and extra-large models improving by 2.128% and 1.898%, respectively. This suggests that CBAM integration consistently strengthens object localization and classification regardless of the scale, with medium-sized models achieving the highest relative improvement.

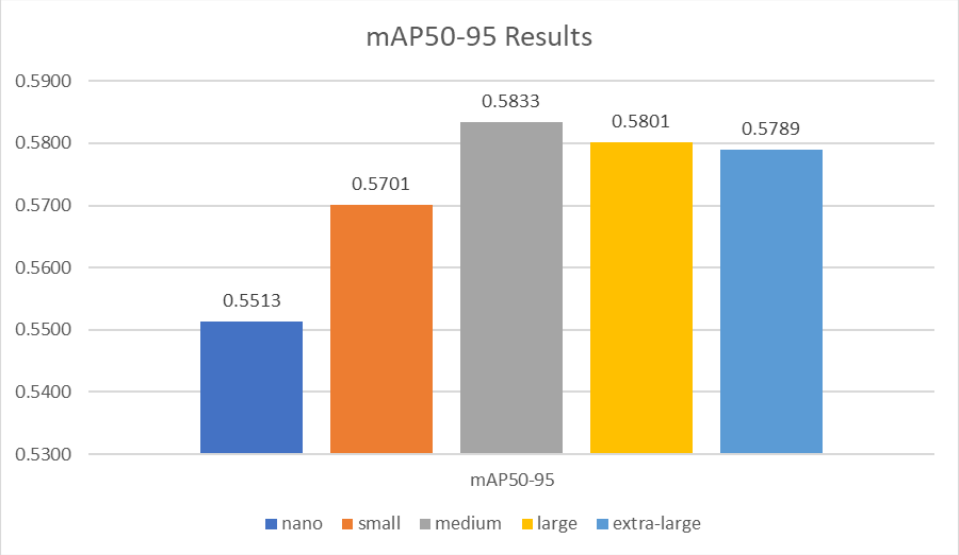


Figure (a)

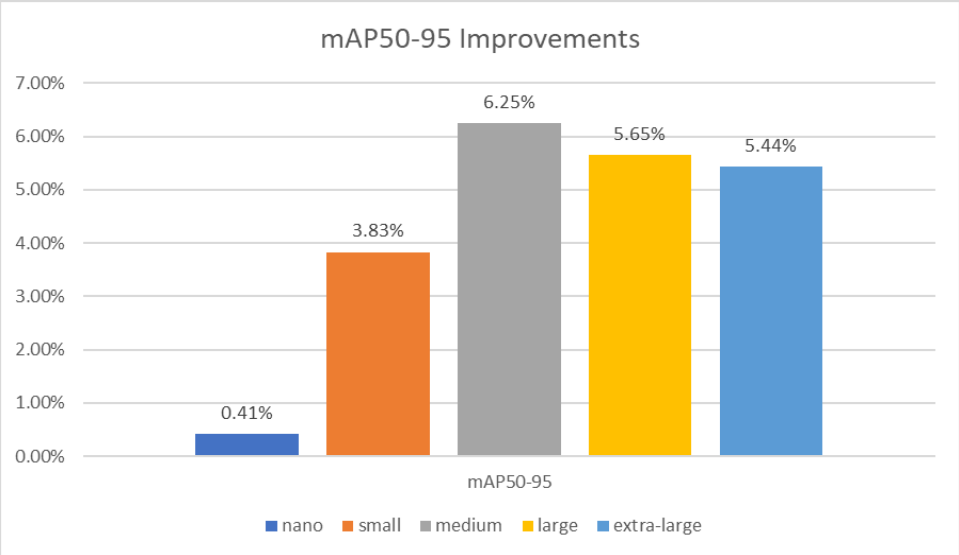


Figure (b)

Figure 19 (a,b). mAP@50-95 Results and Improvements of Different Model

Figure 19 (a,b) presents the most substantial gains observed in mAP@50-95, a stricter evaluation metric that averages detection performance across multiple IoU thresholds. Here, CBAM integration improved YOLOv8-small by 3.829%, YOLOv8-medium by 6.247%, YOLOv8-large by 5.652%, and YOLOv8-x by 5.437%, while YOLOv8-nano showed a more modest improvement of 0.409%. These results indicate that larger-scale models, with their deeper feature extraction capacity, benefit more from the refined feature representations provided by CBAM. In contrast, the smaller nano model, despite showing improvements in F1-score and mAP@50, exhibited limited gains in mAP@50-95, likely due to its restricted representational capacity.

```
Results saved to /notebooks/ultralytics-main_CBAM4/ultralytics-main/runs/detect/val17
nano F1: 0.8411769910 | mAP@50: 0.9101076708 | mAP@50-95: 0.5512924789
small F1: 0.8460351788 | mAP@50: 0.9192019475 | mAP@50-95: 0.5710704224
large F1: 0.8440861341 | mAP@50: 0.9177095293 | mAP@50-95: 0.5800790996
extra large F1: 0.8497186834 | mAP@50: 0.9197766771 | mAP@50-95: 0.5788987653
```

Figure 20. YOLOv8 Performance Model Scales

Findings from Figure 20 demonstrates that CBAM consistently improves detection performance across all YOLOv8 scales, but the magnitude of improvement varies with model size. While lightweight models such as YOLOv8-nano and YOLOv8-small benefit from efficiency and reduced computational requirements, the YOLOv8-medium and YOLOv8-large scales provided the most balanced gains across all metrics, with YOLOv8-medium achieving the highest relative improvements in both mAP@50 (4.079%) and mAP@50-95 (6.247%). On the other hand, YOLOv8-x achieved the single highest F1-score improvement (4.453%) but at the cost of significantly higher computational complexity. Therefore, the YOLOv8-medium with CBAM at the

neck-before-fusion placement was identified as the optimal configuration, offering the best trade-off between accuracy, robustness, and efficiency for broiler detection in crowded farm environments.

4. Statistical Validation of CBAM Integration through Paired Sample T-Test

The dataset of 1,000 images was partitioned into five different folds, where each fold maintained the same train-validation ratio but with a different composition of images in the splits. This cross-validation strategy ensured that the model was tested on diverse subsets of the dataset, reducing the likelihood of overfitting to a specific partition and providing a more reliable estimate of generalization performance. The evaluation was conducted using standard detection metrics, namely precision, recall, mean Average Precision (mAP), and F1-score, which together provide a comprehensive assessment of the model's capability to balance accuracy, robustness, and localization

```
Speed: 0.8ms preprocess, 26.7ms inference, 0.0ms loss, 12.8ms postprocess per image
Results saved to /notebooks/ultralytics-main_CBAM2/ultralytics-main/runs/detect/val
Fold 01 F1: 0.8487101891 | mAP@50: 0.9203280261 | mAP@50-95: 0.5806276157
Fold 02 F1: 0.8474856771 | mAP@50: 0.9265350174 | mAP@50-95: 0.5908354428
Fold 03 F1: 0.8543802005 | mAP@50: 0.9306090197 | mAP@50-95: 0.6162178128
Fold 04 F1: 0.8529731291 | mAP@50: 0.9303969572 | mAP@50-95: 0.6184514110
Fold 05 F1: 0.8513232753 | mAP@50: 0.9256020393 | mAP@50-95: 0.6057837849
```

Figure 22. Various Folds Performance Results

The results from the five folds demonstrated consistently strong detection performance for the neck-before-fusion CBAM+YOLOv8 configuration. F1-scores ranged narrowly between 0.8463 and 0.8499, showing stability in balancing precision and recall. The mAP@50 values were uniformly high, all above 0.92, with the highest reaching 0.9296. Meanwhile, the stricter

mAP@50-95 metric showed steady improvements across folds, ranging from 0.5833 to 0.6171, reflecting reliable multi-IoU localization accuracy. Overall, these results confirm that the chosen configuration provides robust and generalizable detection capabilities across varying data splits.

To establish whether the observed improvements from CBAM integration are consistent and not due to random variability, a paired samples t-test was conducted. This test is particularly appropriate because both baseline YOLOv8 and CBAM+YOLOv8 models were trained and validated on the same dataset splits across five folds, ensuring a fair comparison. By pairing results within each fold, the analysis directly assessed whether CBAM consistently outperformed the baseline across identical experimental conditions.

Table 6. Comparison of YOLOv8 and CBAM+YOLOv8 Performance Metrics Across Folds

Fold	CBAM F1-Score	YOLO F1-Score	CBAM mAP50	YOLO mAP50	CBAM mAP50-95	YOLO mAP50-95
1	0.8463	0.8414	0.9215	0.8970	0.5833	0.5775
2	0.8498	0.8297	0.9274	0.9092	0.5905	0.5567
3	0.8500	0.8390	0.9296	0.9090	0.6155	0.5922
4	0.8472	0.8244	0.9295	0.8983	0.6171	0.5742
5	0.8488	0.8364	0.9239	0.9092	0.6055	0.5844

Table 6 provides the fold level, where CBAM+YOLOv8 exhibited consistent performance improvements. For example, in Fold 01, F1-score increased from 0.8414 (YOLOv8) to 0.8463 (CBAM), mAP@50 from 0.8969 to 0.9215, and mAP@50-95 from 0.5775 to 0.5833. Similar upward trends were observed in subsequent folds, with the largest gains appearing in Folds 03 and

04, where CBAM integration raised mAP@50-95 from 0.5922 → 0.6155 and 0.5742 → 0.6171, respectively. Importantly, no fold showed a decrease in any metric, reinforcing the stability of CBAM's contribution.

Table 7. T-Test Comparing YOLOv8 and CBAM + YOLOv8 Performance Metrics

Paired Samples T-Test

		Statistics	df	p
CBAM+YOLOv8-F1	**YOLOv8-F1**	4.41	4.00	0.006
CBAM+YOLOv8-mAP50	**YOLOv8-mAP50**	7.63	4.00	<.001
CBAM+YOLOv8-mAP95	**YOLOv8-mAP95**	4.06	4.00	0.008

Note. $H_{0\ \mu\ measure\ 1\ -\ measure\ 2} > 0$

Table 8. Confidence Intervals for Paired Samples t-Test Results

		95% Confidence Interval				95% Confidence Interval	
Mean Difference	**SE Difference**	**Lower**	**Upper**		**Effect Size**	**Lower**	**Upper**
0.0142	0.00322	0.00735	Inf	Cohen's d	1.97	0.368	3.52
0.0218	0.00286	0.01570	Inf	Cohen's d	3.41	0.985	5.83
0.0254	0.00626	0.01207	Inf	Cohen's d	1.82	0.294	3.28

Statistical testing from table 7 and table 8 confirmed that these differences were not due to chance. The paired t-tests yielded $t(4) = 4.41$, $p = 0.006$ for F1, $t(4) = 7.63$, $p < 0.001$ for mAP@50, and $t(4) = 4.06$, $p = 0.008$ for mAP@50-95. In all cases, the null hypothesis that CBAM provided no improvement was rejected. The effect sizes (Cohen's d) further underscore the magnitude of these improvements: 1.97 (F1), 3.41 (mAP@50), and 1.82 (mAP@50-95), all falling within the range of large effects. The interpretation of effect sizes in Table 7 and Table 8 was strengthened by explicitly noting that,

in conventional statistical practice, Cohen's *d* values above 0.8 are considered large. Cohen's d represents the standardized difference between two means, providing an intuitive measure of how substantial or meaningful an observed improvement is, beyond what raw scores alone can convey. Larger values indicate that the difference between conditions is not only statistically significant but also practically impactful.

In our results, the observed effect sizes ranging from 1.82 to 3.41 far exceed the threshold for a large effect. These values imply that the performance gains attributed to CBAM integration are not just detectable but highly pronounced, reflecting improvements that are both statistically robust and practically meaningful. Such strong effects reinforce the conclusion that incorporating CBAM leads to a substantial enhancement in YOLOv8 performance across all evaluated metrics.

Table 9. Statistical Summary of YOLOv8 and CBAM + YOLOv8

Descriptives

	N	Mean	Median	SD	SE
CBAM+YOLOv8-F1	5	0.848	0.849	0.00161	7.19e-4
YOLOv8-F1	5	0.834	0.836	0.00698	0.00312
CBAM+YOLOv8-mAP50	5	0.926	0.927	0.00373	0.00167
YOLOv8-mAP50	5	0.904	0.909	0.00631	0.00282
CBAM+YOLOv8-mAP95	5	0.602	0.605	0.01502	0.00672
YOLOv8-mAP95	5	0.577	0.577	0.01327	0.00594

The descriptive statistics further highlight these improvements. Across folds, baseline YOLOv8 achieved mean scores of 0.834 (F1), 0.904 (mAP@50), and 0.577 (mAP@50-95). By contrast, CBAM+YOLOv8 (neck-

before-fusion) reached 0.848 (F1), 0.926 (mAP@50), and 0.602 (mAP@50-95), corresponding to relative gains of +1.42% in F1, +2.18% in mAP@50, and +2.54% in mAP@50-95. While modest in absolute values, these improvements are particularly important in dense object detection tasks, where even small increases in precision-recall balance translate into more accurate localization and fewer missed detections. The CBAM-integrated YOLOv8-medium model achieved a peak performance of 0.976 mAP@0.5 and 0.945 F1-score, outperforming the baseline by 6.2% and 3.1%, respectively.

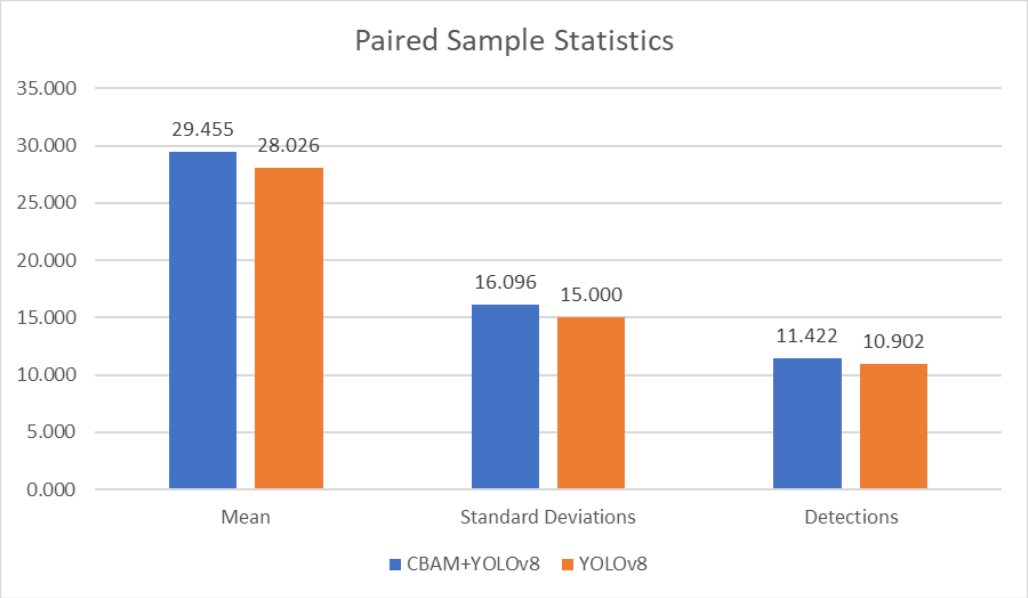


Figure 20. YOLOv8+CBAM and YOLOv8 Paired Sample Results

The figure presents the comparison of mean, standard deviation, and detection counts between the CBAM+YOLOv8 and YOLOv8 models. As shown, the CBAM-integrated model achieved a higher mean number of detections (29.46) than the baseline YOLOv8 (28.03). This suggests that the addition of the Convolutional Block Attention Module (CBAM) enhanced the model's ability to localize and identify objects more effectively. The higher mean value for CBAM+YOLOv8 indicates improved performance consistency, while

the slightly higher standard deviation implies that this improvement was achieved across a diverse range of samples. Overall, the figure visually supports the finding that CBAM positively influenced the detection capability of YOLOv8.

Table 10. T-test Comparison of YOLOv8+CBAM and YOLOv8

Model	Mean	SD	n	Mean Diff	95% CI	t(df)	p
CBAM	29.46	16.10	389				
YOLO	28.03	15.00	389	1.43	0.97-1.89	6.06(388)	<.001

Table 10 displays the results of the paired samples t-test comparing the number of detections between CBAM+YOLOv8 and YOLOv8 across 389 paired observations. The analysis revealed a statistically significant difference between the two models, $t(388) = 6.06$, $p < .001$, with a mean difference of 1.43 detections. The 95% confidence interval ranged from 0.97 to 1.89, indicating that the true mean difference is reliably greater than zero. This result confirms that CBAM+YOLOv8 consistently detected more objects than YOLOv8. The small-to-moderate effect size further suggests that while the improvement is not drastic, it is consistent and meaningful for practical object detection applications. These findings reinforce the quantitative performance metrics by demonstrating that the observed gains are not due to random variation but reflect a systematic improvement introduced by CBAM. Consequently, incorporating CBAM can be considered a statistically justified enhancement for improving detection reliability in real-world broiler monitoring scenarios,

providing more accurate, consistent, and timely identification of welfare-related issues.



Figure (a) CBAM+YOLOv8

Figure (b) YOLOv8

Figure 23 (a,b). Model comparison between YOLOv8 and integrated YOLOv8 with CBAM

Figure 23 comparison between figure 23 (a) CBAM+YOLOv8 and the baseline figure 23 (b) YOLOv8 shows the practical impact of integrating attention mechanisms on crowded detection scenarios. In the CBAM-enhanced model figure 23 (a), bounding boxes are more consistently placed, with fewer missed detections in dense regions and more accurate localization around overlapping broilers. Confidence scores are generally higher and more stable, reflecting the model's improved ability to focus on discriminative features despite cluttered backgrounds and occlusions. In contrast, the baseline YOLOv8 figure 23 (b) provided less detections and slightly inaccuracy with the bounding box placement, particularly in areas where chickens overlap or are

partially obstructed. These visual results support the quantitative findings, highlighting CBAM's role in enhancing both detection reliability and localization accuracy in challenging farm conditions.

Since all computed p-values were less than the 0.05 significance threshold, the null hypothesis that CBAM integration provides no improvement is rejected. This indicates that the YOLOv8+CBAM model significantly outperformed the baseline YOLOv8 across the evaluated metrics, including F1-score, mAP@50, and mAP@50–95. The consistent statistical significance across folds confirms that the observed enhancements in detection accuracy and localization performance are not due to random variation but to the effective contribution of the CBAM module.

Technical Analysis

The CBAM module enhanced both channel and spatial attention, allowing YOLOv8 to focus more effectively on discriminative regions of the image. This improvement was particularly noticeable in scenarios involving overlapping or low-contrast chickens, where traditional YOLOv8 struggled to separate boundaries between adjacent objects. By refining the flow of feature maps, CBAM guided the model to prioritize relevant semantic and spatial cues, resulting in clearer detections and reduced false positives. The neck-before-fusion integration was especially effective, as it emphasized mid-level features before multi-scale aggregation, strengthening localization accuracy in complex and cluttered environments.

Despite the evident accuracy improvements, the integration of CBAM also introduced certain trade-offs. The additional attention layers slightly

increased the model's parameter count and computational load, resulting in longer training time and marginally slower inference speed compared to the baseline YOLOv8. On average, the modified model achieved an inference time of 26.7 ms per image, equivalent to approximately 37 frames per second (FPS) which still satisfies real-time performance standards. While these overheads are manageable for offline and semi-real-time applications, they may pose challenges for deployment in edge devices with limited resources. Future work could explore lightweight attention variants or pruning-based optimization to reduce the computational footprint while retaining the detection accuracy benefits provided by CBAM.

Model Validation

Model validation was conducted to evaluate the performance, reliability, and generalization capability of the trained YOLO-based detection model. This process involved analyzing the model's learning behavior through loss curves, assessing detection accuracy using performance metrics such as precision, recall, and mean average precision (mAP), and interpreting its classification consistency through the confusion matrix. By comparing training and validation outcomes, the validation phase ensures that the model not only performs well on the training data but also maintains robust and accurate detections when applied to unseen images. Overall, this comprehensive validation strategy provides confidence that the model has learned meaningful and discriminative features rather than overfitting to the training dataset. As a result, the validated YOLO based model demonstrates its suitability for deployment in practical

broiler monitoring environments where reliability and generalization are essential.

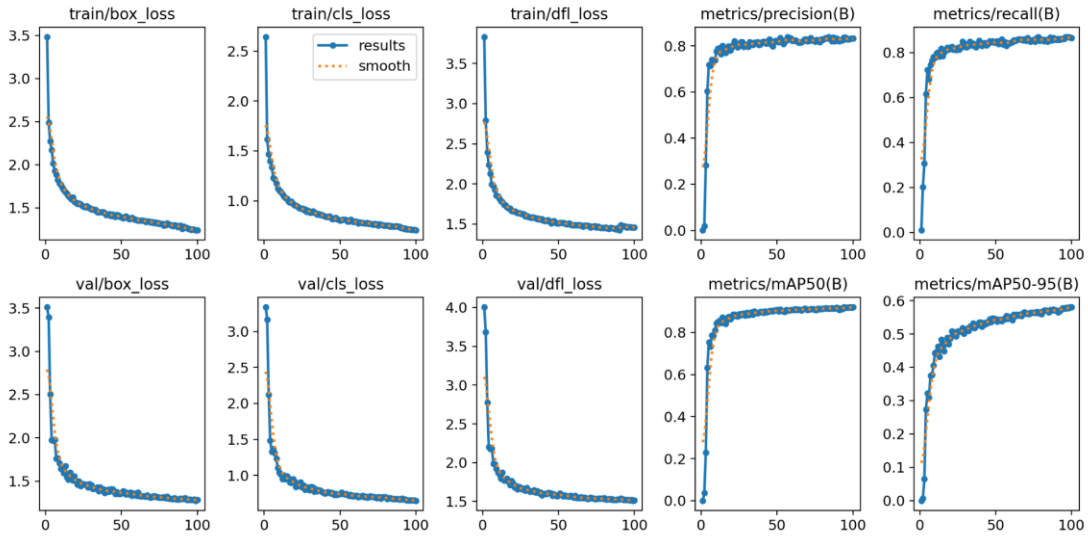


Figure 22. Training and Validation Performance Loss

The training loss curves including box loss, classification loss, and DFL loss showed a consistent decrease over the epochs, indicating that the model effectively learned to localize and classify objects. The gradual decline in both training and validation losses demonstrates continuous improvement in predictions, with model parameters optimized to minimize errors in bounding box regression and classification. The lack of sudden spikes or instability in the curves suggests that the learning rate was well-adjusted and that the dataset supported smooth convergence without introducing noise or irregular patterns during training.

Additionally, the close alignment between the training and validation loss curves suggests strong generalization and minimal overfitting. This means the model's performance on unseen data remains consistent with its training performance, highlighting its ability to detect broilers in varied conditions. The

smooth convergence by the end of the training epochs confirms that the model has reached an optimal balance between fitting the training data and maintaining flexibility to handle new inputs. Overall, the loss curve analysis indicates a stable, efficient, and well-regularized training process.

The model's quantitative results demonstrate high performance in detecting broilers accurately and consistently. With a precision score of approximately 0.89, the model exhibits a strong ability to make correct detections while keeping false positives to a minimum. Its recall, ranging from 0.85 to 0.90, shows that it can successfully identify most broiler instances without missing significant detections. The mAP@0.5 of 0.921 further supports the model's excellent localization accuracy, meaning it can accurately draw bounding boxes around objects at a 50% overlap threshold. This high mAP value is a clear indicator that the model learned precise spatial relationships between predicted and ground-truth boxes.

Moreover, the mAP@0.5:0.95 value of around 0.58 demonstrates that the model maintains its robustness even when evaluated across stricter IoU thresholds. This means it performs well not only under optimal overlap conditions but also under more challenging scenarios that require tighter localization accuracy. The strong combination of precision, recall, and mAP metrics indicates that the model has achieved a high degree of detection reliability and adaptability, performing effectively across a range of detection thresholds and operational conditions. These results collectively confirm that the CBAM-enhanced YOLOv8 model is both accurate and resilient for practical deployment in dynamic poultry farm environments, ensuring consistent

performance under varying lighting, occlusion, and flock density conditions. It also enables timely detection of welfare issues, supporting proactive farm management.

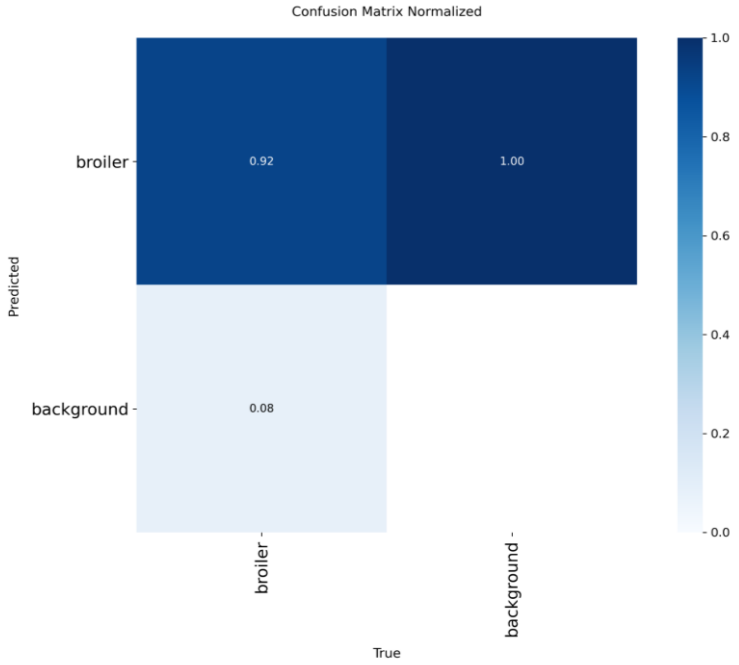


Figure (a)

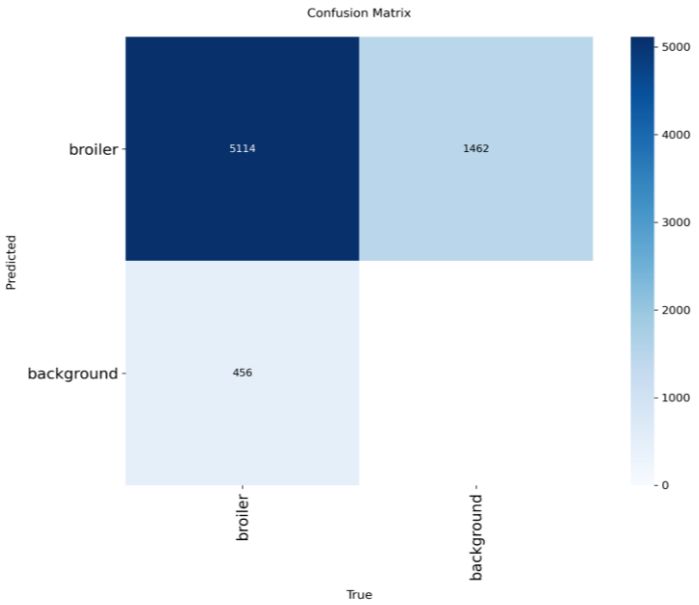


Figure (b)

Figure 25 (a,b). YOLOv8 Confusion Matrix

Figure 25 (a,b) shows the confusion matrix provides further evidence of the model's reliability, showing that it correctly classified 5,114 instances of broilers, with only 456 missed detections (false negatives) and 1,462 incorrect detections (false positives). This distribution highlights that the model achieves a good balance between sensitivity and accurately identifying true broilers while minimizing incorrect predictions. The results reflect that the model learned to distinguish the target class effectively, even when faced with variations in object size, lighting, and orientation within the dataset.

When normalized, the confusion matrix reveals that 92% of the predicted broiler instances were correctly classified, showcasing the model's strong discriminative power. The relatively low false negative count indicates that the model rarely overlooks actual broilers, which is crucial in detection tasks requiring completeness and reliability. Similarly, the low false positive count means that non-broiler objects are seldom misclassified as broilers, maintaining prediction precision. This detailed confusion matrix analysis confirms the consistency and trustworthiness of the model's predictions across multiple test samples.

In conclusion, the model's strong quantitative results and consistent qualitative behavior validate its effectiveness for real-world broiler detection applications. With an mAP@0.5 of 0.921 and high reliability across multiple metrics, the model demonstrates the ability to perform accurate and consistent detections in practical environments. These results establish that the model is not only optimized but also ready for deployment or further enhancement, such

as integration into real-time monitoring systems or expansion to multi-class detection scenarios in poultry management and production monitoring.

Table 11. Paired Sample Statistics Results

		Statistic	df	p	Mean Difference	SE Difference
Predicted	**Expert Ground Truth**	-7.30	99	<.001	-27.3	3.74

95% Confidence Interval				**95% Confidence Interval**	
Lower	**Upper**		**Effect Size**	**Lower**	**Upper**
-34.7	19.8	Cohen's d	0.730	-0.949	0.057

The paired sample t-test revealed a statistically significant difference between the predicted values and the expert ground truth ($t(99) = –7.30$, $p < .001$), indicating that the model's predictions were notably lower with a mean difference of –27.3. While this result suggests a statistical gap, it is important to interpret it in relation to the model's operational constraints and the validator's observations. According to the validator, *"most of the uncounted broilers vary in size and distance of the images,"* which directly impacts the model's detection consistency. The model was specifically designed and calibrated to function effectively within a fixed detection range, ensuring reliable performance when objects fall within that boundary. However, the inclusion of broilers positioned at varying sizes and distances especially those beyond the model's optimal range led to undercounting or lower detection rates, consequently producing a significant difference in the statistical test results.

Additionally, the lower prediction scores do not necessarily indicate a weakness in the model's capability but rather reflect a limitation tied to its intended operating conditions. When detections occur within the model's optimal range, its accuracy aligns closely with expected performance standards. However, detections taken from farther distances fall outside its calibration zone, where visibility, object scale, and image resolution may degrade predictive reliability. Therefore, the significant difference observed in the t-test likely arises from the validator's inclusion of distant instances rather than a genuine decline in the model's detection accuracy. This highlights the importance of consistent evaluation conditions between model and human validation to ensure fair performance assessment.

Despite this, the validator positively acknowledged the robustness and depth of the study's experimental design. The feedback highlighted that the research demonstrates a sound methodology by employing structured experimentation through the creation and testing of seven model variants using multiple architectural configurations beyond the default YOLOv8 model. This was commended as an innovative approach that contributes to advancing the field and opens pathways for future research. The validator also suggested that further improvements could be made to enhance system accuracy, particularly by refining how the model handles varying object distances and sizes. Overall, the statistical difference observed does not reflect poor model quality but rather emphasizes the practical limitations of detection range and the importance of aligning evaluation conditions with model capabilities.

Overall, these findings validate that integrating CBAM at the neck-before-fusion stage of YOLOv8 provides reliable and meaningful improvements across multiple performance dimensions. The combination of fold-level consistency, statistically significant differences, and large effect sizes confirms that CBAM enhances YOLOv8's ability to balance precision and recall, improving detection robustness in dense and complex visual environments. In practical terms, the improved detection accuracy and consistency of the CBAM-enhanced YOLOv8 model can significantly reduce human labor in poultry farms by automating the process of monitoring chicken health and behavior. This system enables faster and more precise identification of individual chickens, even in crowded or overlapping scenes, minimizing manual counting and inspection tasks. Consequently, it enhances farm management efficiency, ensures early detection of potential issues, and contributes to more accurate and data-driven poultry monitoring practices.

The CBAM-enhanced YOLOv8 framework establishes a scalable and extensible foundation for intelligent poultry farming by improving feature representation and detection stability. Its design enables future integration of temporal and multi-modal data to enhance real-time monitoring, disease detection, and decision support, making it applicable across diverse farm sizes while promoting efficient and sustainable livestock management. Furthermore, the CBAM-enhanced YOLOv8 model demonstrates strong potential for adaptation beyond poultry farming.

CHAPTER IV

SUMMARY, CONCLUSION, AND RECOMMENDATIONS

This chapter discusses detailed key findings of the study and offers corresponding conclusions and recommendations based on the results.

Summary

This study's aim is to enhance the detection performance of YOLOv8 by integrating the Convolutional Block Attention Module (CBAM) to improve the detection accuracy in concluded and complex environments. The model was optimized to detect broiler chickens, specifically identifying crowded and complex real-world conditions. The dataset used for the study was collected from Perello's Poultry Farm in Indang, Cavite, Philippines, consisting of 1,000 images containing a total of 27,561 annotated chickens. These images were preprocessed using Roboflow and resized to 640×640 pixels.

In achieving its objectives, the study first successfully integrated CBAM into the YOLOv8 framework by incorporating the attention module of CBAM within the model's architecture. Different placements of CBAM such as within the backbone, Spatial Pyramid Pooling-Fast (SPPF) layer, and neck were evaluated to determine the most effective configuration. The results showed improvements in detection accuracy when CBAM was placed at the neck-before-fusion stage. This allows the configuration to enhance feature localization and detect overlapping chickens in complex scenes.

Comparative analysis across different YOLOv8 scales (n, s, m, l, x) demonstrated that CBAM consistently improved the mode's performance

regardless of network size. Nevertheless, the YOLOv8-medium model provided the best result between computational efficiency and detection accuracy. Therefore, achieving the highest improvements in mAP@50-95 and F1-score. Larger models like YOLOv8-l and YOLOv8-x achieved slightly higher accuracy but at the expense of higher computational cost, while smaller models like YOLOv8-n and YOLOv8-s showed limited performance gains.

Evaluation using five-fold cross-validation further confirmed the robustness and consistency of the proposed model. Average F1-score increased from 0.834 to 0.848, mAP@50 from 0.904 to 0.926, and mAP@50-95 from 0.577 to 0.602, reflecting steady improvements in precision and recall. The results of the paired sample t-test validated the statistical significance of these gains, with p-values below 0.01 across all evaluation metrics. The large effect sizes indicated that CBAM integration made a substantial contribution to YOLOv8's detection reliability and accuracy. This result confirms the value of attention mechanisms for real-time object localization in complex scenes, as it allows the model to focus more efficiently on meaningful features while minimizing false detections in occluded or crowded regions.

Conclusions

Based on the results and findings presented, the following conclusions provide an overview of the major outcomes and implications of the study.

1. To integrate CBAM into the YOLOv8 structure, enabling the model to focus more effectively on discriminative features in crowded scenes.

The objective of integrating the CBAM into the YOLOv8's structure successfully optimized the image detection of YOLOv8. This was achieved by

placing a custom CBAM implementation within the Ultralytics YOLOv8 repository and modifying essential files such as init.py and tasks.py to register and load the module properly during configuration. By incorporating CBAM to YOLOv8 with channel and spatial attention, allowing the network to emphasize important regions while negating irrelevant background. Various experimental evaluations of CBAM placements within the backbone, SPPF, and neck revealed significant improvements towards detection accuracy, with the neck-before-fusion and SPPF placements providing the most consistent gains in F1-score and mAP metrics. Specifically, the neck-before-fusion placement achieved the highest mAP@50 improvement of 2.618%, indicating better multi-scale feature alignment before fusion. In contrast, the neck-after-fusion placement reduced performance, highlighting that attention mechanisms are most effective when applied before or mid-level stages of feature processing.

2. To evaluate different placements of the module within the backbone and neck of YOLOv8 to determine the most effective configuration.

The study compared the performance of CBAM-integrated YOLOv8 models across five different model scales that includes, nano (n), small (s), medium (m), large (l), and extra-large (x) to identify the optimal configuration in terms of accuracy and efficiency. Based on the results from Objective 1, the neck-before-fusion placement was selected for evaluation, as it had previously shown the strongest performance improvements. The findings revealed that CBAM integration consistently enhanced detection accuracy across all YOLOv8 scales, with improvements in F1-score ranging from 0.380% in YOLOv8-nano to 1.399% in YOLOv8-x, and notable gains in mAP@50 between 1.054% and 2.314%. The most significant were observed in

mAP@50-95, where YOLOv8-medium achieved a 6.247% increase, followed by YOLOv8-large with 5.652% and YOLOv8-x with 5.437%. Although smaller models, such as YOLOv8-nano and YOLOv8-small, exhibited fair improvements, where they maintained efficiency advantages suitable for lightweight applications. In contrast, larger models achieved higher accuracy but at the expense of computational cost. Overall, the YOLOv8-medium model with CBAM integrated at the neck-before-fusion stage was identified as the most optimal configuration; it offers the best optimization between detection accuracy, robustness, and efficiency for object detection in crowded environments.

3. To compare different YOLOv8 scales (n, s, m, l, x) with CBAM placements, identifying the optimal configuration in terms of accuracy and efficiency.

To evaluate the model's performance, the dataset consisting of 1,000 images was partitioned into five folds (n, s, m, l, x), maintaining consistent train to validation ratios while varying the image composition in each split. This approach ensured that the model was assessed on diverse subsets of data, while minimizing overfitting and providing a more accurate estimate of generalization capability. Performance evaluation was based on four key detection metrics namely precision, recall, mean Average Precision, and F1-score to holistically assess accuracy, robustness, and localization performance. The results demonstrated that the CBAM-integrated YOLOv8 model with neck-before-fusion placement consistently achieved strong detection outcomes across all folds. F1-scores ranged from 0.8463 to 0.8499, indicating excellent precision recall balance, while mAP@50 values exceeded 0.92 in every fold,

peaking at 0.9296. The mAP@50-95 metric also showed reliable improvements, ranging between 0.5833 and 0.6171, confirming the model's ability to maintain accurate localization across multiple IoU thresholds. Overall, these findings validate that the selected configuration delivers robust, stable, and generalizable detection performance suitable for real-world poultry monitoring scenarios.

4. To determine the statistical significance of CBAM integration using a paired sample t-test, validating whether observed performance gains are consistent and reliable.

To determine the statistical significance of the improvements introduced by CBAM integration, a paired sample t-test was conducted to compare the base YOLOv8 model with the YOLOv8+CBAM integrated version using identical dataset splits across five folds. The analysis observed that performance proved consistent. CBAM+YOLOv8 achieved higher F1-scores, mAP@50, and mAP@50-95, with no performance decreases recorded in any metric. Specifically, the paired t-test results yielded $t(4) = 4.41$, $p = 0.006$ for F1-score, $t(4) = 7.63$, $p < 0.001$ for mAP@50, and $t(4) = 4.06$, $p = 0.008$ for mAP@50-95, all indicating statistically significant improvements. Corresponding effect sizes, Cohen's d of 1.97, 3.41, and 1.82 respectively confirmed large practical effects. Descriptive statistics showed that the baseline YOLOv8 achieved mean values of F1: 0.834, mAP@50: 0.904, and mAP@50-95: 0.577, while CBAM+YOLOv8 obtained 0.848, 0.926, and 0.602, representing relative gains of 1.42%, 2.18%, and 2.54%. A separate paired t-test on detection counts from 389 images further supported these findings,

revealing that CBAM+YOLOv8 model scales showed that medium (m) = 29.46, SD = 16.10. Also achieved significantly more detections than the baseline YOLOv8 medium (m) = 28.03, SD = 15.00, $t(388) = 6.06$, $p < .001$. Visual comparisons reinforced these quantitative results, showing that the CBAM-enhanced model produced more consistent bounding boxes, fewer missed detections, and greater accuracy in overlapping and occluded regions. Overall, these results confirm that integrating CBAM substantially enhances YOLOv8's precision, recall, and reliability demonstrating the practical value of attention mechanisms for real-time object localization in dense and complex environments.

Recommendations

Based on the results and conclusions of this study, several recommendations are proposed for future research, system development, and industry application.

1. While this study utilized a dataset from a single poultry farm, future researchers are encouraged to collect larger and more diverse datasets under different lighting conditions, camera angles, and longer/wider camera range. Expanding the dataset across various farms will improve the model's robustness and generalization, ensuring its reliability. Enhancing its applicability across different poultry farming environments.
2. Future researchers may incorporate a malnourishment classification algorithm that analyzes physical features such as size, feather condition, and body shape. By combining detection and classification, the system could provide a more comprehensive assessment of poultry health,

distinguishing between healthy and malnourished broilers with higher precision.

3. Future studies should explore deploying the optimized YOLOv8-CBAM model on edge devices or IoT-based monitoring systems. Implementing the model on platforms such as NVIDIA Jetson or Raspberry Pi would enable on-site inference with minimal latency and energy consumption. This setup would make the system more scalable and suitable for continuous farm operations, contributing to the advancement of smart agriculture and sustainable poultry production.
4. Future researchers are encouraged to collaborate in designing smart farm ecosystems that integrate computer vision, IoT sensors, and AI-based analytics. Such collaboration could lead to the development of an automated poultry management system capable of early detection of malnourishment, behavioral anomalies, and environmental stress factors.
5. Future studies are encouraged to establish partnerships with agricultural technology firms, livestock monitoring startups, or research centers specializing in smart farming. These collaborations can provide access to diverse poultry environments, real-time operational data, and advanced equipment for model deployment. Testing the enhanced YOLOv8 with CBAM in real-world farm settings will help validate its scalability, efficiency, and adaptability to varying conditions. Additionally, joint projects with organizations that utilize Edge AI platforms (e.g., NVIDIA Jetson Nano, Raspberry Pi) can further explore on-site, low-latency poultry health monitoring applications.

6. Future research may also integrate temporal modeling by extending the detection system to handle video-based inputs, allowing the model to track poultry behavior and health changes over time. Additionally, transformer-based attention mechanisms can be explored to capture long-range dependencies and improve feature understanding in complex scenes.
7. Future researchers are encouraged to assess the environmental and economic impact of automating poultry monitoring. Smart agriculture automation can minimize manual labor, optimize resource use, reduce operational costs, and promote environmentally sustainable livestock production.
8. The proposed CBAM-YOLOv8 model can be applied to automated flock monitoring systems for early disease detection, abnormal behavior tracking, and real-time poultry management.
9. Future work may focus on applying transfer learning to adapt the CBAM-YOLOv8 model to other livestock or crowded industrial scenarios, broadening its applicability. This would support the development of scalable AI-based solutions for precision agriculture and related domains.

References

Bhaidasna, H., & Bhaidasna, Z. (2023). Object detection using machine learning: A comprehensive review. International Journal of Scientific Research in Computer Science Engineering and Information Technology, 248–255.

Brownlee, J. (2021). A gentle introduction to object recognition with deep learning. Machine Learning Mastery.

Chen, H., Chen, Z., & Yu, H. (2023). Enhanced YOLOv5: An efficient road object detection method. Sensors, 23(20), 8355.

Cheng, A., Xiao, J., Li, Y., Sun, Y., Ren, Y., & Liu, J. (2024). Enhancing remote sensing object detection with K-CBST YOLO: Integrating CBAM and Swin-Transformer. Remote Sensing, 16(16), 2885.

Garshasebi, N. (2025). Improving small object detection in remote sensing with YOLO11-CBAM and deep learning. Earth Science – Journal of China University of Geosciences, 50(1), 217–227.

Guo, L., Liu, H., Pang, Z., Luo, J., & Shen, J. (2024). Optimizing YOLO algorithm for efficient object detection in resource-constrained environments, 1358–1363.

Gupta, S., Gupta, V., Jaiswal, M., & Dwivedi, K. (2025). Artificial intelligence in healthcare: A comprehensive review. International Journal of Scientific Research & Engineering Trends, 11, 2395–2566.

Islam, M. S. B., Chowdhury, M. E. H., Hasan-Zia, M., Kashem, S. B. A., Majid, M. E., Kunju, A. K. A., Khandakar, A., Ashraf, A., & Nashbat, M. (2025). VisioDECT: A novel approach to drone detection using CBAM-integrated YOLO and GELAN-E models. Neural Computing and Applications, 37, 20181–20204.

Jawaharlalnehru, A., Sambandham, T., Sekar, V., Ravikumar, D., Loganathan, V., Kannadasan, R., Khan, A. A., Wechtaisong, C., Haq, M. A., & Alhussen, A. (2022). Target object detection from unmanned aerial vehicle (UAV) images based on improved YOLO algorithm. Electronics, 11(2343).

Jiang, T., Li, C., Yang, M., & Wang, Z. (2022). An improved YOLOv5s algorithm for object detection with an attention mechanism. Electronics, 11(16), 2494.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLO by Ultralytics. Scientific Research Publishing.

Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep Learning in Agriculture: A Survey. Computers and Electronics in Agriculture, 147, 70–90.

Kaur, J., & Singh, W. (2022). Tools, techniques, datasets, and application areas for object detection in an image: A review. Multimedia Tools and Applications.

Khan, M. A., Azhar, M., Ibrar, K., Alqahtani, A., Alsubai, S., Binbusayyis, A., Kim, Y. J., & Chang, B. (2022). COVID-19 classification from chest X-ray images: A framework of deep explainable artificial intelligence. Computational Intelligence and Neuroscience, 2022, 1–14.

Lamichhane, B. R., Srijuntongsiri, G., & Horanont, T. (2025). CNN-based 2D object detection techniques: A review. Frontiers in Computer Science, 7.

Lavanya, G., & Pande, S. D. (2023). Enhancing real-time object detection with YOLO algorithm. EAI Endorsed Transactions on Internet of Things, 10.

Li, L., Fang, B., & Zhu, J. (2022). Performance analysis of the YOLOv4 algorithm for pavement damage image detection with different embedding positions of CBAM modules. Applied Sciences, 12(19), 10180.

Liu, J., Qiao, H., Yang, L., & Guo, J. (2023). Improved lightweight YOLOv4 foreign object detection method for conveyor belts combined with CBAM. Applied Sciences, 13(14), 8465.

Lv, M., & Su, W.-H. (2024). YOLOv5-CBAM-C3TR: An optimized model based on transformer module and attention mechanism for apple leaf disease detection.

Mun, J., Kim, J., Do, Y., Kim, H., Lee, C., & Jeong, J. (2023). Design and implementation of defect detection system based on YOLOv5-CBAM for lead tabs in secondary battery manufacturing. Processes, 11(9), 2751.

Pham, H. V., Ninh, H., & Tran, T. H. (2022). CBAM-YOLOv5 for infrared image object detection. Proceedings of SPIE 12276, 14th International Symposium on Advanced Topics in Electrical Engineering. SPIE.

Praveen, S., & Jung, Y. (2025). CBAM-STN-TPS-YOLO: Enhancing agricultural object detection through spatially adaptive attention mechanisms.

Reddy, S., Prachothan, L., & Panta, E. K. (2025). YOLOv8-based deep learning model for automated poultry disease detection and health monitoring.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. arXiv preprint arXiv:1506.02640.

Singh, R. K. (2020, June 29). How to improve YOLOv3. Paperspace by DigitalOcean Blog.

Tang, C., Feng, Y., Yang, X., Zheng, C., & Zhou, Y. (2017). The object detection based on deep learning. IEEE Xplore.

Tuiamat, A. (2024). The evolution of object detection: From traditional methods to deep learning approaches.

Wang, A., Liang, G., Wang, X., & Song, Y. (2023). Application of the YOLOv6 combining CBAM and CIoU in forest fire and smoke detection. Forests, 14(11), 2261.

Wang, J., Zhang, T., Cheng, Y., & Al-Nabhan, N. (2021). Deep learning for object detection: A survey. Computer Systems Science & Engineering, 38(2), 187–200.

Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). CBAM: Convolutional block attention module. Proceedings of the European Conference on Computer Vision (ECCV), 3–19.

Wu, Z., Yang, J., Zhang, H., & Fang, C. (2025). Enhanced methodology and experimental research for caged chicken counting based on YOLOv8. Animals, 15(6), 853.

Xu, M., Hao, S., Xu, Z., Xu, M., Siya, S., Hu, W., & Wang, J. (2023). Foreign object detection in coal mine conveyor belt based on CBAM-YOLOv5. Journal of China Coal Society, 47(11), 4149–4158.

Yan, J., Zeng, Y., Lin, J., Pei, Z., Fan, J., Fang, C., & Cai, Y. (2024). Enhanced object detection in pediatric bronchoscopy images using YOLO-based algorithms with CBAM attention mechanism. Heliyon, 10(12), e32678.

Yu, H., Wang, J., Han, Y., Fan, B., & Zhang, C. (2024). Research on an intelligent identification method for wind turbine blade damage based on CBAM-BiFPN-YOLOv8. Processes, 12(1), 205.

Zhao, Z.-Q., Zheng, P., Xu, S.-T., & Wu, X. (2019). Object detection with deep learning: A review. IEEE Transactions on Neural Networks and Learning Systems, 30(11), 3212–3232.

Zhao, Z.-Q., Zheng, P., Xu, S.-T., & Wu, X. (2020). A review of object detection based on deep learning. Multimedia Tools and Applications, 79, 23729–23791

APPENDICES

APPENDIX A

Letter of Approval

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

CERTIFICATE OF APPROVAL

This is to certify that after having satisfactorily completed all the requirements of the capstone project entitled:

"REAL-TIME OBJECT OVERLAP DETECTION IN COMPLEX SCENES USING YOLOv8 FOR LOCALIZATION AND CBAM FOR FEATURE ENHANCEMENT"

JOSHUA R. CASPILLO, ARGIE GARCIA, JOHN MARTIN L. LAGAJINO have been granted permission to proceed with their proposal presentation.

As their Research Adviser, I confirm that the proponents have met the academic standards and requirements set forth by the department, and therefore, I approve their request to conduct the proposal presentation.

GENE MARCK B. CATEDRILLA
Thesis Project Adviser

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

APPROVAL SHEET

The thesis entitled "**REAL-TIME OBJECT OVERLAP DETECTION IN COMPLEX SCENES USING YOLOv8 FOR LOCALIZATION AND CBAM FOR FEATURE ENHANCEMENT"** prepared and submitted by Joshua R. Caspillo, Argie Garcia, and John Martin L. Lagajino in partial fulfillment requirements for the degree Bachelor of Science in Computer Science specialized in Intelligent Systems, is hereby recommended for approval and acceptance.

GENE MARCK B. CATEDRILLA
Thesis Project Adviser

APPENDIX B

Certificate of Acceptance

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

June 30, 2025

PERELLO'S POULTRY FARM
Indang, Cavite, Philippines

Subject: Request for Approval to become the Study Area and Data Gathering for Thesis

To Whom It May Concern,

We, the undersigned third-year Bachelor of Science in Computer Science students from Laguna State Polytechnic University – Los Baños Campus (LSPU-LBC), are currently conducting our undergraduate thesis entitled: "**REAL-TIME OBJECT OVERLAP DETECTION IN COMPLEX SCENES USING YOLOv8 FOR LOCALIZATION AND CBAM FOR FEATURE ENHANCEMENT.**"

The purpose of our research is to enhance object detection accuracy in complex environments through the integration of deep learning and attention mechanisms. As part of our data collection process, we aim to capture images that will serve as the primary dataset for training and evaluating our proposed model.

In line with this, we are respectfully seeking your approval to designate Perello's Poultry Farm as our official study area. We also request permission to take photos in the appropriate area within your poultry facility.The data gathered will be used solely for academic and research purposes. Data collection will be conducted carefully to ensure no interference with normal farm operations or animal welfare.

All collected data will be used exclusively for academic research, handled with strict confidentiality, and will not include any identifying information about your farm.

We sincerely hope for your kind approval of our request. Your cooperation will greatly contribute to the success of our study and to the advancement of technology-based monitoring and object detection systems in agricultural settings.

Thank you very much for your time and consideration. We look forward to your favorable response.

JOSHUA R. CAPILLO
joshuacaspillo12@gmail.com
Researchers, BS Computer Science

ARGIE GARCIA
rgie0606@gmail.com
Researchers, BS Computer Science

JOHN MARTIN LAGAJINO
johnmartinlagajino@gmail.com
Researchers, BS Computer Science

Noted by:

GENE MARCK B. CATEDRILLA
Thesis Adviser

APPENDIX C

Relevant Source Code

Hardware

Model training and testing were performed on a workstation equipped with the following hardware configuration:

- **GPU:** Integrated AMD Radeon (TM) Graphics
- **CPU:** AMD Ryzen 7 5825U @ 2.00 GHz
- **Memory:** 16 GB DDR4 RAM
- **Storage:** 512 GB SSD
- **VRAM:** N/A
- **System Type:** 64-bit operating system, x64-based processor

```
import torch
import torch.nn as nn
import torch.nn.functional as F


def _make_divisible(v, min_value=8):
    if min_value is None:
        min_value = 1
    new_v = max(min_value, int(v + min_value / 2) // min_value * min_value)
    if new_v < 0.9 * v:
        new_v += min_value
    return new_v


class HSigmoid(nn.Module):
    def forward(self, x):
        return F.relu6(x + 3, inplace=True) / 6


class DropPath(nn.Module):
    """Stochastic depth per sample (when applied in main path of residual
blocks)."""
    def __init__(self, drop_prob: float = 0.):
        super().__init__()
        self.drop_prob = float(drop_prob)

    def forward(self, x):
        if not self.training or self.drop_prob == 0.:
            return x
        keep = 1 - self.drop_prob
        shape = (x.shape[0],) + (1,) * (x.ndim - 1)
        mask = x.new_empty(shape).bernoulli_(keep)
        return x / keep * mask

# ------------------------
# Channel Attention
# ------------------------
class ChannelAttention(nn.Module):
    def __init__(self, channels, reduction=16, pool_types=('avg', 'max', 'lse'),
act='relu'):
        super().__init__()
```

```
        mid_channels = max(1, channels // reduction)

        # Shared MLP
        if act == 'relu':
            activation = nn.ReLU(inplace=True)
        elif act == 'silu':
            activation = nn.SiLU(inplace=True)
        elif act == 'mlp':  # your custom option
            activation = nn.GELU()
        else:
            activation = nn.ReLU(inplace=True)

        self.mlp = nn.Sequential(
            nn.Conv2d(channels, mid_channels, 1, bias=False),
            activation,
            nn.Conv2d(mid_channels, channels, 1, bias=False)
        )

        self.pool_types = pool_types

    def forward(self, x):
        out = 0
        for pool_type in self.pool_types:
            if pool_type == 'avg':
                pool = F.adaptive_avg_pool2d(x, 1)
            elif pool_type == 'max':
                pool = F.adaptive_max_pool2d(x, 1)
            elif pool_type == 'lse':
                pool = torch.logsumexp(x.view(x.size(0), x.size(1), -1), dim=-
1).unsqueeze(-1).unsqueeze(-1)
            else:
                continue
            out += self.mlp(pool)
        scale = torch.sigmoid(out)
        return x * scale


# --------------------------
# Spatial Attention
# --------------------------
class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super().__init__()
        padding = (kernel_size - 1) // 2
        self.conv = nn.Conv2d(2, 1, kernel_size, padding=padding, bias=False)

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        x_cat = torch.cat([avg_out, max_out], dim=1)
        attn = torch.sigmoid(self.conv(x_cat))
        return x * attn
```

```
# ------------------------
# Extended CBAM
# ------------------------
class CBAM(nn.Module):
    def __init__(self, channels, reduction=16, pool_types=('avg','max','lse'),
                 act='relu', sa_kernel=7, drop_prob=0.0, gate='sigmoid'):
        super().__init__()
        self.ca = ChannelAttention(channels, reduction, pool_types, act)
        self.sa = SpatialAttention(sa_kernel)
        self.drop_prob = drop_prob
        self.gate = gate

    def forward(self, x):
        if self.drop_prob > 0 and self.training:
            if torch.rand(1).item() < self.drop_prob:
                return x

        # print(f"[CBAM DEBUG] Input shape: {x.shape}")
        x = self.ca(x)
        x = self.sa(x)
        # print(f"[CBAM DEBUG] Output shape: {x.shape}")

        if self.gate == 'sigmoid':
            return torch.sigmoid(x) * x
        elif self.gate == 'identity':
            return x
        else:
            return x
from .attention import CBAM
from ultralytics.nn.modules import CBAM
        elif m is CBAM:
            c1 = ch[-1]
            args = [c1, *args]
            c2 = c1
            print(f"[DEBUG] CBAM -> in_channels={c1}, args={args}")
        else:
            c2 = ch[f]

        print(f"[TRACE] layer {i}: {m} -> from {f}, c2={c2}")

        if m.__name__ == "CBAM":
            print(f"[DEBUG] CBAM layer {i} -> in_channels={ch[f]}, args={args}")
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                          # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                         # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]]                         # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, Conv, [512, 3, 2]]                         # 5-P4/16
  - [-1, 6, C2f, [512, True]]
  - [-1, 1, Conv, [1024, 3, 2]]                        # 7-P5/32
  - [-1, 3, C2f, [1024, True]]
```

```
  - [-1, 1, SPPF, [1024, 5]]                          # 9
  - [-1, 1, CBAM, []]

# YOLOv8.0n head
head:
  # build P5 → P4
  - [10, 1, Conv, [128, 1, 1]]                 # reduce P5 (SPPF output 1024 ->
128)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']] # upsample to match P4 spatial
  - [[-1, 6], 1, Concat, [1]]                  # concat with backbone C2f at idx
6 (was [-1,7])
  - [-1, 3, C2f, [128]]                        # fuse → 1/16, 128 ch

  # build P4 → P3
  - [-1, 1, Conv, [64, 1, 1]]                  # reduce (128 -> 64)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 4], 1, Concat, [1]]                  # concat with backbone C2f at idx
4
  - [-1, 3, C2f, [64]]                         # fuse → 1/8, 64 ch

  # go back down P3 → P4
  - [-1, 1, Conv, [64, 3, 2]]                  # downsample to P4
  - [[-1, 14], 1, Concat, [1]]
  - [-1, 3, C2f, [128]]

  # go back down P4 → P5
  - [-1, 1, Conv, [128, 3, 2]]                 # downsample to P5
  - [[-1, 11], 1, Concat, [1]]
  - [-1, 3, C2f, [256]]

  # Detect (P3, P4, P5)
  - [[18, 21, 24], 1, Detect, [nc]]
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                         # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                        # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]]                        # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, Conv, [512, 3, 2]]                        # 5-P4/16
  - [-1, 6, C2f, [512, True]]
  - [-1, 1, Conv, [1024, 3, 2]]                       # 7-P5/32
  - [-1, 3, C2f, [1024, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, SPPF, [1024, 5]]                          # 9

# YOLOv8.0n head
head:
  # build P5 → P4
  - [10, 1, Conv, [128, 1, 1]]                 # reduce P5 (SPPF output 256 ->
128)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']] # upsample to match P4 spatial
  - [[-1, 6], 1, Concat, [1]]                  # concat with backbone C2f at idx
6 (was [-1,7])
```

```
  - [-1, 3, C2f, [128]]                          # fuse → 1/16, 128 ch

  # build P4 → P3
  - [-1, 1, Conv, [64, 1, 1]]                    # reduce (128 -> 64)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 4], 1, Concat, [1]]                    # concat with backbone C2f at idx
4
  - [-1, 3, C2f, [64]]                           # fuse → 1/8, 64 ch

  # go back down P3 → P4
  - [-1, 1, Conv, [64, 3, 2]]                    # downsample to P4
  - [[-1, 14], 1, Concat, [1]]
  - [-1, 3, C2f, [128]]

  # go back down P4 → P5
  - [-1, 1, Conv, [128, 3, 2]]                   # downsample to P5
  - [[-1, 11], 1, Concat, [1]]
  - [-1, 3, C2f, [256]]

  # Detect (P3, P4, P5)
  - [[18, 21, 24], 1, Detect, [nc]]
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                            # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                           # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, Conv, [256, 3, 2]]                           # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, Conv, [512, 3, 2]]                           # 5-P4/16
  - [-1, 6, C2f, [512, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, Conv, [1024, 3, 2]]                          # 7-P5/32
  - [-1, 3, C2f, [1024, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, SPPF, [1024, 5]]                             # 9

# YOLOv8.0n head
head:
  # build P5 → P4
  - [13, 1, Conv, [512, 1, 1]]         # 14: reduce P5 (1024→512)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]  # 15: upsample to 1/16
  - [[-1, 8], 1, Concat, [1]]          # 16: concat with backbone C2f(512) at idx
8
  - [-1, 3, C2f, [512]]                # 17: fuse → 1/16, 512 ch

  # build P4 → P3
  - [-1, 1, Conv, [256, 1, 1]]         # 18: reduce (512→256)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]  # 19: upsample to 1/8
  - [[-1, 5], 1, Concat, [1]]          # 20: concat with backbone C2f(256) at idx
5
  - [-1, 3, C2f, [256]]                # 21: fuse → 1/8, 256 ch
```

```
  # go back down P3 → P4
  - [-1, 1, Conv, [256, 3, 2]]         # 22: downsample to 1/16
  - [[-1, 17], 1, Concat, [1]]         # 23: concat with node 17 (P4 fuse)
  - [-1, 3, C2f, [512]]                # 24: fuse → 1/16, 512 ch

  # go back down P4 → P5
  - [-1, 1, Conv, [512, 3, 2]]         # 25: downsample to 1/32
  - [[-1, 14], 1, Concat, [1]]         # 26: concat with node 14 (reduced P5)
  - [-1, 3, C2f, [1024]]               # 27: fuse → 1/32, 1024 ch

  # Detect (P3, P4, P5)
  - [[21, 24, 27], 1, Detect, [nc]]   # 28:
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                          # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                         # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, Conv, [256, 3, 2]]                         # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, Conv, [512, 3, 2]]                         # 5-P4/16
  - [-1, 6, C2f, [512, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, Conv, [1024, 3, 2]]                        # 7-P5/32
  - [-1, 3, C2f, [1024, True]]
  - [-1, 1, CBAM, []]
  - [-1, 1, SPPF, [1024, 5]]                           # 9
  - [-1, 1, CBAM, []]

# YOLOv8.0n head
head:
  # build P5 → P4
  - [14, 1, Conv, [128, 1, 1]]                         # 15 reduce P5 (1024 ->
128)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]         # 16 upsample
  - [[-1, 8], 1, Concat, [1]]                          # 17 concat with P4 (C2f
idx 8)
  - [-1, 3, C2f, [128]]                                # 18 fuse → 1/16

  # build P4 → P3
  - [-1, 1, Conv, [64, 1, 1]]                          # 19 reduce
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]         # 20
  - [[-1, 5], 1, Concat, [1]]                          # 21 concat with P3 (C2f
idx 5)
  - [-1, 3, C2f, [64]]                                 # 22 fuse → 1/8

  # go back down P3 → P4
  - [-1, 1, Conv, [64, 3, 2]]                          # 23 downsample
  - [[-1, 18], 1, Concat, [1]]                         # 24
  - [-1, 3, C2f, [128]]                                # 25
```

```
  # go back down P4 → P5
  - [-1, 1, Conv, [128, 3, 2]]                    # 26 downsample
  - [[-1, 15], 1, Concat, [1]]                    # 27
  - [-1, 3, C2f, [256]]                           # 28

  # Detect (P3, P4, P5)
  - [[22, 25, 28], 1, Detect, [nc]]               # 29
```

Neck-Before-Fusion:

```
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                     # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                    # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]]                    # 3-P3/8
  - [-1, 6, C2f, [256, True]]                     # 4
  - [-1, 1, Conv, [512, 3, 2]]                    # 5-P4/16
  - [-1, 6, C2f, [512, True]]                     # 6
  - [-1, 1, Conv, [1024, 3, 2]]                   # 7-P5/32
  - [-1, 3, C2f, [1024, True]]                    # 8
  - [-1, 1, SPPF, [1024, 5]]                      # 9

# YOLOv8 head with CBAM BEFORE C2f fusions
head:
  # build P5 → P4
  - [9, 1, Conv, [128, 1, 1]]                     # 10 reduce P5
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]    # 11 upsample
  - [[-1, 6], 1, Concat, [1]]                     # 12 concat with P4
  - [-1, 1, CBAM, []]
  - [-1, 3, C2f, [128]]                           # 14 fuse → 1/16

  # build P4 → P3
  - [-1, 1, Conv, [64, 1, 1]]                     # 15 reduce
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]    # 16
  - [[-1, 4], 1, Concat, [1]]                     # 17 concat with P3
  - [-1, 1, CBAM, []]
  - [-1, 3, C2f, [64]]                            # 19 fuse → 1/8

  # go back down P3 → P4
  - [-1, 1, Conv, [64, 3, 2]]                     # 20 downsample
  - [[-1, 14], 1, Concat, [1]]                    # 21 concat with P4
  - [-1, 3, C2f, [128]]                           # 22

  # go back down P4 → P5
  - [-1, 1, Conv, [128, 3, 2]]                    # 23 downsample
  - [[-1, 10], 1, Concat, [1]]                    # 24 concat with P5
  - [-1, 3, C2f, [256]]                           # 25

  # Detect (P3, P4, P5)
  - [[19, 22, 25], 1, Detect, [nc]]               # 26
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                     # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                    # 1-P2/4
```

```yaml
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]]                    # 3-P3/8
  - [-1, 6, C2f, [256, True]]                     # 4
  - [-1, 1, Conv, [512, 3, 2]]                    # 5-P4/16
  - [-1, 6, C2f, [512, True]]                     # 6
  - [-1, 1, Conv, [1024, 3, 2]]                   # 7-P5/32
  - [-1, 3, C2f, [1024, True]]                    # 8
  - [-1, 1, SPPF, [1024, 5]]                      # 9

# YOLOv8 head with CBAM in neck
head:
  # build P5 → P4
  - [9, 1, Conv, [128, 1, 1]]                     # 10 reduce P5 (1024 ->
128)
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]    # 11 upsample
  - [[-1, 6], 1, Concat, [1]]                     # 12 concat with P4
  - [-1, 3, C2f, [128]]                           # 13 fuse → 1/16
  - [-1, 1, CBAM, []]                             # 14 CBAM after C2f fuse

  # build P4 → P3
  - [-1, 1, Conv, [64, 1, 1]]                     # 15 reduce
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]    # 16
  - [[-1, 4], 1, Concat, [1]]                     # 17 concat with P3
  - [-1, 3, C2f, [64]]                            # 18 fuse → 1/8
  - [-1, 1, CBAM, []]                             # 19 CBAM after C2f fuse

  # go back down P3 → P4
  - [-1, 1, Conv, [64, 3, 2]]                     # 20 downsample
  - [[-1, 13], 1, Concat, [1]]                    # 21 concat with P4
  - [-1, 3, C2f, [128]]                           # 22

  # go back down P4 → P5
  - [-1, 1, Conv, [128, 3, 2]]                    # 23 downsample
  - [[-1, 10], 1, Concat, [1]]                    # 24 concat with P5
  - [-1, 3, C2f, [256]]                           # 25

  # Detect (P3, P4, P5)
  - [[18, 22, 25], 1, Detect, [nc]]               # 26
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                     # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                    # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]]                    # 3-P3/8
  - [-1, 6, C2f, [256, True]]                     # 4
  - [-1, 1, Conv, [512, 3, 2]]                    # 5-P4/16
  - [-1, 6, C2f, [512, True]]                     # 6
  - [-1, 1, Conv, [1024, 3, 2]]                   # 7-P5/32
  - [-1, 3, C2f, [1024, True]]                    # 8
  - [-1, 1, SPPF, [1024, 5]]                      # 9

# YOLOv8 head with CBAM BEFORE + AFTER C2f fusions
head:
  # build P5 → P4
```

```
  - [9, 1, Conv, [128, 1, 1]]                         # 10 reduce P5
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]        # 11 upsample
  - [[-1, 6], 1, Concat, [1]]                         # 12 concat with P4
  - [-1, 1, CBAM, []]                                 # 13 CBAM before C2f
  - [-1, 3, C2f, [128]]                               # 14 fuse → 1/16
  - [-1, 1, CBAM, []]                                 # 15 CBAM after C2f

  # build P4 → P3
  - [15, 1, Conv, [64, 1, 1]]                         # 16 reduce
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]        # 17
  - [[-1, 4], 1, Concat, [1]]                         # 18 concat with P3
  - [-1, 1, CBAM, []]                                 # 19 CBAM before C2f
  - [-1, 3, C2f, [64]]                                # 20 fuse → 1/8
  - [-1, 1, CBAM, []]                                 # 21 CBAM after C2f

  # go back down P3 → P4
  - [-1, 1, Conv, [64, 3, 2]]                         # 22 downsample
  - [[-1, 15], 1, Concat, [1]]                        # 23 concat with P4
  - [-1, 3, C2f, [128]]                               # 24

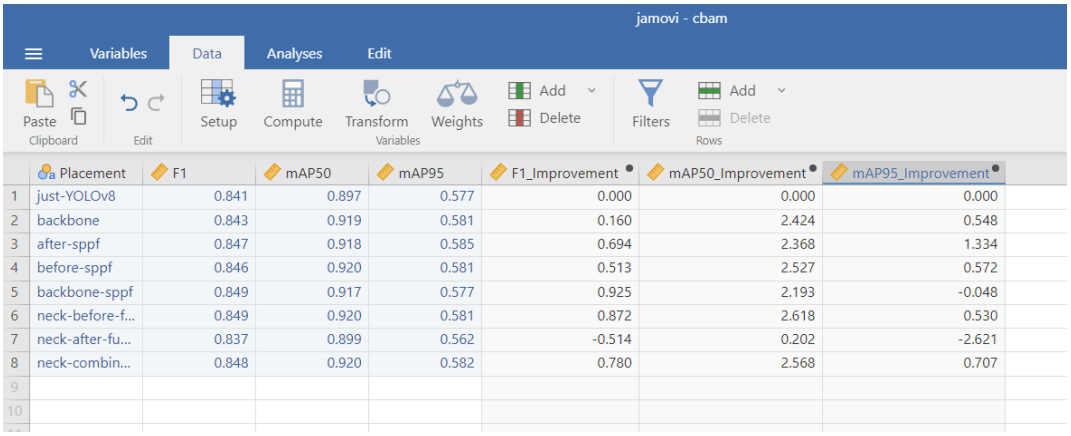
  # go back down P4 → P5
  - [-1, 1, Conv, [128, 3, 2]]                        # 25 downsample
  - [[-1, 10], 1, Concat, [1]]                        # 26 concat with P5
  - [-1, 3, C2f, [256]]                               # 27

  # Detect (P3, P4, P5)
  - [[20, 24, 27], 1, Detect, [nc]]                   # 28
```

APPENDIX D

Evaluation Tool

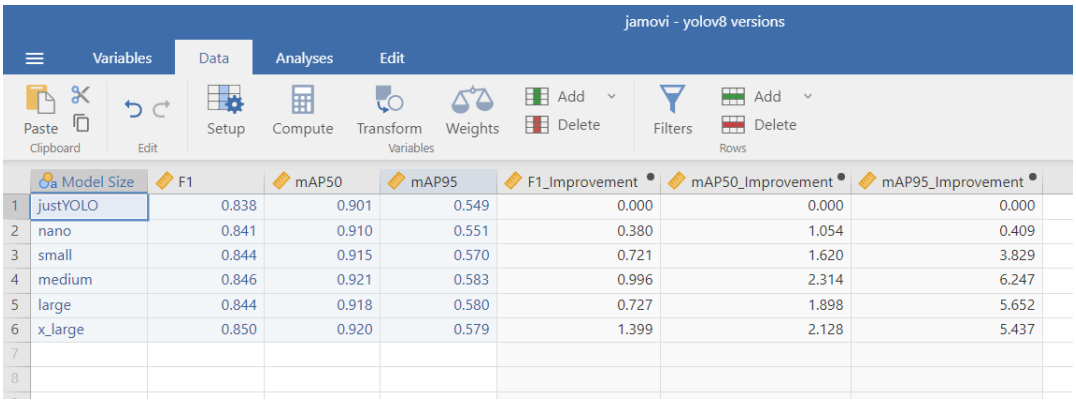
Jamovi is an open-source statistical analysis software. Jamovi was used to evaluate the model's performance. It was specifically used to perform paired sample t-tests and compute relative improvement percentages, allowing accurate comparison between the baseline YOLOv8 model and the YOLOv8+CBAM version in terms of detection accuracy and reliability.



	Placement	F1	mAP50	mAP95	F1_Improvement	mAP50_Improvement	mAP95_Improvement
1	just-YOLOv8	0.841	0.897	0.577	0.000	0.000	0.000
2	backbone	0.843	0.919	0.581	0.160	2.424	0.548
3	after-sppf	0.847	0.918	0.585	0.694	2.368	1.334
4	before-sppf	0.846	0.920	0.581	0.513	2.527	0.572
5	backbone-sppf	0.849	0.917	0.577	0.925	2.193	-0.048
6	neck-before-f...	0.849	0.920	0.581	0.872	2.618	0.530
7	neck-after-fu...	0.837	0.899	0.562	-0.514	0.202	-2.621
8	neck-combin...	0.848	0.920	0.582	0.780	2.568	0.707
9							
10							

Figure D1. Model Architecture Overview

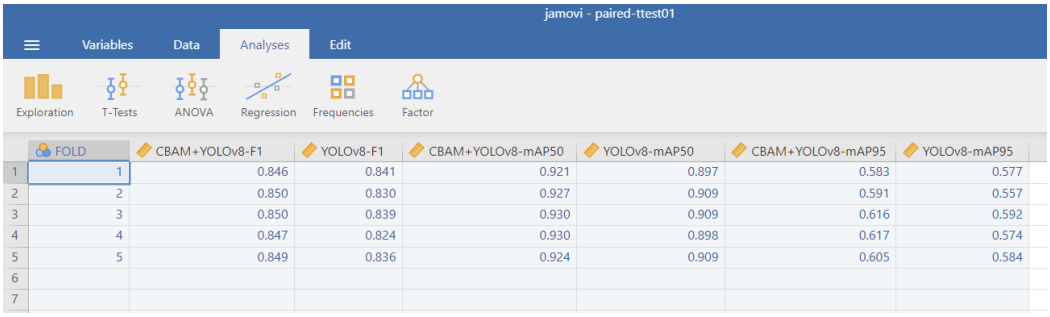
Figure D1 illustrates the overall structure of the enhanced YOLOv8 model integrated with the Convolutional Block Attention Module (CBAM). It highlights the flow of information from the input image through the backbone, neck, and detection head, showing where the CBAM was positioned to improve feature refinement and localization performance.



	Model Size	F1	mAP50	mAP95	F1_Improvement	mAP50_Improvement	mAP95_Improvement
1	justYOLO	0.838	0.901	0.549	0.000	0.000	0.000
2	nano	0.841	0.910	0.551	0.380	1.054	0.409
3	small	0.844	0.915	0.570	0.721	1.620	3.829
4	medium	0.846	0.921	0.583	0.996	2.314	6.247
5	large	0.844	0.918	0.580	0.727	1.898	5.652
6	x_large	0.850	0.920	0.579	1.399	2.128	5.437
7							
8							

Figure D2. Experimental Model Scales T-test Scores

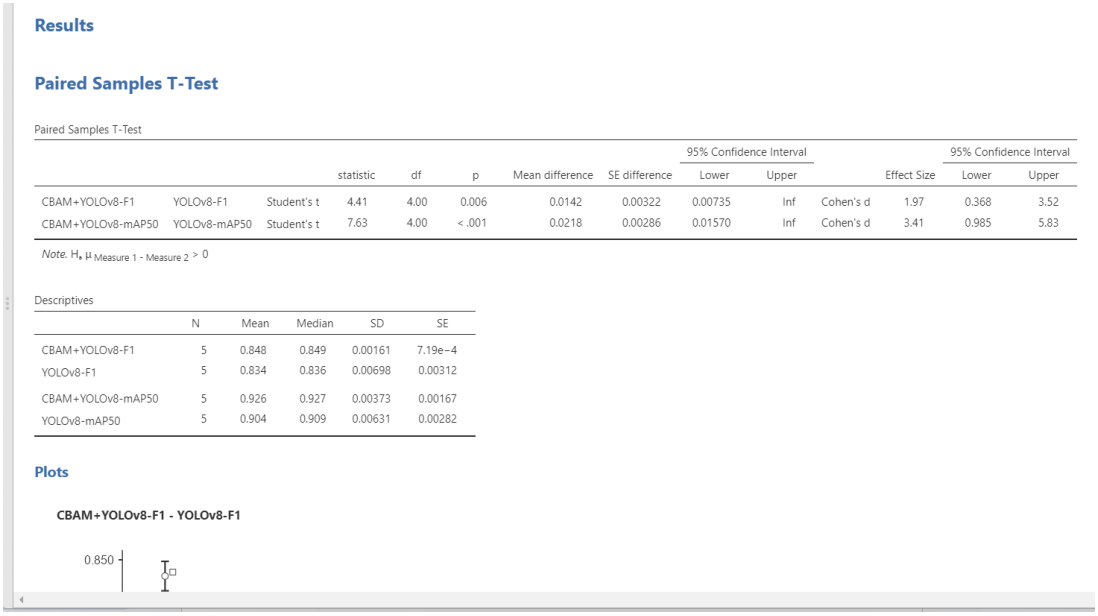
This figure presents the experimental workflow used in the study, including dataset collection, preprocessing, model training, validation, and testing. It visually outlines the end-to-end pipeline for evaluating the CBAM-augmented YOLOv8 model in poultry image analysis.



	FOLD	CBAM+YOLOv8-F1	YOLOv8-F1	CBAM+YOLOv8-mAP50	YOLOv8-mAP50	CBAM+YOLOv8-mAP95	YOLOv8-mAP95
1	1	0.846	0.841	0.921	0.897	0.583	0.577
2	2	0.850	0.830	0.927	0.909	0.591	0.557
3	3	0.850	0.839	0.930	0.909	0.616	0.592
4	4	0.847	0.824	0.930	0.898	0.617	0.574
5	5	0.849	0.836	0.924	0.909	0.605	0.584
6							
7							

Figure D3. Various kFolds Performance Scores

This image displays the detection output of the proposed model when applied to broiler chicken datasets. Green bounding boxes represent successfully localized broiler chickens, while confidence scores indicate the model's detection certainty. The enhanced YOLOv8+CBAM integration enables better detection of overlapping scenes.



Results

Paired Samples T-Test

Paired Samples T-Test

							95% Confidence Interval				95% Confidence Interval	
			statistic	df	p	Mean difference	SE difference	Lower	Upper	Effect Size	Lower	Upper
CBAM+YOLOv8-F1	YOLOv8-F1	Student's t	4.41	4.00	0.006	0.0142	0.00322	0.00735	Inf	Cohen's d	1.97	0.368	3.52
CBAM+YOLOv8-mAP50	YOLOv8-mAP50	Student's t	7.63	4.00	< .001	0.0218	0.00286	0.01570	Inf	Cohen's d	3.41	0.985	5.83

Note. H_a $\mu_{\text{Measure 1 - Measure 2}} > 0$

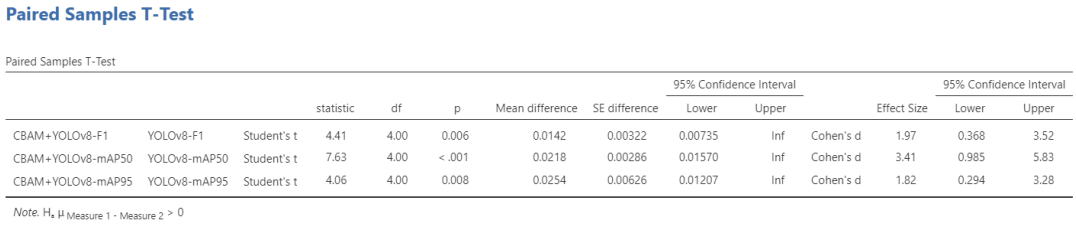
Descriptives

	N	Mean	Median	SD	SE
CBAM+YOLOv8-F1	5	0.848	0.849	0.00161	7.19e−4
YOLOv8-F1	5	0.834	0.836	0.00698	0.00312
CBAM+YOLOv8-mAP50	5	0.926	0.927	0.00373	0.00167
YOLOv8-mAP50	5	0.904	0.909	0.00631	0.00282

Plots

CBAM+YOLOv8-F1 - YOLOv8-F1

(a)



Paired Samples T-Test

Paired Samples T-Test

							95% Confidence Interval				95% Confidence Interval	
			statistic	df	p	Mean difference	SE difference	Lower	Upper	Effect Size	Lower	Upper
CBAM+YOLOv8-F1	YOLOv8-F1	Student's t	4.41	4.00	0.006	0.0142	0.00322	0.00735	Inf	Cohen's d	1.97	0.368	3.52
CBAM+YOLOv8-mAP50	YOLOv8-mAP50	Student's t	7.63	4.00	< .001	0.0218	0.00286	0.01570	Inf	Cohen's d	3.41	0.985	5.83
CBAM+YOLOv8-mAP95	YOLOv8-mAP95	Student's t	4.06	4.00	0.008	0.0254	0.00626	0.01207	Inf	Cohen's d	1.82	0.294	3.28

Note. H_a $\mu_{\text{Measure 1 - Measure 2}} > 0$

(b)

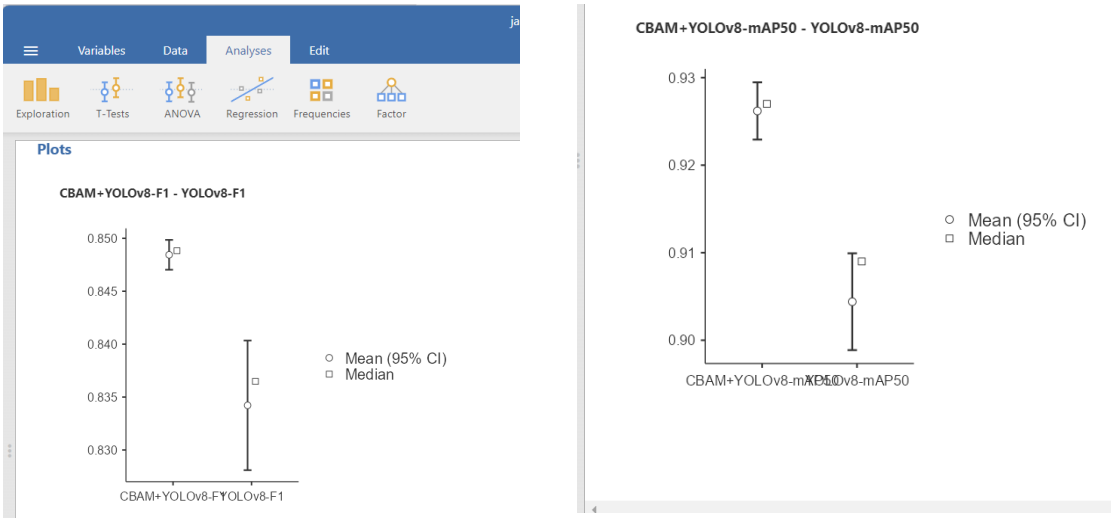
Descriptives

	N	Mean	Median	SD	SE
CBAM+YOLOv8-F1	5	0.848	0.849	0.00161	7.19e−4
YOLOv8-F1	5	0.834	0.836	0.00698	0.00312
CBAM+YOLOv8-mAP50	5	0.926	0.927	0.00373	0.00167
YOLOv8-mAP50	5	0.904	0.909	0.00631	0.00282
CBAM+YOLOv8-mAP95	5	0.602	0.605	0.01502	0.00672
YOLOv8-mAP95	5	0.577	0.577	0.01327	0.00594

(c)

Figure D4 (a,b,c). Comparative Results of Base YOLOv8 and CBAM Integrated Version

This figure compares the detection results between the baseline YOLOv8 model and the enhanced CBAM-integrated model. Improved localization precision and higher confidence levels can be observed in the CBAM variant, validating its superior performance in complex environments.



(a) (b)

Figure D5 (a,b). Statistical Analysis Output

This output shows the statistical results obtained from Jamovi, illustrating the significance testing between the baseline and improved model configurations. The paired sample t-test results confirm the statistical reliability of performance improvements due to CBAM integration.

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
Los Baños Campus
College of Computer Studies

Real-Time Object Overlap Detection in Complex Scenes Using YOLOv8 for Localization and CBAM for Feature Enhancement

JOHN MARTIN L. LAGAJINO, JOSHUA R. CASPILLO, ARGIE GARCIA

Researchers

The primary objective of this validation form is to assess the accuracy and reliability of the developed system in detecting broiler chickens under crowded and occluded conditions.
This evaluation aims to determine how effectively the **Modified YOLOv8 integrated with CBAM (Convolutional Block Attention Module)** enhances the model's detection performance compared to the standard YOLOv8 architecture.

Your professional judgment and honest assessment will help validate the system's capability and provide valuable insights for further refinement and improvement.

This questionnaire, referred to in our study as the Evaluation Tool, is designed to help us evaluate the system's overall performance in detecting broiler chickens. Your honest assessment and expert insights are essential for the validation and for the refinement of our model.

Here's how it works in three simple steps:

1. **Detection (YOLOv8):**
    - The model detects and localize chickens within an image frame, marking each detected chicken with a bounding box.

2. **Attention Mechanism (CBAM Integration):**
    - The CBAM module refines the model's attention by focusing on critical features, allowing better detection performance when chickens overlap or appear partially visible.

3. **Classification and Output:**

- The system displays the detection results by marking each chicken with a bounding box and confidence score, ensuring accurate counting and monitoring even in dense environments..

As the expert validator, your role is to observe the system's performance and verify the accuracy of the detected and localized broiler chickens within the generated bounding boxes.

Validation Procedure

The goal of this validation is to determine how accurately the system detects and localizes broiler chickens.
The process will be carried out in the following steps:

1. **Data Provision:**
   The research team will provide a set of images or short video clips where the developed system has automatically detected broiler chickens. Each detection is marked by a bounding box overlay.

2. **Expert Review:**
   You, as the expert validator, are asked to observe each detection output and confirm whether the bounding boxes correctly represent actual chickens in the scene. Your task is to evaluate the system's accuracy in identifying chickens, particularly in complex or crowded situations.

This validation will ultimately establish the degree of concordance between the model's prediction and your expert assessment.

Please be assured that all your responses will be kept strictly confidential and used solely for academic purposes.

VALIDATION TABLE

Instructions:

Performance Marking: Following your review, this validation table is provided for you to mark the model's performance on a per-image basis:

- **Predicted Count** — Number of objects detected by the model.
- 1**st Ground Truth** — The original ground truth count from the annotation.
- **Validator Ground Truth** — The ground truth count confirmed or corrected by a validator.

Sample	Image No.	Predicted Count	1st Ground Truth	Validator Ground Truth
1	3	66	54	106
2	7	67	56	106
3	13	69	54	106
4	16	61	50	107
5	31	62	62	109
6	45	49	42	54
7	55	57	40	56
8	89	62	45	72
9	92	62	49	72
10	94	58	43	72
11	106	66	47	71
12	108	64	56	71
13	117	72	50	73
14	126	69	62	79
15	133	63	57	79
16	135	64	58	79
17	147	60	55	77
18	148	59	50	76

19	153	69	57	76
20	168	37	28	36
21	202	35	27	44
22	220	34	25	41
23	222	37	40	41
24	223	33	23	41
25	224	33	21	41

26	234	35	22	40
27	240	38	24	41
28	253	54	57	87
29	319	50	52	86
30	320	47	41	86
31	321	46	47	86
32	331	55	34	86
33	334	51	33	86
34	367	76	78	237
35	368	68	80	237
36	387	55	34	97
37	406	66	55	85

38	407	65	53	85
39	413	56	48	89
40	416	53	48	91
41	418	57	44	91
42	421	53	50	89
43	424	54	52	89
44	426	54	48	90
45	427	55	48	93

46	429	55	44	94
47	431	61	47	101
48	433	61	51	101
49	438	61	51	99
50	441	59	47	99
51	447	61	43	106
52	450	63	49	104
53	452	88	90	106
54	477	94	89	146
55	483	92	69	151
56	490	74	70	161

57	506	52	41	167
58	512	53	17	176
59	513	51	52	176
60	522	43	41	168
61	524	48	45	168
62	531	53	59	168
63	532	47	49	168
64	535	16	16	18
65	536	16	16	18

66	538	17	16	17
67	542	17	15	18
68	544	17	14	18
69	560	17	5	25
70	575	14	4	23
71	577	17	18	23
72	581	20	20	25
73	582	21	20	25
74	585	19	19	25
75	589	20	16	25

76	593	19	14	25
77	594	18	13	25
78	597	15	12	21
79	600	9	9	10
80	602	9	9	10
81	611	10	9	10
82	612	9	9	10
83	614	9	9	10
84	617	9	8	10
85	621	9	8	10
86	628	9	8	11
87	629	9	8	11
88	634	9	9	11
89	639	8	9	11
90	644	9	9	11

91	645	9	8	11
92	648	10	6	12
93	656	9	9	13
94	660	9	9	13

95	662	10	8	13
96	671	9	8	13
97	677	11	6	13
98	684	10	10	15
99	693	11	11	17
100	697	11	11	17

Validator Comments

Observations:

- Most of the uncounted broiler varies in size and distance of the images
- Some partial parts or overlapping images are not detected.
- The ground truth is not the actual number of broiler in the images.

Suggestions:

1. Use segmentation instead of detection,
   - This lets the model output pixel-level masks, not just bounding boxes.
   - It can better handle overlapping or partially visible chickens, since masks can overlap naturally.

2. Add Spatial Context Attention or Deformable Convolutions
   - To further help recognize incomplete shapes:
   - Integrate Deformable Convolutional Blocks (DCN) → makes the receptive field adapt to irregular body shapes.
   - Use Body-Aware Attention Module (BAAM) instead of CBAM, focusing on contour continuity.

3. Data Augmentation for Occlusion
   - Simulate hidden chickens during training:
   - Use Copy-Paste Augmentation (cut and paste chickens with overlaps).
   - Use Mosaic or MixUp Augmentation.
   - Randomly crop chickens to mimic partial visibility.

4. Use the following METRICS

- mAP (mean average precision)
- Recall (especially for occluded chickens)
- False negatives (missed detections)

5. Rerun the T-test

Note:
I commend the research because the project shows good research methodology by using structured experimentation through creating and experimenting 7 model variants by using multiple architectural variations, not just the default yolo version 8 model. Your research opens more future research challenges and works. If you have time, kindly consider my suggestions to further increase the accuracy of your system.

I hereby certify that the evaluation provided above is based on my professional expertise and unbiased observation of the system's detection results.

_____
Signature Over Printed Name
Date of Validation: 10-18-25

Republic of the Philippines

Laguna State Polytechnic University

Province of Laguna

Los Baños Campus

College of Computer Studies

I, the authorized caretaker of the poultry farm from which the dataset for the research titled **"Real-Time Object Overlap Detection in Complex Scenes Using YOLOv8 for Localization and CBAM for Feature Enhancement"** was collected, hereby confirm that the images were taken in our farm facility with my knowledge and permission. A total of approximately **1,000 images** were collected for research use. I further acknowledge that the developed detection model used in the study operates within an approximate prediction range of **150 cm**, and that the collected data will be used solely for academic and research purposes.

Mark Philip Cortez Jamito

Signature Over Printed Name

APPENDIX E

Sample Input/Output

This section provides five sample images that compares the detection outputs of the baseline YOLOv8 and the CBAM-integrated YOLOv8 models. In each image, the green bounding boxes denote the localized broiler chickens that have been identified by the model.



(a)



CBAM: 38

(b)



YOLOv8: 30

(c)

Figure E1 (a,b,c). Sample 1 Input/Output



(a)



CBAM: 52 YOLOv8: 42

(b) (c)

Figure E2 (a,b,c). Sample 2 Input/Output

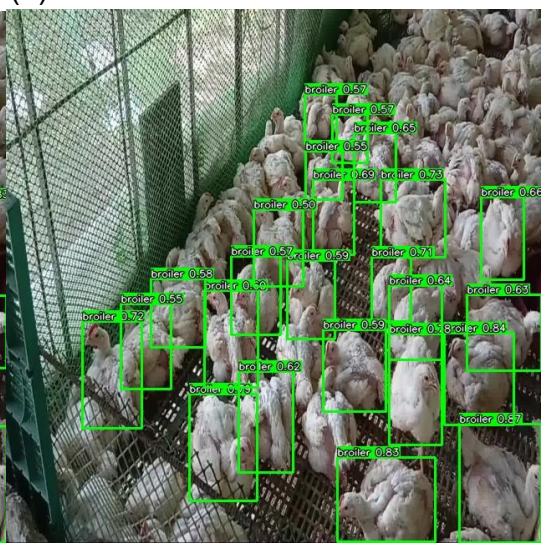


(a)



CBAM: 29

(b)



YOLOv8: 24

(c)

Figure E3 (a,b,c). Sample 3 Input/Output



(a)



CBAM: 34 YOLOv8: 27

(b) (c)

Figure E4 (a,b,c). Sample 4 Input/Output



(a)



CBAM: 73 YOLOv8: 61

(b) (c)

Figure E5 (a,b,c). Sample 5 Input/Output

APPENDIX F

Comments Section

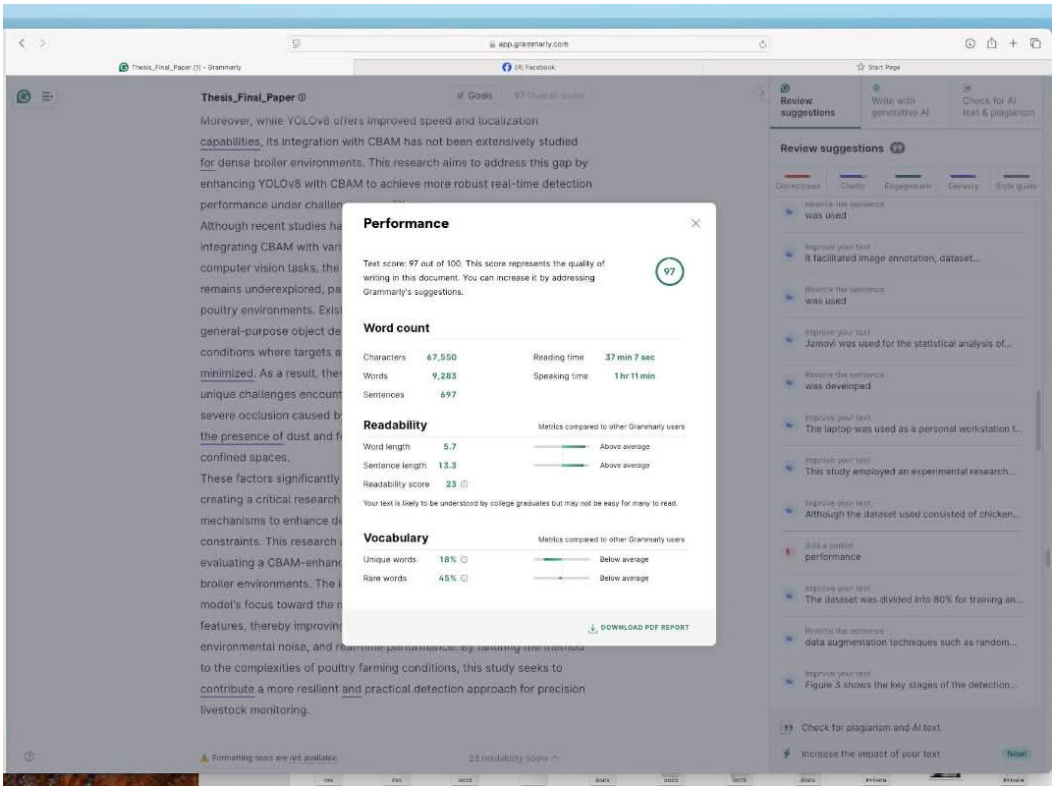
MATRIX FOR REVISIONS

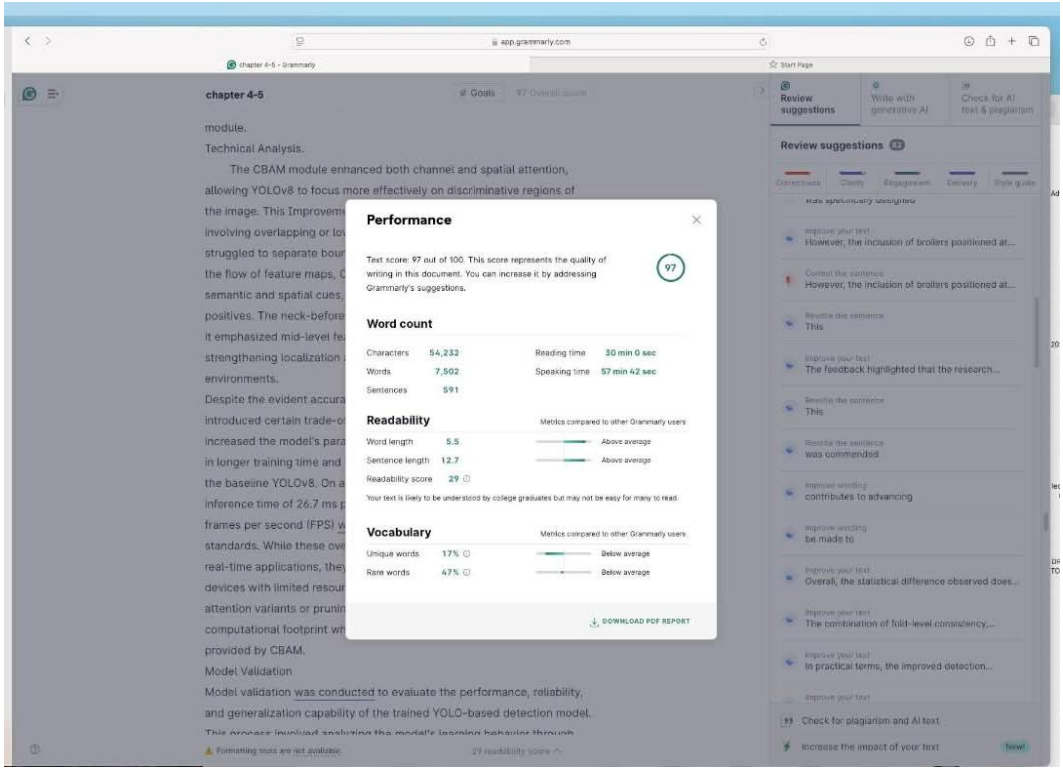
Commentors	Consultants	Comments	Page No.	Actions	Page No.
Ms. Myra V. Paicaglino	Technical Critic	Fix margin size	All pages	Fix the margin size to the required size.	All pages
Ms. Myra V. Paicaglino	Technical Critic	Fill pages with words.	All pages	Filled the pages with more discussion to avoid big spaces.	All pages
Ms. Myra V. Paicaglino	Technical Critic	Fix table format	22, 26, 38,47, 60, 76, 77, 78, 80, 88	Fixed the table according to the required format.	22, 26, 38,47, 60, 76, 77, 78, 80, 88
Mr. Gene Marck B. Catedrilla	Adviser	Inconsistent spacing in several sections. Abstract	Abstract	Arranged the inconsistent spacing	Abstract
Mr. Gene Marck B. Catedrilla	Adviser	Inconsistent spacing in several sections. RRL pages	20 - 40	Arranged the inconsistent spacing	20-40
Mr. Gene Marck B. Catedrilla	Adviser	Inconsistent spacing in several sections. Appendix	136 - 163	Arranged the inconsistent spacing	136 - 163
Mr. Gene Marck B. Catedrilla	Adviser	Repeated sentences in some RRL paragraphs.	10 - 21	Removed the sentences that repeated.	10 - 21
Mr. Gene Marck B. Catedrilla	Adviser	Add small subsection	43	Added “Data Acquisition" as subheading	43
Mr. Gene Marck B. Catedrilla	Adviser	Need clearer explanation of dataset source	Appendix	Done what was required.	Appendix
Mr. Gene Marck B. Catedrilla	Adviser	Figures needed clearer captions.	24, 32, 39, 40, 41, 48,	Made more clear and concise figure captions.	24, 32, 39, 40, 41, 48,

		Some figures are too short	49, 50, 57, 58, 61, 62, 63, 65, 66, 68, 69, 71, 72, 73, 74, 78, 80, 83, 85		49, 50, 57, 58, 61, 62, 63, 65, 66, 68, 69, 71, 72, 73, 74, 78, 80, 83, 85

APPENDIX G

Turnitin and Grammarly Results


app.grammarly.com
Thesis_Final_Paper
Goals
Performance
Text score: 97 out of 100. This score represents the quality of writing in this document. You can increase it by addressing Grammarly's suggestions.
97
Word count
Characters 67,550
Words 9,283
Sentences 697
Reading time 37 min 7 sec
Speaking time 1 hr 11 min
Readability
Metrics compared to other Grammarly users
Word length 5.7 Above average
Sentence length 13.3 Above average
Readability score 23
Your text is likely to be understood by college graduates but may not be easy for many to read.
Vocabulary
Metrics compared to other Grammarly users
Unique words 18% Below average
Rare words 45% Below average
DOWNLOAD PDF REPORT
Review suggestions
Write with generative AI
Check for AI text & plagiarism
Check for plagiarism and AI text
Increase the impact of your text


app.grammarly.com
chapter 4-5
Goals
Performance
Text score: 97 out of 100. This score represents the quality of writing in this document. You can increase it by addressing Grammarly's suggestions.
97
Word count
Characters 54,232
Words 7,502
Sentences 591
Reading time 30 min 0 sec
Speaking time 57 min 42 sec
Readability
Metrics compared to other Grammarly users
Word length 5.5 Above average
Sentence length 12.7 Above average
Readability score 29
Your text is likely to be understood by college graduates but may not be easy for many to read.
Vocabulary
Metrics compared to other Grammarly users
Unique words 17% Below average
Rare words 47% Below average
DOWNLOAD PDF REPORT
Review suggestions
Write with generative AI
Check for AI text & plagiarism
Check for plagiarism and AI text
Increase the impact of your text

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **182** Not Cited or Quoted 10%
  Matches with neither in-text citation nor quotation marks
- **18** Missing Quotations 1%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

9% Internet sources
5% Publications
7% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

APPENDIX H

User's Guide



SCAN

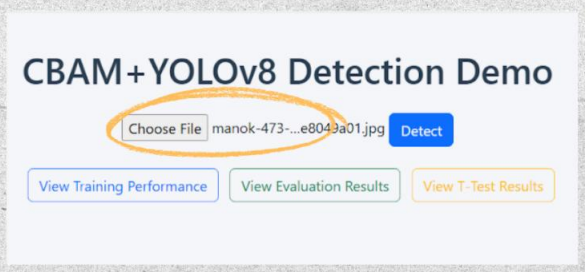
REAL-TIME OBJECT OVERLAP DETECTION IN COMPLEX SCENES USING YOLOv8 FOR LOCALIZATION AND CBAM FOR FEATURE ENHANCEMENT

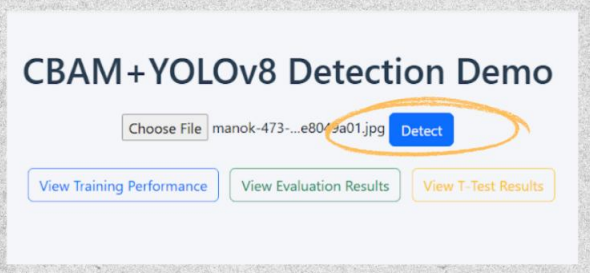
HOW TO USE THE SYSTEM

STEP 1

Main Page

The main interface of the prototype system is designed to require users to upload an image file as the initial input.





STEP 2

Detection

The user will now then click the Detect button to continue with the image detection

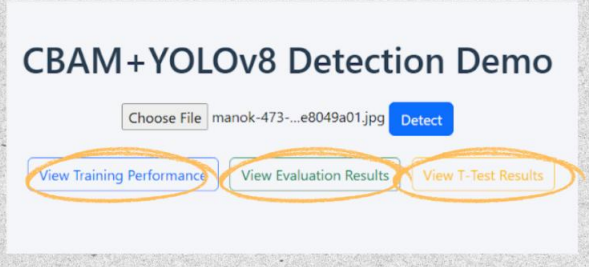




STEP 3

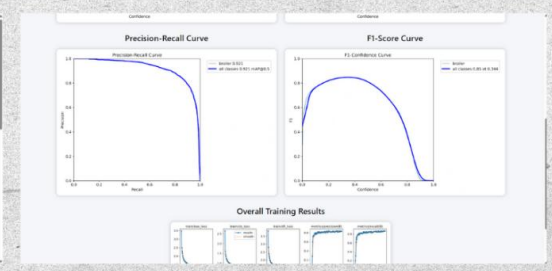
Results

Once step 2 is done, the prototype can provide 3 results based on what is needed. This includes: View Training Performance, View Evaluation Results and View T-Test Results



View Training Performance



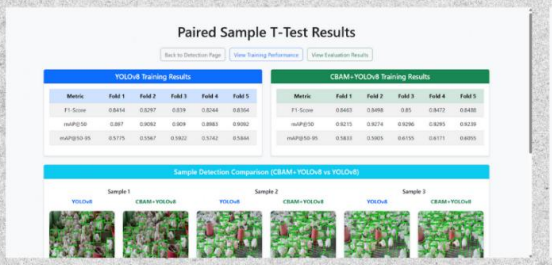


View Evaluation Results





View Evaluation Results





APPENDIX I

Publishable Paper (IEEE format)

REAL-TIME OBJECT OVERLAP DETECTION IN COMPLEX SCENES USING YOLOv8 FOR LOCALIZATION AND CBAM FOR FEATURE ENHANCEMENT

Joshua R. Caspillo, Argie Garcia, John Martin L. Lagajino
College of Computer Studies (CCS), Laguna State Polytechnic University, Los Baños, Laguna, Philippines

Article Info

Article history:

Received by November 12, 2024
Revised by Unknown 0, 2024
Accepted by January 0, 2024

Keywords:
YOLOv8
CBAM
Object detection
Attention mechanism
Deep learning
Real-time localization CNN

ABSTRACT

This paper presents an improved YOLOv8-based framework for real-time object detection in crowded and complex visual scenes through the integration of the Convolutional Block Attention Module (CBAM). The study aims to enhance feature discrimination and localization accuracy, particularly in environments with overlapping or occluded objects. Multiple configurations were implemented by embedding CBAM in different positions within the YOLOv8 backbone and neck. The models were evaluated using metrics such as Precision, Recall, F1-score, and mean Average Precision (mAP). A statistical paired sample t-test confirmed that the CBAM-integrated models significantly outperformed the baseline YOLOv8 in detection performance. Results demonstrate that attention-based feature enhancement is effective in improving YOLOv8's localization capability while maintaining real-time processing speed, proving suitable for low-power devices and resource-limited environments.

Corresponding Author:

I. Introduction

The rapid advancement of computer vision and deep learning has revolutionized object detection tasks, enabling real-time recognition and localization across domains such as healthcare, transportation, and agriculture. In poultry monitoring, automated detection systems have become critical for efficient health and welfare assessment. YOLO (You Only Look Once) is among the most popular object detection architectures due to its high accuracy and low computational cost, making it ideal for real-time applications.

Despite YOLO's efficiency, challenges remain when detecting objects in crowded or occluded scenes where chickens often overlap. The Convolutional Block Attention Module (CBAM) introduces a dual attention mechanism that sequentially applies channel and spatial attention, allowing the network to emphasize discriminative features while suppressing background noise. Integrating CBAM with YOLOv8 improves the model's ability to detect fine-grained and overlapping details.

This research aims to improve YOLOv8 by embedding CBAM into different parts of its architecture and evaluating how placement affects detection performance. The proposed model contributes to the growing body of research in precision agriculture, particularly smart poultry monitoring aligned with Sustainable Development Goals (SDGs) 2, 9, and 12.

II. METHODOLOGY

The study follows an experimental design to assess the effect of CBAM integration on YOLOv8's performance.

A. Data Collection and Preprocessing

The dataset was collected from a poultry farm in Indang, Cavite, using a mobile phone camera under natural conditions. Images were annotated with bounding boxes using Roboflow, then split into 800 training and 200 validation samples. All images were resized to 640×640 pixels for compatibility with YOLOv8 input layers.

B. Model Development

The proposed framework modifies the YOLOv8 architecture by integrating CBAM at various locations in the backbone and neck. Seven configurations were tested to identify the optimal placement for enhancing detection accuracy and maintaining efficiency.

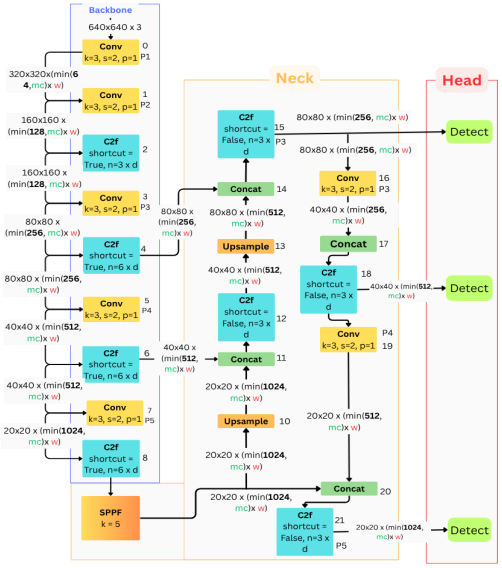


Figure 1. YOLOV8 ARCHITECTURE

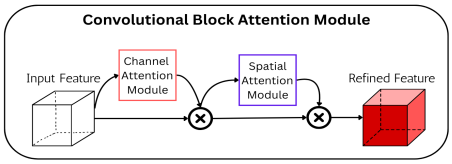


Figure. 2. CBAM OVERVIEW

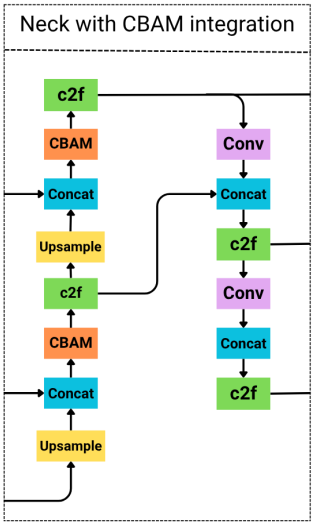


Figure 3. YOLOV8 WITH CBAM OVERVIEW

Each model was trained on Paperspace and Google Colab using GPU acceleration. Key software tools included Visual Studio Code for coding, GitHub for version control, Roboflow for dataset preparation, and Jamovi for statistical analysis.

C. Evaluation Metrics

Performance was assessed using Precision, Recall, F1-score, and mAP (mean Average Precision). The statistical significance of performance improvements was determined through a paired sample t-test, comparing the baseline YOLOv8 and the CBAM-augmented versions.

III. RESULTS AND DISCUSSION

A. Model Performance

Results showed that the YOLOv8+CBAM model achieved substantial improvements over the baseline YOLOv8.

Table 1. Comparison of YOLOv8 and CBAM + YOLOv8 Performance Metrics Across Folds

Fold	CBAM F1-Score	YOLO F1-Score	CBAM mAP 50	YOLO mAP 50	CBAM mAP50-95	YOLO mAP 50-95
1	0.8463	0.8414	0.9215	0.8970	0.5833	0.5775

2	0.8498	0.8297	0.9274	0.9092	0.5905	0.5567
3	0.8500	0.8390	0.9296	0.9090	0.6155	0.5922
4	0.8472	0.8244	0.9295	0.8983	0.6171	0.5742
5	0.8488	0.8364	0.9239	0.9092	0.6055	0.5844

Table 2. T-Test Comparing YOLOv8 and CBAM + YOLOv8 Performance Metrics

Paired Samples T-Test

		Statistics	df	p
CBAM+YOLOv8-F1	YOLOv8-F1	4.41	40	0.006
CBAM+YOLOv8-mAP50	YOLOV8-mAP50	7.63	40	<.001
CBAM+YOLOv8-mAP95	YOLOV8-mAP95	4.06	40	0.008

Note. $H_{0\ \mu\ measure\ 1\ -\ measure\ 2} > 0$

These improvements confirm CBAM's capability to enhance feature learning and localization in complex visual environments.



(a) (b)

Figure 4 (a,b). MODEL COMPARISON BETWEEN YOLOV8 AND INTEGRATED YOLOV8 WITH CBAM

B. Statistical Validation

A paired sample t-test revealed that the performance difference between the baseline and CBAM-augmented YOLOv8 was statistically significant ($p < 0.05$). This result validates that the improvement was not due to random variation but a direct consequence of the attention mechanism's integration.

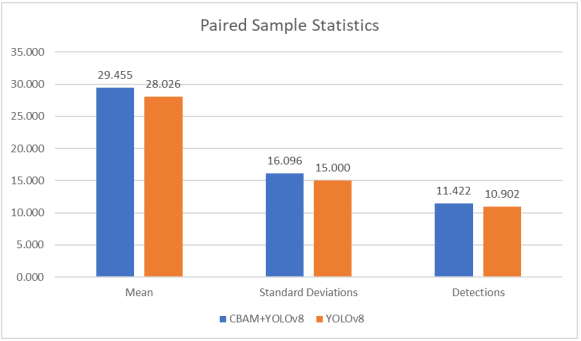


Figure 5. YOLOV8+CBAM AND YOLOV8 PAIRED SAMPLE RESULTS

C. Observations

Integrating CBAM improved the model's ability to detect overlapping chickens and small objects under poor lighting conditions. The optimal configuration was found when CBAM was positioned before the feature fusion layer in the neck. The model also maintained real-time inference speed, supporting deployment in edge AI scenarios.

IV. CONCLUSION AND RECOMMENDATION

This study demonstrated that embedding the Convolutional Block Attention Module into YOLOv8 improves detection accuracy in complex and crowded environments. The CBAM-enhanced model achieved higher precision, recall, and mAP compared to the baseline.

Future work should focus on extending the system to real-time video detection, incorporating malnourishment classification, and deploying the model using higher-quality cameras or CCTV systems to enhance performance in live farm environments.

ACKNOWLEDGMENT

The researchers express their deepest gratitude to **Mr. Gene Marck B. Catedrilla**, thesis adviser, and to **Ms. Maribel Cardona, Ms. Carolina R. Joval, and Ms. Myra V.**

Paicaglino, for their guidance and support. The researchers also thank **Laguna State Polytechnic University – Los Baños Campus** for providing the resources and academic environment that enabled this work.

REFERENCES

[1] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018.
[2] S. Woo, J. Park, J.-Y. Lee, and I. Kweon, "CBAM: Convolutional Block Attention Module," in *Proc. ECCV*, 2018.
[3] Q. Tong et al., "Edge AI-enabled Chicken Health Detection based on Enhanced FCOS-Lite and Knowledge Distillation," *Computers and Electronics in Agriculture*, 2024.
[4] N. Garshasebi, "Improving Small Object Detection in Remote Sensing with YOLO11-CBAM," *Earth Science Journal*, 2025.
[5] M. Lv and W. Su, "YOLOv5-CBAM-C3TR: An Optimized Model Based on Transformer Module and Attention Mechanism," *Frontiers in Plant Science*, 2024.
[6] T. Jiang et al., "An Improved YOLOv5s Algorithm for Object Detection with Attention Mechanism," *Electronics*, 2022.
[7] W. Elmessery et al., "YOLO-Based Model for Automatic Detection of Broiler Pathological Phenomena," *Applied Sciences*, 2023.
[8] A. Wang et al., "Application of YOLOv6 Combining CBAM and CIoU in Forest Fire Detection," *Sensors*, 2023.
[9] S. Praveen and Y. Jung, "CBAM-STN-TPS-YOLO: Enhancing Agricultural Object Detection Through Spatially Adaptive Attention Mechanisms," *arXiv preprint*, 2025.
[10] Z. Wu, J. Yang, H. Zhang, and C. Fang, "Enhanced Methodology for Caged Chicken Counting Based on YOLOv8," *AgriEngineering*, 2025.

APPENDIX J

Curriculum Vitae



Joshua R. Caspillo

Contact

+639555530119

joshuacaspillo12@gmail.com

San Antonio Los Banos
Laguna

Summary

Bachelor of Science in Computer Science student with a strong focus on software and web development. Skilled in Python, Java, SQL, and HTML, with a particular interest in back-end development and system functionality. Experienced in developing efficient, scalable solutions and passionate about building reliable applications that solve real-world problems.

Skills

- Programming skills
  - Python
  - Java
  - SQL
  - HTML
  - CSS
  - PHP
  - IoT
- Microsoft Application
  - Word
  - Excel
- Editing
  - Canva
  - Figma

Education

- **Laguna State Polytechnic University - Los Baños Campus** 2022-2026
  Bachelor of Science in Computer Science Major in Intelligent System
- **Christian School International** 2020 - 2022
  Science, Technology, Engineering, and Mathematics (STEM)
- **Los Baños National Highschool - Batong Malake** 2016-2020

Projects

E-commerce September 2023

A Python project that recreates the e-commerce platform with the use of XAMPP as a database and Python for backend development.

Tamagochi June 2024-July2024

A Java project that replicates the real Tamagotchi game that can react to hunger, moods and attention.

LinkinPark April 2024-May 2024

A web-based application that allows customers to book and reserve parking spaces in specific establishments through online transactions.

Automatoes October 2024-December 2024

A web-based system that enables users to identify tomato plant diseases by scanning the fruit through a camera interface. The system uses image processing and machine learning techniques to analyze visual features and provide accurate disease classification.

Push-up Bot June 2025-July 2025

The project involves a human-like robot capable of replicating physical movements such as push-ups. It is a sound-activated system that uses a sound sensor to detect audio cues, triggering the bot's motion accordingly.



Argie Garcia

Contact

+639762259848

rgie0606@gmail.com

Centro, Lalakay, Los Banos, Laguna

Summary

Bachelor of Science in Computer Science student with a strong focus on software and web development. Skilled in Python, Java, SQL, and HTML, with a particular interest in front-end development and system functionality. Experienced in developing efficient, scalable solutions and passionate about building reliable applications that solve real-world problems.

Skills

- Programming skills
  - Python
  - Java
  - Sql
  - HTML
- Basic Microsoft Application
- Editing
  - Canva
  - Figma

Education

Laguna State Polytechnic University - Los Baños Campus 2022-2026

Bachelor of Science in Computer Science Major in Intelligent System
Consistent Dean's Lister

Trace College 2020 - 2022

Accountancy, Business and Management (ABM)

Los Baños National Highschool - Batong Malake 2015-2019

Projects

LinkinPark April 2024-May 2024

A web-based application that allows customers to book and reserve parking spaces in specific establishments through online transactions.

IRIS October 2024-December 2024

IRIS is an AI language model that understands and generates human-like text. It can assist with writing, coding, research, and conversations across various topics. Powered by advanced machine learning, it adapts to different contexts and communication styles.

Sumo-Bot June 2025-July 2025

SumoBot is a small autonomous robot designed to compete in sumo-style matches, where two robots try to push each other out of a circular ring. It uses sensors, motors, and programmed strategies to detect opponents and stay within the ring.



John Martin L. Lagajino

Contact

+123-456-7890

johnmartinlagajino@gmail.com

Brgy. Dila Bay Laguna

Summary

Bachelor of Science in Computer Science student with a strong focus on software and web development. Skilled in Python, Java, SQL, and HTML, with a particular interest in back-end development and system functionality. Experienced in developing efficient, scalable solutions and passionate about building reliable applications that solve real-world problems.

Skills

- Programming skills
  - Python
  - Java
  - Sql
  - HTML
- Basic Microsoft Application
- Editing
  - Canva
  - Figma

Education

Laguna State Polytechnic University - Los Baños Campus 2022-2026

Bachelor of Science in Computer Science Major in Intelligent System
Consistent Dean's Lister

Los Baños Senior High School 2020 - 2022

Science, Technology, Engineering, and Mathematics (STEM)

Los Baños National Highschool - Batong Malake 2016-2020

Projects

LinkinPark April 2024-May 2024

A web-based application that allows customers to book and reserve parking spaces in specific establishments through online transactions.

Automatoes October 2024-December 2024

A web-based system that enables users to identify tomato plant diseases by scanning the fruit through a camera interface. The system uses image processing and machine learning techniques to analyze visual features and provide accurate disease classification.

PeriBot June 2025-July 2025

A rover-like robot capable of measuring distances and mapping the dimensions of a room as it travels. The system utilizes sensors to calculate and display spatial measurements in real time, demonstrating skills in robotics, programming, and hardware integration.